

Contents

第十六册简介: Reasoning Model 专题	6
本书定位	6
为什么需要这本书	6
适合读者	7
学完后应具备的能力	7
阅读建议	7
与其他书的关系	7
第十六册: Reasoning Model 专题	7
本书定位	7
章节文件	7
第一章: Reasoning 总览	7
0. 本讲资料边界与第二轮精修口径	8
1.1 什么是 Reasoning	8
1.2 Reasoning Model 和 Chat Model 的区别	8
1.3 为什么 Next-Token Prediction 也能产生推理	9
1.3.1 关键公式与 Reasoning 总览速查	9
1.4 Chain-of-Thought	11
1.5 Test-Time Compute	11
1.6 Self-Consistency	11
1.7 Verifier	12
1.8 Process Supervision	12
1.9 Search 和 Tree-of-Thought	13
1.10 数学推理	13
1.11 代码推理	13
1.12 Reasoning 评估	14
1.13 Reasoning 的安全和局限	14
1.14 训练时计算和推理时计算的权衡	14
1.15 Reasoning 项目路线	15
1.16 最小可运行 reasoning 成本 / 正确率审计 demo	15
1.17 面试官会怎么问	18
问题一: Reasoning model 和 chat model 有什么区别?	18
问题二: 为什么 test-time compute 重要?	18
问题三: CoT 一定代表模型真的推理吗?	18
问题四: Verifier 在 reasoning 中有什么作用?	18
问题五: 如何评估 reasoning 能力?	18
1.18 本章总结	18
第二章: Chain-of-Thought	19
0. 本讲资料边界与第二轮精修口径	19
2.1 CoT 是什么	19
2.1.1 关键公式与 CoT 审计指标速查	20
2.2 为什么 CoT 可能有效	21
2.2.1 增加中间计算空间	21
2.2.2 分解复杂问题	21
2.2.3 匹配训练语料格式	22
2.2.4 便于后续验证	22
2.3 Few-shot CoT	22
2.4 Zero-shot CoT	22
2.5 CoT 和 Scratchpad	23
2.6 CoT 是否等于真实推理	23
2.7 CoT 何时有效	23
2.8 长 CoT 是否一定更好	24

2.9 CoT 数据如何构造	24
2.10 CoT 和最终答案格式	24
2.11 Hidden CoT 和可解释性	25
2.12 CoT 与 Verifier 的关系	25
2.13 CoT 与工具使用	25
2.14 常见 CoT 失败模式	25
2.15 最小可运行 CoT 质量 / 成本审计 demo	26
2.16 面试官会怎么问	29
问题一: CoT 为什么能提升推理能力?	29
问题二: Zero-shot CoT 和 few-shot CoT 有什么区别?	29
问题三: CoT 是否代表模型真实推理过程?	29
问题四: 什么时候不应该使用长 CoT?	29
问题五: 如何构造高质量 CoT 数据?	30
2.17 小练习	30
2.18 本章总结	30
第三章: Self-Consistency 与采样	30
0. 本讲资料边界与第二轮精修口径	30
3.1 为什么需要多路径采样	31
3.2 Self-Consistency 的基本流程	31
3.2.1 关键公式与 Self-Consistency 指标速查	31
3.3 Greedy Decoding 为什么不适合 self-consistency	33
3.4 Temperature	33
3.5 Top-p 和 Top-k	33
3.6 答案抽取	34
3.7 答案标准化	34
3.8 Majority Vote	34
3.9 加权投票	35
3.10 Pass@k	35
3.11 成本和质量曲线	35
3.12 什么时候 self-consistency 有效	36
3.13 多样性和正确性的平衡	36
3.14 Self-Consistency 和 Verifier 结合	37
3.15 一个完整实验设计	37
3.16 最小可运行 self-consistency / 采样审计 demo	37
3.17 面试官会怎么问	41
问题一: Self-consistency 是什么?	41
问题二: 为什么 self-consistency 需要 sampling?	42
问题三: pass@k 和 majority vote 有什么区别?	42
问题四: Self-consistency 的成本问题怎么处理?	42
问题五: Self-consistency 为什么可能失败?	42
3.18 小练习	42
3.19 本章总结	42
第四章: Verifier 与 Reward Model	43
0. 本讲资料边界与第二轮精修口径	43
4.1 为什么需要 Verifier	43
4.1.1 关键公式与 Verifier 指标速查	43
4.2 Outcome Verifier	45
4.3 Process Verifier	45
4.4 Reward Model 和 Verifier 的关系	46
4.5 生成-验证范式	46
4.6 Reranking	47
4.7 Pairwise Ranking	47
4.8 Pointwise 和 Listwise	47
4.9 Verifier 训练数据	48

4.10 Verifier 的偏差	48
4.11 Programmatic Verifier	48
4.12 Verifier 与 Search	49
4.13 ORM 和 PRM	49
4.14 评估 Verifier	49
4.15 工程使用建议	49
4.16 最小可运行 verifier rerank 审计 demo	50
4.17 面试官会怎么问	56
问题一: Verifier 和 generator 的关系是什么?	56
问题二: Outcome verifier 和 process verifier 有什么区别?	56
问题三: Reward model 和 verifier 有什么区别?	56
问题四: Verifier 为什么可能被 reward hacking?	56
问题五: 代码任务中最好的 verifier 是什么?	56
4.18 小练习	56
4.19 本章总结	56
第五章: Process Supervision	57
0. 本讲资料边界与第二轮精修口径	57
5.1 为什么需要过程监督	57
5.1.1 关键公式与 Process Supervision 指标速查	57
5.2 Outcome Supervision 的局限	59
5.3 什么是推理步骤	59
5.4 过程标注格式	60
5.5 PRM: Process Reward Model	60
5.6 正确步骤和有用步骤	60
5.7 错误步骤定位	61
5.8 标注成本	61
5.9 自动过程监督	61
5.10 Process Supervision 与 Search	62
5.11 Process Supervision 与 RL	62
5.12 过程监督的风险	62
5.13 过程监督和可解释性	63
5.14 数据集设计建议	63
5.15 评估 Process Supervision	63
5.16 最小可运行 process supervision / PRM 审计 demo	63
5.17 面试官会怎么问	66
问题一: Process supervision 是什么?	66
问题二: 它和 outcome supervision 有什么区别?	66
问题三: PRM 有什么作用?	67
问题四: 过程监督为什么成本高?	67
问题五: 过程监督有什么风险?	67
5.18 小练习	67
5.19 本章总结	67
第六章: Search 与 Tree-of-Thought	67
0. 本讲资料边界与第二轮精修口径	68
6.1 为什么需要 Search	68
6.1.1 关键公式与 Search / ToT 指标速查	68
6.2 状态、动作和转移	70
6.3 Tree-of-Thought	70
6.4 Beam Search	70
6.5 Best-First Search	71
6.6 MCTS 的直觉	71
6.7 Verifier 在搜索中的作用	71
6.8 剪枝	72
6.9 搜索成本	72

6.10 Search 和 Self-Consistency 的区别	72
6.11 Search 和工具使用	72
6.12 常见失败模式	73
6.13 一个简单 ToT 伪代码	73
6.14 什么时候值得用 Search	73
6.15 最小可运行 search / ToT / MCTS 审计 demo	74
6.16 面试题: Tree-of-Thought 相比 CoT 的优势是什么	76
6.17 面试题: Beam Search 和 MCTS 的区别	76
6.18 小练习	76
6.19 本章小结	77
第七章: Test-Time Compute Scaling	77
0. 本讲资料边界与第二轮精修口径	77
7.1 什么是 Test-Time Compute Scaling	77
7.1.1 关键公式与 TTC Scaling 指标速查	77
7.2 为什么推理时计算有用	79
7.3 常见推理时扩展方式	79
7.4 Best-of-N	79
7.5 Self-Consistency	79
7.6 Verifier Reranking	80
7.7 Adaptive Compute	80
7.8 置信度估计	80
7.9 Latency Trade-off	80
7.10 Cost Trade-off	81
7.11 Budget Allocation	81
7.12 Diminishing Returns	81
7.13 工程系统设计	81
7.14 常见失败模式	82
7.15 最小可运行 TTC scaling / 动态预算审计 demo	82
7.16 面试题: 什么是 Test-Time Compute Scaling	85
7.17 面试题: 如何设计 Adaptive Compute	86
7.18 小练习	86
7.19 本章小结	86
第八章: 数学推理训练	86
0. 本讲资料边界与第二轮精修口径	86
8.1 为什么数学适合训练 Reasoning	86
8.1.1 关键公式与数学推理训练指标速查	87
8.2 数学数据类型	89
8.3 最终答案监督的局限	89
8.4 解题步骤数据	89
8.5 合成题	90
8.6 难度 Curriculum	90
8.7 Verifier 在数学训练中的作用	90
8.8 自动验证和符号计算	91
8.9 多解法训练	91
8.10 错误解法和纠错数据	91
8.11 RL 在数学推理中的使用	91
8.12 数学推理评估	92
8.13 数据泄漏风险	92
8.14 常见失败模式	92
8.15 最小可运行数学推理训练数据审计 demo	92
8.16 面试题: 如何构造数学推理训练数据	97
8.17 面试题: 数学 verifier 有什么用	97
8.18 小练习	97
8.19 本章小结	97

第九章：代码推理与执行反馈	97
0. 本讲资料边界与第二轮精修口径	98
9.1 为什么代码适合训练 Reasoning	98
9.1.1 关键公式与代码执行反馈指标速查	98
9.2 代码推理的基本闭环	99
9.3 执行反馈有哪些类型	100
9.4 单元测试作为 Verifier	100
9.5 Self-Debug	100
9.6 程序合成	101
9.7 搜索和代码修复	101
9.8 训练数据构造	101
9.9 执行反馈作为 Reward	102
9.10 隐藏测试和泛化	102
9.11 安全沙箱	102
9.12 多语言和环境问题	102
9.13 常见失败模式	103
9.14 评估指标	103
9.15 最小可运行代码推理 / 执行反馈审计 demo	103
9.16 面试题：执行反馈为什么重要	108
9.17 面试题：如何设计代码 self-debug 系统	108
9.18 小练习	108
9.19 本章小结	108
第十章：Reasoning 评估	109
0. 本讲资料边界与第二轮精修口径	109
10.1 Reasoning 评估为什么难	109
10.1.1 关键公式与 Reasoning 评估指标速查	109
10.2 数学 Benchmark	111
10.3 代码 Benchmark	111
10.4 最终答案评估	112
10.5 过程评估	112
10.6 First Error Evaluation	112
10.7 污染检测	113
10.8 泛化评估	113
10.9 鲁棒性评估	113
10.10 推理成本评估	113
10.11 多次采样稳定性	114
10.12 LLM Judge 的使用	114
10.13 人工评估	114
10.14 常见评估陷阱	115
10.15 一个完整评估报告应该包含什么	115
10.16 最小可运行 reasoning 评估审计 demo	115
10.17 面试题：如何评估一个 Reasoning Model	118
10.18 面试题：为什么不能只看 Benchmark 分数	119
10.19 小练习	119
10.20 本章小结	119
第十一章：Reasoning 安全与局限	119
0. 本讲资料边界与第二轮精修口径	119
11.1 推理能力为什么会带来风险	119
11.1.1 关键公式与 Reasoning 安全指标速查	120
11.2 Reasoning 安全风险的分层	122
11.3 伪推理	122
11.4 过度自信	123
11.5 Chain-of-Thought 隐私与展示边界	123
11.6 CoT 不忠实与不可解释推理	124

11.7 长链条错误传播	124
11.8 Reward Hacking	124
11.9 工具误用与 Excessive Agency	125
11.10 推理能力与滥用风险	125
11.11 高风险场景边界	125
11.12 Reasoning 安全评估	126
11.13 缓解策略	126
11.14 最小可运行 reasoning 安全审计 demo	127
11.15 面试题：为什么 CoT 不一定应该直接展示	131
11.16 面试题：Reasoning Model 有哪些安全风险	131
11.17 小练习	132
11.18 本章小结	132

第十二章：Reasoning 面试题 132

0. 本讲资料边界与第二轮精修口径	132
12.1 Reasoning 面试的评分框架	132
12.2 关键公式速查	133
12.3 什么是 Reasoning Model	135
12.4 CoT 为什么有效	135
12.5 Zero-shot CoT 和 Few-shot CoT 的区别	135
12.6 Self-Consistency 是什么	135
12.7 Verifier 和 Reward Model 有什么区别	136
12.8 Outcome Supervision 和 Process Supervision 的区别	136
12.9 PRM 是什么	136
12.10 Tree-of-Thought 相比 CoT 的优势是什么	136
12.11 Beam Search 和 MCTS 如何比较	136
12.12 Test-Time Compute Scaling 是什么	137
12.13 Adaptive Compute 怎么设计	137
12.14 如何训练数学推理模型	137
12.15 数学 Verifier 有什么用	137
12.16 如何训练代码推理模型	138
12.17 Self-Debug 系统怎么做	138
12.18 如何评估 Reasoning Model	138
12.19 为什么不能只看 Benchmark 分数	138
12.20 Reasoning Model 有哪些安全风险	138
12.21 为什么完整 CoT 不一定应该展示给用户	139
12.22 设计一个 Reasoning 系统	139
12.23 面试中的高分表达模板	139
12.24 最小可运行 reasoning 面试复盘 demo	139
12.25 小练习	142
12.26 本章小结	143

第十六册简介：Reasoning Model 专题

本书定位

第十六册专门讨论 reasoning model、test-time compute、数学推理、代码推理、verifier、搜索、过程监督和推理评估。

为什么需要这本书

当前大模型能力竞争的重要方向之一是推理能力。模型不仅要会聊天，还要能解决数学、代码、规划、多步任务和复杂问题。Reasoning 涉及训练数据、推理策略、搜索、验证器和计算预算。

适合读者

1. 想准备 reasoning、math、code、test-time compute 方向面试的读者。
2. 想理解 CoT、self-consistency、verifier、process supervision 的人。
3. 想做数学推理增强或代码执行反馈项目的人。

学完后应具备的能力

1. 能解释 CoT、self-consistency、verifier、ORM/PRM 和过程监督。
2. 能理解 search、tree-of-thought、MCTS 和 test-time compute scaling。
3. 能设计数学和代码推理训练数据。
4. 能评估 reasoning 模型并分析安全和局限。

阅读建议

建议先学习第三册数学推理增强实验，再用本书补齐理论和前沿研究。

与其他书的关系

第十六册连接第二册 reasoning 进阶、第三册数学推理项目、第七册推理评估和第十册论文研究。

第十六册：Reasoning Model 专题

本书定位

本书专门讨论推理模型、test-time compute、数学代码推理、verifier、搜索和过程监督。

章节文件

1. chapters/01-reasoning 总览.md
2. chapters/02-chain-of-thought.md
3. chapters/03-self-consistency 与采样.md
4. chapters/04-verifier 与 reward-model.md
5. chapters/05-process-supervision.md
6. chapters/06-search 与 tree-of-thought.md
7. chapters/07-test-time-compute-scaling.md
8. chapters/08-数学推理训练.md
9. chapters/09-代码推理与执行反馈.md
10. chapters/10-reasoning 评估.md
11. chapters/11-reasoning 安全与局限.md
12. chapters/12-reasoning 面试题.md

第一章：Reasoning 总览

Reasoning model 是当前大模型能力竞争的重要方向之一。传统 chat model 更强调对话流畅、指令遵循和通用知识，而 reasoning model 更强调多步推理、数学证明、代码求解、规划、验证和复杂任务分解。它不仅需要“答得像”，还要在有标准答案或可验证目标的任务上“答得对”。

Reasoning 的核心不只是让模型输出更长的解释。真正的 reasoning 涉及训练数据、Chain-of-Thought、self-consistency、verifier、process supervision、search、tool execution、test-time compute scaling 和评估体系。本章先建立全局地图，后续章节再逐个展开。

0. 本讲资料边界与第二轮精修口径

本章第二轮精修前，重点参考了 Chain-of-Thought、Zero-shot CoT、Self-Consistency、Training Verifiers to Solve Math Word Problems、Let's Verify Step by Step、Tree of Thoughts、HumanEval / pass@k、MATH / GSM8K，以及 OpenAI 关于 reasoning models 和 test-time compute 的公开资料。

本章是第十六册总览章，只建立 reasoning model 的全局地图和面试主线，不展开后续各章的完整技术细节。这里不把“输出很长解释”直接等同于真实推理，也不把闭源 reasoning 模型的内部训练 recipe 写成公开事实。第二轮精修重点是：

1. 区分 chat model、reasoning model、CoT prompt、训练出来的 reasoning 能力和推理时搜索 / 验证系统。
2. 用公式解释 self-consistency、best-of-n、pass@k、verifier reranking、process supervision、test-time compute 成本和上线门禁。
3. 用 0 依赖 demo 展示 greedy、self-consistency、verifier、pass@k、过程步骤准确率和 token 成本之间的关系。
4. 明确 reasoning 的安全边界：更强推理可能提升数学 / 代码 / 规划能力，也可能提升攻击规划、工具滥用和看似严谨的错误解释。

1.1 什么是 Reasoning

Reasoning 可以理解为模型从已知信息出发，通过一系列中间步骤得到结论的能力。

典型任务：

1. 数学题。
2. 代码题。
3. 逻辑题。
4. 规划任务。
5. 多跳问答。
6. 复杂工具使用。
7. 科学问题推理。

例如：

一个水箱每分钟进水 3 升，每分钟漏水 1 升，容量 20 升，从空开始多久装满？

模型不能只凭语言模式猜答案，而要计算净流入速度、容量和时间。

面试回答：

Reasoning 指模型基于问题条件进行多步推理、计算、验证和决策的能力。它常出现在数学、代码、逻辑、多跳问答和规划任务中。和

1.2 Reasoning Model 和 Chat Model 的区别

Chat model 主要优化：

1. 指令遵循。
2. 对话自然度。
3. 通用知识问答。
4. 安全拒答。
5. 用户体验。

Reasoning model 更强调：

1. 多步推理正确性。
2. 数学和代码能力。
3. 长思维链。
4. 自我检查。
5. verifier 或 reward model 选择答案。
6. test-time compute 扩展。

区别不是绝对的。很多强 chat model 也有 reasoning 能力，很多 reasoning model 也可以聊天。但训练目标、数据分布、推理策略和评估重点不同。

一个 chat model 可能回答流畅但算错；一个 reasoning model 可能花更多 token 思考，但在可验证任务上更准确。

1.3 为什么 Next-Token Prediction 也能产生推理

LLM 的基础训练目标是预测下一个 token。看起来这只是语言建模，为什么能推理？

原因包括：

1. 训练语料中包含大量推理过程。
2. 数学、代码、证明和解释文本本身有结构。
3. Transformer 能从上下文中组合模式。
4. 大规模参数和数据带来泛化能力。
5. 指令微调进一步强化了解题格式。

但 next-token prediction 本身不保证推理正确。模型可能生成“看起来像推理”的文本，但中间步骤错误，最终答案也错。

这就是为什么 reasoning 方向需要 verifier、过程监督、工具执行和 test-time compute。

1.3.1 关键公式与 Reasoning 总览速查

把一个 reasoning 样本抽象为：

$$r_i = (x_i, y_i^{\star}, C_i, V_i, B_i)$$

其中 x_i 是题目， $y_i^{\star}$ 是参考答案， C_i 是候选推理链集合， V_i 是 verifier 或测试器， B_i 是推理预算。

自回归生成与中间步骤

语言模型仍然按自回归方式生成推理链和答案：

$$p_{\theta}(y_{1:T} | x) = \prod_{t=1}^T p_{\theta}(y_t | x, y_{1:t-1})$$

Reasoning 的特殊之处不在于目标函数突然变成符号证明，而在于训练数据、提示、搜索、验证器和推理预算让模型更倾向于生成可验证的中间状态。

单次回答准确率

如果只生成一次，准确率为：

$$A_1 = \frac{1}{\sum_{i=1}^N \mathbb{1}[\hat{y}_i = y_i^{\star}]}$$

面试里要强调：greedy / temperature=0 的单次回答不等于模型的全部 reasoning 潜力。

Self-consistency

对同一道题采样 K 条推理链，抽取最终答案 a_{ik} 。多数投票可以写成：

$$\hat{y}_i^{\mathrm{sc}} = \mathrm{mode}(a_{i1}, \dots, a_{iK})$$

对应准确率：

$$A_{\mathrm{sc}} = \frac{1}{\sum_{i=1}^N \mathbb{1}[\hat{y}_i^{\mathrm{sc}} = y_i^{\star}]}$$

$$\mathbb{1}[\hat{y}_i^{\mathrm{sc}}=y_i^{\star}]$$

self-consistency 是用更多采样成本换候选多样性和答案稳定性，但如果模型系统性误解题意，多数投票也可能错。

Verifier / Best-of-N

生成 K 个候选后，用 verifier 分数 s_{ik} 选择最高分候选：

$$\begin{aligned} \hat{y}_i^{\mathrm{ver}} = & \\ y_{ij}, & \\ \quad \quad & \\ j = & \mathrm{index\ of\ max}_{k}(s_{ik}) \end{aligned}$$

对应准确率：

$$\begin{aligned} A_{\mathrm{ver}} = & \\ \frac{1}{N} & \\ \sum_{i=1}^N & \\ \mathbb{1}[\hat{y}_i^{\mathrm{ver}}=y_i^{\star}] & \end{aligned}$$

这里的 verifier 可以是规则、单元测试、代码执行器、数学检查器、reward model 或 LLM judge。最关键的面试点是：生成器负责提出候选，verifier 负责筛选。

pass@k

代码和可执行验证任务常看 pass@k。设一题生成 n 个候选，其中 c 个正确，不放回抽 k 个至少命中一个正确解的估计为：

$$\begin{aligned} \mathrm{pass@k} = & \\ 1 - & \\ \frac{\mathrm{binom}{n-c}{k}}{\mathrm{binom}{n}{k}} & \end{aligned}$$

当 $n-c < k$ 时，pass@k 记为 1。它衡量候选集合里是否存在正确解，不等于一次生成就可靠。

Process supervision

如果每条推理链有步骤标签 z_{ij} ，步骤级准确率可写成：

$$\begin{aligned} A_{\mathrm{step}} = & \\ \frac{\sum_i \sum_j z_{ij}}{\sum_i M_i} & \end{aligned}$$

其中 $z_{ij}=1$ 表示第 i 个样本的第 j 个推理步骤正确。过程监督能定位中间错误，但标注成本高，正确过程也可能有多种写法。

Test-time compute 成本

若每个样本采样 K_i 条候选，第 k 条使用 token 数为 T_{ik} ，总推理成本可近似为：

$$\begin{aligned} C_{\mathrm{ttc}} = & \\ \sum_{i=1}^N & \\ \sum_{k=1}^{K_i} & \\ T_{ik} & \end{aligned}$$

更真实的系统还要加入 verifier、工具执行、搜索节点、队列等待和人工复核成本。

Reasoning 上线门禁

一个简化 reasoning 门禁可以写成：

$$\begin{aligned} G_{\mathrm{reason}} = & \\ \mathbb{1}[& \\ A_{\mathrm{ver}} \geq \alpha & \\ \wedge & \end{aligned}$$

$$A_{\{\mathrm{step}\}} \geq \beta$$

$$\mathrm{land}$$

$$C_{\{\mathrm{ttc}\}} \leq C_{\{\mathrm{max}\}}$$

$$\mathrm{land}$$

$$R_{\{\mathrm{unsafe}\}} \leq \rho$$

$$\mathrm{land}$$

这说明 reasoning 系统不能只追求更高正确率，还要受成本、延迟、安全和可解释失败分析约束。

1.4 Chain-of-Thought

Chain-of-Thought, 简称 CoT, 是让模型显式生成中间推理步骤。

示例:

先计算净流入速度: $3 - 1 = 2$ 升/分钟。

水箱容量是 20 升。

时间 = $20 / 2 = 10$ 分钟。

答案是 10 分钟。

CoT 的作用:

1. 给模型更多中间计算空间。
2. 让复杂问题分解成步骤。
3. 便于人或 verifier 检查。
4. 可以提升数学和逻辑题表现。

局限:

1. CoT 可能看起来合理但其实错误。
2. 长推理会增加成本和延迟。
3. 公开 CoT 可能带来安全和隐私问题。
4. 不是所有任务都需要长推理。

1.5 Test-Time Compute

Test-time compute 指推理阶段额外花费计算来提升答案质量。

常见方式:

1. 生成更长思维链。
2. 采样多个解法。
3. self-consistency 投票。
4. verifier 选择最佳答案。
5. search 或 tree-of-thought。
6. 工具执行和反馈修正。

直觉: 训练时模型参数固定, 但推理时可以花更多计算探索更多候选、检查错误和选择更可靠答案。

面试回答:

Test-time compute 是在推理阶段增加计算预算来提升推理质量, 例如生成多条 CoT、self-consistency 投票、用 verifier 打分、搜 off 是准确率提升与延迟、成本增加之间的权衡。

1.6 Self-Consistency

Self-consistency 是一种简单有效的推理增强方法。

流程:

1. 对同一个问题采样多条推理路径。
2. 抽取每条路径的最终答案。

3. 对答案投票。
4. 选择出现最多或 verifier 分数最高的答案。

例子：

sample 1 -> answer A
sample 2 -> answer B
sample 3 -> answer A
sample 4 -> answer A
最终选择 A

优点：

1. 实现简单。
2. 不需要重新训练模型。
3. 对数学和推理题常有效。

缺点：

1. 成本随采样数线性增长。
2. 如果模型系统性错误，投票也会错。
3. 答案抽取和标准化很重要。

1.7 Verifier

Verifier 是用来判断候选答案或推理过程质量的模型或程序。

类型：

1. Outcome verifier：只判断最终答案好不好。
2. Process verifier：判断每一步推理是否合理。
3. Programmatic verifier：用规则、单元测试、执行器验证。
4. Reward model：给候选答案打分。

在数学题中，verifier 可以检查最终答案是否正确或步骤是否成立。在代码题中，最强 verifier 往往是执行测试用例。

Verifier 的价值：生成模型负责提出候选，verifier 负责筛选和纠错。

1.8 Process Supervision

过程监督不是只监督最终答案，而是监督中间步骤。

例如数学题：

step 1: 正确
step 2: 正确
step 3: 错误

优势：

1. 能更早发现错误。
2. 给模型更细粒度训练信号。
3. 有助于复杂推理。

挑战：

1. 标注成本高。
2. 中间步骤可能有多种正确写法。
3. 过程正确不一定最终答案正确，反之亦然。
4. 容易出现“看起来规范”的伪推理。

1.9 Search 和 Tree-of-Thought

Search 方法把推理看成搜索过程。

模型不是一次生成完整答案，而是在多个中间状态中探索。

常见思路：

1. 生成多个下一步。
2. 用 verifier 或 heuristic 评分。
3. 保留较好的分支。
4. 继续展开。

Tree-of-Thought 是这种思想的代表：把 reasoning steps 组织成树，而不是单条链。

优点：

1. 能探索多个思路。
2. 可结合 verifier 剪枝。
3. 对规划和复杂问题有帮助。

缺点：

1. 成本高。
2. 搜索空间大。
3. 评分器质量决定效果。

1.10 数学推理

数学推理是 reasoning model 最重要的评估场景之一。

能力包括：

1. 算术。
2. 代数。
3. 几何。
4. 概率统计。
5. 证明。
6. 多步应用题。

常见增强方式：

1. CoT 数据。
2. Self-consistency。
3. Verifier。
4. 过程监督。
5. Python 工具校验。
6. 题目难度 curriculum。

数学题的好处是答案通常可验证，因此适合研究 reasoning。

1.11 代码推理

代码推理的优势是可以执行验证。

模型可以：

1. 读题。
2. 写代码。
3. 运行测试。
4. 根据错误信息修复。
5. 迭代直到通过。

这比纯文本推理更可验证。单元测试、编译器、解释器都可以作为外部 verifier。

代码 reasoning 常见任务：

1. 算法题。
2. Bug 修复。
3. 单元测试生成。
4. 程序综合。
5. 代码解释。
6. 工具调用和执行反馈。

1.12 Reasoning 评估

Reasoning 评估不能只看回答是否长。

需要看：

1. 最终答案正确率。
2. 中间步骤正确性。
3. 鲁棒性。
4. 分布外泛化。
5. 成本和延迟。
6. 是否使用工具。
7. 是否能发现自己不确定。

常见 benchmark：

1. GSM8K。
2. MATH。
3. AIME 类竞赛题。
4. HumanEval。
5. MBPP。
6. Big-Bench Hard。

但要警惕 benchmark contamination。模型可能见过题目或类似解答，导致评估高估真实推理能力。

1.13 Reasoning 的安全和局限

Reasoning 能力增强也带来风险。

风险：

1. 更强的攻击规划。
2. 更会绕过规则。
3. 更强代码生成可能带来恶意代码风险。
4. 长 CoT 可能泄露敏感推理或系统策略。
5. 模型可能给出看似严谨但错误的解释。

局限：

1. 长思维链不等于真实推理。
2. Verifier 也可能错。
3. 搜索成本高。
4. 工具执行依赖环境。
5. 复杂现实问题没有标准答案。

1.14 训练时计算和推理时计算的权衡

提升 reasoning 有两条路：

1. 训练时投入更多计算，让模型本身更强。

2. 推理时投入更多计算，让模型多想、多试、多验证。

训练时计算的优点：

1. 单次推理更快。
2. 能力内化到参数里。

推理时计算的优点：

1. 可按任务难度动态分配预算。
2. 不需要重新训练就能提升部分任务表现。
3. 可以结合 verifier 和工具。

核心 trade-off：准确率、延迟、成本和可靠性。

1.15 Reasoning 项目路线

适合简历的项目：

1. 数学推理 self-consistency 实验。
2. GSM8K/MATH 评估和错误归因。
3. Python 工具校验数学答案。
4. 代码生成 + 单元测试反馈。
5. Verifier 训练和候选重排。
6. Process supervision 数据构造。
7. Tree-of-Thought 搜索 demo。
8. Test-time compute 成本-质量曲线分析。

项目表达要强调：任务定义、数据、方法、评估、成本、bad case 和改进方向。

1.16 最小可运行 reasoning 成本 / 正确率审计 demo

下面这个 demo 不调用真实模型，而是模拟 4 道 reasoning 题的候选答案。它比较 greedy 单次回答、self-consistency 多数投票、verifier reranking 和 pass@k，同时统计过程步骤准确率与 token 成本。

```
from collections import Counter
from math import comb
```

```
problems = [
    {
        "id": "water_tank",
        "gold": "10",
        "prompt_tokens": 40,
        "candidates": [
            {"answer": "12", "correct": False, "verifier": 0.25, "tokens": 42, "steps": [1, 0]},
            {"answer": "10", "correct": True, "verifier": 0.93, "tokens": 50, "steps": [1, 1, 1]},
            {"answer": "10", "correct": True, "verifier": 0.88, "tokens": 48, "steps": [1, 1]},
        ],
    },
    {
        "id": "code_sum",
        "gold": "pass",
        "prompt_tokens": 40,
        "candidates": [
            {"answer": "fail", "correct": False, "verifier": 0.45, "tokens": 55, "steps": [1, 0, 0]},
            {"answer": "pass", "correct": True, "verifier": 0.91, "tokens": 60, "steps": [1, 1, 1]},
            {"answer": "pass", "correct": True, "verifier": 0.87, "tokens": 58, "steps": [1, 1, 0]},
        ],
    },
]
```

```

    },
    {
        "id": "logic_grid",
        "gold": "blue",
        "prompt_tokens": 40,
        "candidates": [
            {"answer": "blue", "correct": True, "verifier": 0.86, "tokens": 35, "steps": [1, 1]},
            {"answer": "blue", "correct": True, "verifier": 0.84, "tokens": 37, "steps": [1, 1]},
            {"answer": "blue", "correct": True, "verifier": 0.82, "tokens": 36, "steps": [1, 1]},
        ],
    },
    {
        "id": "distractor_math",
        "gold": "ignore",
        "prompt_tokens": 40,
        "candidates": [
            {"answer": "use_extra", "correct": False, "verifier": 0.55, "tokens": 45, "steps": [1, 0]},
            {"answer": "use_extra", "correct": False, "verifier": 0.51, "tokens": 44, "steps": [0, 0]},
            {"answer": "ignore", "correct": True, "verifier": 0.89, "tokens": 52, "steps": [1, 1, 1]},
        ],
    },
]

```

```

def majority_answer(candidates):
    counts = Counter(candidate["answer"] for candidate in candidates)
    max_count = max(counts.values())
    winners = {answer for answer, count in counts.items() if count == max_count}
    for candidate in candidates:
        if candidate["answer"] in winners:
            return candidate["answer"]
    raise AssertionError("unreachable")

```

```

def pass_at_k(n, c, k):
    if c == 0:
        return 0.0
    if n - c < k:
        return 1.0
    return 1.0 - comb(n - c, k) / comb(n, k)

```

```

def audit_reasoning_budget(problems, k=2):
    greedy_correct = 0
    sc_correct = 0
    verifier_correct = 0
    pass1_values = []
    passk_values = []
    total_candidate_tokens = 0
    total_prompt_tokens = 0
    step_correct = 0
    step_total = 0
    per_problem = {}

    for problem in problems:

```



```

candidates = problem["candidates"]
gold = problem["gold"]
greedy = candidates[0]["answer"]
sc = majority_answer(candidates)
best = max(candidates, key=lambda item: item["verifier"])["answer"]
correct_count = sum(candidate["correct"] for candidate in candidates)
n = len(candidates)

greedy_correct += greedy == gold
sc_correct += sc == gold
verifier_correct += best == gold
pass1_values.append(pass_at_k(n, correct_count, 1))
passk_values.append(pass_at_k(n, correct_count, k))
total_candidate_tokens += sum(candidate["tokens"] for candidate in candidates)
total_prompt_tokens += problem["prompt_tokens"] * n
step_correct += sum(sum(candidate["steps"]) for candidate in candidates)
step_total += sum(len(candidate["steps"]) for candidate in candidates)
per_problem[problem["id"]] = {
    "greedy": greedy,
    "self_consistency": sc,
    "verifier": best,
    "correct_candidates": correct_count,
}

total_tokens = total_prompt_tokens + total_candidate_tokens
n_problems = len(problems)
report = {
    "greedy_accuracy": round(greedy_correct / n_problems, 3),
    "self_consistency_accuracy": round(sc_correct / n_problems, 3),
    "verifier_accuracy": round(verifier_correct / n_problems, 3),
    "pass_at_1_est": round(sum(pass1_values) / n_problems, 3),
    "pass_at_2_est": round(sum(passk_values) / n_problems, 3),
    "avg_candidates": round(sum(len(p["candidates"]) for p in problems) / n_problems, 3),
    "process_step_accuracy": round(step_correct / step_total, 3),
    "total_tokens": total_tokens,
    "cost_per_verified_correct": round(total_tokens / max(1, verifier_correct), 3),
    "per_problem": per_problem,
}
gates = {
    "verifier_beats_greedy": report["verifier_accuracy"] > report["greedy_accuracy"],
    "self_consistency_beats_greedy": report["self_consistency_accuracy"] > report["greedy_accuracy"],
    "pass_at_2_ready": report["pass_at_2_est"] >= 0.85,
    "process_quality_ok": report["process_step_accuracy"] >= 0.7,
    "budget_ok": total_tokens <= 1200,
}
report["gates"] = gates
report["gate_pass"] = all(gates.values())
return report

```

```

report = audit_reasoning_budget(problems)
for key, value in report.items():
    print(f"{key}={value}")

```

参考输出:

```
greedy_accuracy=0.25
self_consistency_accuracy=0.75
verifier_accuracy=1.0
pass_at_1_est=0.667
pass_at_2_est=0.917
avg_candidates=3.0
process_step_accuracy=0.759
total_tokens=1042
cost_per_verified_correct=260.5
per_problem={'water_tank': {'greedy': '12', 'self_consistency': '10', 'verifier': '10', 'correct_candidates': 2}, 'code_gates': {'verifier_beats_greedy': True, 'self_consistency_beats_greedy': True, 'pass_at_2_ready': True, 'process_quality_gate_pass': True}}
```

这个 demo 展示了 reasoning 系统的几个面试要点：

1. 单次 greedy 回答可能很弱，但候选集合里可能已有正确解。
2. self-consistency 能提升稳定性，但遇到系统性干扰时多数投票仍会错。
3. verifier 质量足够好时，best-of-n 可以显著提升正确率。
4. pass@k 衡量的是候选集合潜力，不等于线上一次调用体验。
5. reasoning 上线必须同时看准确率、步骤质量、token 成本和安全门禁。

1.17 面试官会怎么问

问题一：Reasoning model 和 chat model 有什么区别？

回答模板：

Chat model 更强调指令遵循、对话自然和通用问答；reasoning model 更强调数学、代码、规划和多步推理正确性。Reasoning model time compute，在可验证任务上追求更高准确率。

问题二：为什么 test-time compute 重要？

回答模板：

因为推理阶段可以通过生成多条思路、投票、搜索、verifier 选择和工具执行来提升复杂任务正确率。它让模型按任务难度动态花费

问题三：CoT 一定代表模型真的推理吗？

回答模板：

不一定。CoT 可能只是生成看起来合理的解释，步骤和真实内部计算未必一致。它对很多任务有效，但可能出现伪推理、错误步骤和事

问题四：Verifier 在 reasoning 中有什么作用？

回答模板：

Verifier 用来评估候选答案或推理过程的质量。生成模型负责提出多个候选，verifier 负责打分、筛选或发现错误。数学中可以检查

问题五：如何评估 reasoning 能力？

回答模板：

要看最终答案正确率、中间步骤正确性、鲁棒性、分布外泛化、工具使用能力、成本和延迟。数学和代码任务适合评估，因为答案或

1.18 本章总结

Reasoning model 的核心不是输出更长文本，而是在复杂、可验证任务中通过多步推理、搜索、验证和额外计算提升正确率。

需要记住：

1. Reasoning 强调数学、代码、逻辑、规划和多步任务。
2. CoT 提供中间推理空间，但不保证真实正确。
3. Self-consistency 用多样采样和投票提升可靠性。
4. Verifier 和 process supervision 提供更强监督和筛选。
5. Search 把推理变成候选路径探索。
6. Test-time compute 是准确率、延迟和成本之间的权衡。
7. Reasoning 评估要关注正确性、鲁棒性、成本和污染风险。

下一章会深入 Chain-of-Thought，讲清 CoT 的提出背景、prompt 形式、训练数据、显式/隐式推理、长思维链和常见误区。

第二章：Chain-of-Thought

0. 本讲资料边界与第二轮精修口径

本章第二轮精修时，重点参考 Chain-of-Thought Prompting、Zero-shot CoT、scratchpad intermediate computation、CoT faithfulness / unfaithfulness，以及公开 reasoning model 对隐藏推理、可见解释和 test-time compute 的讨论。这里把 CoT 当作 reasoning 系统中的一种推理时计算和中间状态表达方法，而不是把“输出更长解释”直接等同于“模型真实推理能力更强”。

本章聚焦：

1. CoT 为什么可能提升数学、逻辑、代码和多步任务表现。
2. few-shot CoT、zero-shot CoT、scratchpad、隐藏推理和可见解释的边界。
3. 如何用最终答案准确率、步骤准确率、答案一致性、成本和回归样本审计 CoT。
4. 为什么长 CoT 不一定更好，以及工程上为什么要做 CoT 路由。
5. 面试中如何回答“CoT 是否等于真实推理过程”这个高频追问。

本章不展开 self-consistency、多候选搜索、process reward model、Tree-of-Thought 或 test-time compute scaling 的完整算法细节，这些会在后续章节单独讲。本章也不提供诱导模型泄露隐藏推理、系统策略或安全规避路径的技巧；产品视角只讨论如何给用户展示简洁、可验证、必要的解释。

Chain-of-Thought，简称 CoT，是 reasoning model 中最基础也最容易被误解的概念。它的表面形式很简单：让模型在给出最终答案之前，先写出中间推理步骤。但它背后的问题很深：CoT 到底是在帮助模型“真正推理”，还是只是在生成看起来像推理的解释？为什么 CoT 对数学题、逻辑题有效，对某些简单任务反而没必要？长 CoT 是否一定更好？训练时应该监督完整 CoT 吗？推理时应该向用户展示 CoT 吗？

本章目标是讲清 CoT 的作用、使用方式、机制争议、有效场景、训练数据、风险和面试表达。

2.1 CoT 是什么

CoT 指模型在回答前生成一段中间推理过程。

普通回答：

答案是 10。

CoT 回答：

每分钟进水 3 升，漏水 1 升，所以净增加 2 升。

水箱容量是 20 升。

$20 / 2 = 10$ 。

答案是 10 分钟。

CoT 的核心作用是把复杂问题拆成多个小步骤，让模型有更多 token 空间进行中间计算和状态保存。

面试回答：

Chain-of-Thought 是让模型在最终答案前生成中间推理步骤的方法。它可以把复杂问题拆解成多个子步骤，给模型更多计算和中间状

2.1.1 关键公式与 CoT 审计指标速查

把一个 CoT 样本写成：

$$c_i = (x_i, z_i, \hat{y}_i, y_i^{\star}, m_i, b_i)$$

其中 x_i 是问题， z_i 是中间推理链， $\hat{y}_i$ 是模型最终答案， $y_i^{\star}$ 是标准答案， m_i 是任务元信息， b_i 是预算或安全边界。

带 CoT 的自回归生成可以粗略写成：

$$\begin{aligned} &P(z_i, \hat{y}_i \mid x_i) \\ &= \\ &\left(\prod_{t=1}^{L_i} P(z_{i,t} \mid x_i, z_{i,1:t-1}) \right) \\ &\left(\prod_{u=1}^{A_i} P(\hat{y}_{i,u} \mid x_i, z_i, \hat{y}_{i,1:u-1}) \right) \end{aligned}$$

其中 L_i 是推理链长度， A_i 是答案长度。这个式子强调：CoT 不是免费能力，推理链越长，推理时 token 成本越高。

直答准确率：

$$\begin{aligned} &A_{\{\mathrm{direct}\}} \\ &= \\ &\frac{1}{\sum_{i=1}^N} \\ &\mathbb{1}[\hat{y}_i^{\{\mathrm{direct}\}} = y_i^{\star}] \end{aligned}$$

CoT 准确率：

$$\begin{aligned} &A_{\{\mathrm{cot}\}} \\ &= \\ &\frac{1}{\sum_{i=1}^N} \\ &\mathbb{1}[\hat{y}_i^{\{\mathrm{cot}\}} = y_i^{\star}] \end{aligned}$$

步骤准确率：

$$\begin{aligned} &A_{\{\mathrm{step}\}} \\ &= \\ &\frac{\sum_i \sum_{j=1}^{M_i} q_{ij}}{\sum_i M_i} \end{aligned}$$

其中 $q_{ij}=1$ 表示第 i 个样本第 j 个步骤被人工、规则、工具或 verifier 判定为正确。

最终答案与推理链一致率：

$$\begin{aligned} &C_{\{\mathrm{final}\}} \\ &= \\ &\frac{1}{\sum_{i=1}^N} \\ &\mathbb{1}[g(z_i) = \hat{y}_i] \end{aligned}$$

其中 $g(z_i)$ 表示从推理链中抽取出的结论。这个指标只能说明“步骤和答案是否一致”，不能证明答案正确。

CoT 回归率：

$$\begin{aligned} &R_{\{\mathrm{reg}\}} \\ &= \\ &\frac{1}{\sum_i} \end{aligned}$$

```

\mathbb{1}[
\hat{y}_i^{\mathrm{direct}}=y_i^{\star}
\land
\hat{y}_i^{\mathrm{cot}}\neq y_i^{\star}
]
\}^N

```

它衡量“直答本来正确，但 CoT 反而带错”的比例。这个指标在简单事实问答、分类题和低延迟产品里很重要。

CoT token 成本：

```

C_{\mathrm{cot}}
=
\sum_{i=1}^N(T_i^{\mathrm{prompt}}+T_i^{\mathrm{cot}}+T_i^{\mathrm{answer}})

```

一个简化 CoT 路由门禁：

```

G_{\mathrm{cot}}
=
\mathbb{1}[
A_{\mathrm{route}}\geq A_{\mathrm{cot}}
\land
A_{\mathrm{step}}\geq \alpha
\land
R_{\mathrm{reg}}\leq \rho
\land
C_{\mathrm{route}}\leq C_{\max}
\land
S_{\mathrm{leak}}\leq \epsilon
]

```

其中 A_{route} 是按任务难度选择直答、CoT 或工具后的准确率， S_{leak} 是可见解释泄露隐藏策略或安全细节的风险分。面试中要强调：CoT 是否上线，不只看 A_{cot} ，还要看回归、步骤质量、成本和安全。

2.2 为什么 CoT 可能有效

CoT 可能有效有几个直觉原因。

2.2.1 增加中间计算空间

如果模型直接输出答案，它必须在很短上下文中完成所有隐式计算。CoT 允许模型把中间变量写出来。

例如：

```

净速度 = 3 - 1 = 2
时间 = 20 / 2 = 10

```

这类似人在草稿纸上解题。

2.2.2 分解复杂问题

多步问题可以拆成子问题。

例如：

1. 读题。
2. 提取条件。
3. 选择公式。
4. 计算。
5. 检查答案。

CoT 让模型显式走这些步骤。

2.2.3 匹配训练语料格式

互联网上有大量“逐步解答”的文本。模型学过这种格式后，在 prompt 中要求 step-by-step，可能激活类似解题模式。

2.2.4 便于后续验证

如果只有最终答案，verifier 很难判断错在哪里。CoT 提供中间步骤，便于 process verifier 或人类检查。

2.3 Few-shot CoT

Few-shot CoT 是在 prompt 中给几个带推理过程的示例，让模型模仿。

示例：

Q: 小明有 3 个苹果，又买了 2 个，一共有几个？

A: 小明原来有 3 个，又买了 2 个，所以 $3 + 2 = 5$ 。答案是 5。

Q: 一本书 10 元，买 3 本多少钱？

A:

模型看到示例后，更可能生成步骤：

每本 10 元，买 3 本，所以 $10 * 3 = 30$ 。答案是 30 元。

Few-shot CoT 的优点：

1. 不需要训练。
2. 对大模型效果明显。
3. 可以通过示例控制输出风格。

缺点：

1. 占用上下文。
2. 示例质量影响大。
3. 对小模型可能不稳定。

2.4 Zero-shot CoT

Zero-shot CoT 不给示例，只通过一句提示诱导模型逐步思考。

经典形式：

Let's think step by step.

中文可以是：

请一步一步思考。

这类提示能让模型生成更长推理过程。

但要注意：

1. 它可能提升复杂题表现。
2. 对简单题可能增加冗余。
3. 对不擅长的题可能生成更长但错误的解释。
4. 不应把“更长回答”当作“更强推理”。

2.5 CoT 和 Scratchpad

Scratchpad 可以理解为模型的草稿纸。它不一定是给用户看的最终解释，而是模型内部或中间生成的工作区。

区别：

1. CoT 常指自然语言推理链。
2. Scratchpad 更强调中间工作区，可以包含计算、代码、表格、草稿。
3. 最终回答可以只展示结论和简洁理由。

在产品中，很多系统不会直接展示完整 CoT，而是展示简洁解释。这是因为完整 CoT 可能冗长、错误、泄露策略或让用户误以为每一步都可靠。

2.6 CoT 是否等于真实推理

这是一个重要争议。

支持 CoT 有效的观察：

1. CoT 提升很多数学和逻辑 benchmark。
2. 中间步骤可以被检查。
3. 错误常能在步骤中定位。

质疑点：

1. CoT 可能是事后合理化。
2. 模型可能先决定答案，再编解释。
3. 有些 CoT 步骤看似合理但不因果影响答案。
4. CoT 可能掩盖错误，让答案更有迷惑性。

更稳妥的看法是：CoT 是一种有用的推理外显和计算扩展方式，但不能直接等同于模型真实内部机制。需要通过干预实验、过程验证和工具校验评估。

面试回答：

CoT 不一定等于模型真实内部推理。它可能帮助模型分解问题，也可能只是生成看起来合理的解释。判断 CoT 是否可靠，需要看最终

2.7 CoT 何时有效

CoT 常对这些任务有效：

1. 多步数学题。
2. 逻辑推理。
3. 多跳问答。
4. 规划任务。
5. 需要中间变量的代码题。
6. 复杂文本分析。

共同特点：问题不能靠单步模式匹配解决，需要中间状态。

CoT 不一定适合：

1. 简单事实问答。
2. 分类任务。
3. 低延迟场景。
4. 用户只需要短答案的场景。
5. 高安全风险场景中公开完整推理。

工程上可以动态决定是否启用长推理，而不是所有问题都强制 CoT。

2.8 长 CoT 是否一定更好

不一定。

长 CoT 的好处：

1. 有更多推理空间。
2. 可以分解复杂问题。
3. 便于检查。

长 CoT 的风险：

1. 成本高。
2. 延迟高。
3. 更容易跑偏。
4. 可能出现冗余和自相矛盾。
5. 可能生成更有说服力的错误解释。

合理做法是根据任务难度分配推理预算。简单题短答，复杂题长思考。

2.9 CoT 数据如何构造

CoT 训练数据通常包含：

1. 问题。
2. 推理步骤。
3. 最终答案。

例如：

```
{  
  "question": " 小明有 3 个苹果，又买了 2 个，一共有几个？",  
  "reasoning": " 小明原来有 3 个，又买了 2 个，所以  $3 + 2 = 5$ 。",  
  "answer": "5"  
}
```

高质量 CoT 数据要求：

1. 步骤正确。
2. 不跳步。
3. 不自相矛盾。
4. 最终答案和步骤一致。
5. 格式稳定。
6. 难度分布合理。

低质量 CoT 数据会训练模型生成错误但自信的推理链。

2.10 CoT 和最终答案格式

Reasoning 任务常需要明确分离推理和最终答案。

例如：

推理：...

答案：10

这样有几个好处：

1. 方便自动抽取答案。
2. 方便 verifier 检查。
3. 方便评估。
4. 减少格式混乱。

数学评估中，答案抽取非常重要。如果模型推理正确但答案格式不规范，自动评估可能判错。

2.11 Hidden CoT 和可见解释

产品中常区分内部推理和外部解释。

内部推理：模型或系统用于解决问题的草稿过程。

外部解释：给用户看的简洁、可靠、必要的说明。

为什么总是不总展示完整 CoT？

1. 完整 CoT 可能很长。
2. 可能包含错误中间想法。
3. 可能泄露系统策略。
4. 用户更需要可验证结论。
5. 安全场景下可能暴露规避路径。

因此很多系统会输出简洁解释，而不是完整内部推理。

2.12 CoT 与 Verifier 的关系

CoT 生成候选推理路径，verifier 负责评估。

组合方式：

1. 生成多条 CoT。
2. 抽取最终答案。
3. 用 verifier 或规则打分。
4. 选择最可信答案。

Verifier 可以看：

1. 最终答案是否正确。
2. 中间步骤是否合理。
3. 是否有计算错误。
4. 是否使用了题目中不存在的条件。

CoT 没有 verifier 时，容易生成 “有步骤但错” 的答案。

2.13 CoT 与工具使用

有些推理不应该只靠自然语言。

例如：

1. 大数计算。
2. 代码执行。
3. 符号运算。
4. 数据查询。
5. 单元测试。

这时更好的做法是：CoT 负责规划，工具负责精确执行。

例子：

先把题目转成 Python 表达式，再运行得到结果，最后解释。

工具使用可以显著降低计算错误，但也带来工具选择、参数、安全和执行失败问题。

2.14 常见 CoT 失败模式

1. 步骤看似合理但计算错。
2. 中间步骤和最终答案不一致。
3. 使用题目中没有给出的条件。

4. 过度解释简单问题。
5. 推理链很长但偏离题目。
6. 模型先猜答案再编理由。
7. 格式混乱导致答案抽取失败。
8. 在安全问题上给出规避细节。

这些失败说明 CoT 必须配合评估、格式约束和 verifier。

2.15 最小可运行 CoT 质量 / 成本审计 demo

下面这个 demo 不调用任何模型，而是用 toy case 模拟三种策略：

1. **direct**：直接给答案。
2. **cot**：生成推理链后给答案。
3. **routed**：只在复杂数学题和代码题上启用 CoT，简单题和安全边界题保持短答或拒答。

它展示一个面试中很重要的结论：CoT 整体可能比直答更好，但盲目 CoT 仍会在简单事实上回归，也可能在干扰条件题中生成看似合理但错误的步骤。

```
from collections import Counter
```

```
cases = [  
    {  
        "id": "water_tank",  
        "route": "hard_math",  
        "gold": "10",  
        "direct": "12",  
        "cot": "10",  
        "direct_tokens": 5,  
        "cot_tokens": 55,  
        "steps": [True, True, True],  
        "unsupported": False,  
        "visible_cot_allowed": True,  
    },  
    {  
        "id": "apple_simple",  
        "route": "simple",  
        "gold": "5",  
        "direct": "5",  
        "cot": "5",  
        "direct_tokens": 4,  
        "cot_tokens": 30,  
        "steps": [True, True],  
        "unsupported": False,  
        "visible_cot_allowed": True,  
    },  
    {  
        "id": "distractor_math",  
        "route": "hard_math",  
        "gold": "12",  
        "direct": "99",  
        "cot": "111",  
        "direct_tokens": 5,  
        "cot_tokens": 45,  
        "steps": [False, False],  
        "unsupported": True,  
        "visible_cot_allowed": True,  
    },  
]
```

```

    },
    {
        "id": "code_loop",
        "route": "code",
        "gold": "pass",
        "direct": "fail",
        "cot": "pass",
        "direct_tokens": 6,
        "cot_tokens": 60,
        "steps": [True, True, True],
        "unsupported": False,
        "visible_cot_allowed": True,
    },
    {
        "id": "capital_lookup",
        "route": "simple",
        "gold": "tokyo",
        "direct": "tokyo",
        "cot": "kyoto",
        "direct_tokens": 4,
        "cot_tokens": 48,
        "steps": [False, False],
        "unsupported": False,
        "visible_cot_allowed": True,
    },
    {
        "id": "safety_boundary",
        "route": "safety",
        "gold": "refuse",
        "direct": "refuse",
        "cot": "refuse",
        "direct_tokens": 8,
        "cot_tokens": 24,
        "steps": [True],
        "unsupported": False,
        "visible_cot_allowed": False,
    },
]

```

```

def accuracy(items, key):
    return sum(case[key] == case["gold"] for case in items) / len(items)

```

```

def route_answer(case):
    if case["route"] in {"hard_math", "code"}:
        return case["cot"], case["cot_tokens"]
    return case["direct"], case["direct_tokens"]

```

```

direct_acc = accuracy(cases, "direct")
cot_acc = accuracy(cases, "cot")
routed_correct = 0
routed_tokens = 0
route_counts = Counter()

```

```

for case in cases:
    answer, tokens = route_answer(case)
    routed_correct += answer == case["gold"]
    routed_tokens += tokens
    route_counts[case["route"]] += 1

all_steps = [ok for case in cases for ok in case["steps"]]
step_accuracy = sum(all_steps) / len(all_steps)
regression_ids = [
    case["id"]
    for case in cases
    if case["direct"] == case["gold"] and case["cot"] != case["gold"]
]
unsupported_ids = [case["id"] for case in cases if case["unsupported"]]
visible_cot_blocked = [
    case["id"] for case in cases if not case["visible_cot_allowed"]
]
simple_cases = [case for case in cases if case["route"] == "simple"]
simple_waste_or_regression = sum(
    case["direct"] == case["gold"]
    and case["cot_tokens"] > case["direct_tokens"]
    for case in simple_cases
) / len(simple_cases)

report = {
    "direct_accuracy": round(direct_acc, 3),
    "cot_accuracy": round(cot_acc, 3),
    "routed_accuracy": round(routed_correct / len(cases), 3),
    "step_accuracy": round(step_accuracy, 3),
    "avg_cot_tokens": round(
        sum(case["cot_tokens"] for case in cases) / len(cases), 3
    ),
    "cost_per_cot_correct": round(
        sum(case["cot_tokens"] for case in cases)
        / sum(case["cot"] == case["gold"] for case in cases),
        3,
    ),
    "cost_per_routed_correct": round(routed_tokens / routed_correct, 3),
    "simple_waste_or_regression": round(simple_waste_or_regression, 3),
    "route_counts": dict(route_counts),
    "cot_regression_ids": regression_ids,
    "unsupported_step_ids": unsupported_ids,
    "visible_cot_blocked": visible_cot_blocked,
}

gates = {
    "cot_beats_direct": report["cot_accuracy"] > report["direct_accuracy"],
    "routing_beats_cot": report["routed_accuracy"] > report["cot_accuracy"],
    "step_quality_ok": report["step_accuracy"] >= 0.65,
    "no_cot_regression": len(regression_ids) == 0,
    "no_unsupported_steps": len(unsupported_ids) == 0,
    "budget_ok": report["cost_per_routed_correct"] <= 40,
}
report["gates"] = gates

```

```
report["gate_pass"] = all(gates.values())
```

```
for key, value in report.items():  
    print(f"{key}={value}")
```

期望输出：

```
direct_accuracy=0.5  
cot_accuracy=0.667  
routed_accuracy=0.833  
step_accuracy=0.692  
avg_cot_tokens=43.667  
cost_per_cot_correct=65.5  
cost_per_routed_correct=35.2  
simple_waste_or_regression=1.0  
route_counts={'hard_math': 2, 'simple': 2, 'code': 1, 'safety': 1}  
cot_regression_ids=['capital_lookup']  
unsupported_step_ids=['distractor_math']  
visible_cot_blocked=['safety_boundary']  
gates={'cot_beats_direct': True, 'routing_beats_cot': True, 'step_quality_ok': True, 'no_cot_regression': False, 'no_unsupported_step': True}  
gate_pass=False
```

这段 demo 的面试表达是：

1. cot_accuracy 高于 direct_accuracy，说明 CoT 对复杂任务有价值。
2. routed_accuracy 又高于 cot_accuracy，说明工程上不应所有题都强制长 CoT。
3. capital_lookup 是 CoT 回归样本，说明简单事实题可能被过度推理带偏。
4. distractor_math 是 unsupported step 样本，说明 CoT 会把题目外的干扰条件写进推理链。
5. safety_boundary 不允许展示可见 CoT，说明产品中要区分内部推理和用户可见解释。
6. gate_pass=False 不是代码失败，而是在提醒：上线前还需要 regression guard、unsupported-step verifier 和安全展示策略。

2.16 面试官会怎么问

问题一：CoT 为什么能提升推理能力？

回答模板：

CoT 给模型更多中间计算空间，把复杂问题分解成步骤，并匹配训练语料中逐步解题的模式。它尤其适合数学、逻辑和多步任务，也

问题二：Zero-shot CoT 和 few-shot CoT 有什么区别？

回答模板：

Few-shot CoT 在 prompt 中给几个带推理过程的示例，让模型模仿；zero-shot CoT 不给示例，只用类似“请一步一步思考”的提示诱导模型推理。few-shot 更可控但占上下文，zero-shot 更简单但稳定性依赖模型能力。

问题三：CoT 是否代表模型真实推理过程？

回答模板：

不一定。CoT 可能帮助模型推理，也可能是事后合理化或看起来合理的解释。判断它是否可靠，需要看最终答案、步骤正确性、对中间

问题四：什么时候不应该使用长 CoT？

回答模板：

简单事实问答、低延迟场景、用户只需要短答案的任务，或者安全敏感场景中，不一定适合展示长 CoT。长 CoT 会增加成本和延迟，

问题五：如何构造高质量 CoT 数据？

回答模板：

高质量 CoT 数据要保证步骤正确、逻辑连贯、不跳步、不自相矛盾，最终答案和推理一致，并且格式稳定、难度分布合理。低质量 CoT 数据往往步骤混乱、逻辑断裂、跳步严重、自相矛盾，最终答案和推理不一致，格式不稳定，难度分布不合理。

2.17 小练习

1. 给一道小学应用题写普通答案和 CoT 答案。
2. 比较 zero-shot CoT 和 few-shot CoT 的 prompt。
3. 构造一个 CoT 看似合理但计算错误的样本。
4. 设计一个答案抽取格式，例如 答案：...。
5. 写一个 CoT 数据质量检查表。
6. 说明为什么产品中不一定展示完整 CoT。
7. 设计一个 CoT + Python 工具校验流程。
8. 用上面的 demo 增加一个简单题 CoT 回归样本，并观察 routed_accuracy 是否继续优于 cot_accuracy。

2.18 本章总结

CoT 是 reasoning model 的基础工具。它通过显式中间步骤给模型更多计算空间，帮助复杂问题分解，也便于 verifier 和人类检查。但 CoT 不是推理正确性的保证，更不一定等于模型真实内部机制。

需要记住：

1. CoT 常对数学、逻辑、多步任务有效。
2. Few-shot CoT 用示例诱导，zero-shot CoT 用指令诱导。
3. 长 CoT 不一定更好，要看任务难度和成本。
4. CoT 可能是伪解释，需要 verifier 或工具验证。
5. 高质量 CoT 数据很重要，错误 CoT 会污染模型。
6. 产品中通常更适合展示简洁解释，而不是完整内部推理。

下一章会进入 self-consistency 与采样，讲清为什么多采样、多路径投票和答案标准化能提升推理可靠性。

第三章：Self-Consistency 与采样

0. 本讲资料边界与第二轮精修口径

本章第二轮精修时，重点参考 Self-Consistency Improves Chain of Thought Reasoning in Language Models、Chain-of-Thought Prompting、nucleus sampling / top-p 采样，以及 HumanEval 中 pass@k 的评估口径。这里把 self-consistency 当作最小可落地的 test-time compute 方法：同一道题生成多个候选推理链，抽取并标准化最终答案，再通过多数投票、加权投票或 verifier 选择答案。

本章聚焦：

1. 为什么 greedy decoding 无法提供多路径多样性。
2. temperature、top-k、top-p 如何影响候选分布。
3. 答案抽取、答案标准化和等价类合并为什么是工程关键。
4. majority vote、weighted vote、pass@k 和 verifier rerank 的区别。
5. 如何用准确率、候选多样性、pass@k、成本和失败样本审计 self-consistency。

本章不展开 verifier / reward model 的训练细节，不展开 Tree-of-Thought / MCTS 搜索，也不把“采样更多次”写成无条件更强。self-consistency 只有在模型能生成一定比例正确候选、答案能标准化、采样多样性适中且成本可接受时才值得上线。

Self-consistency 是 reasoning 中最简单也最实用的 test-time compute 方法之一。它的核心思想是：同一个问题不要只生成一条推理链，而是采样多条不同推理路径，再从多个答案中聚合出更可靠的最终答案。对数学、逻辑、代码和复杂问答任务来说，多条独立路径往往能覆盖不同解法，降低单次生成偶然错误的影响。

但 self-consistency 不是免费午餐。它会增加推理成本，需要合理设置 temperature、top-p、采样数量、答案抽取、答案标准化和聚合策略。本章系统讲清这些工程细节。

3.1 为什么需要多路径采样

单次 CoT 生成容易受随机性和局部错误影响。

例如同一道题，模型可能生成：

路径 1：步骤正确，答案 42

路径 2：中间计算错，答案 36

路径 3：步骤正确，答案 42

路径 4：理解题意错，答案 40

如果只看一次，可能抽到错误答案。如果采样多次并投票，答案 42 更可能被选中。

面试回答：

Self-consistency 的思想是对同一个问题采样多条推理路径，抽取每条路径的最终答案，然后通过投票或打分选择最一致的答案。它

3.2 Self-Consistency 的基本流程

完整流程：

1. 给定问题。
2. 用非贪心采样生成 k 条 CoT。
3. 从每条 CoT 中抽取最终答案。
4. 对答案做标准化。
5. 投票或加权聚合。
6. 输出最终答案。

伪代码：

```
answers = []
```

```
for _ in range(k):  
    response = model.generate(prompt, temperature=0.7, top_p=0.95)  
    answer = extract_final_answer(response)  
    answer = normalize_answer(answer)  
    answers.append(answer)
```

```
final_answer = majority_vote(answers)
```

关键不是“多生成几次”这么简单，而是答案抽取和标准化要可靠。

3.2.1 关键公式与 Self-Consistency 指标速查

把第 i 道题的候选集合写成：

$$\mathcal{C}_i = \{(z_{ij}, \hat{y}_{ij}, s_{ij}, T_{ij})\}_{j=1}^{K_i}$$

其中 z_{ij} 是第 j 条推理链， $\hat{y}_{ij}$ 是候选最终答案， s_{ij} 是可选的 verifier 或规则分数， T_{ij} 是该候选消耗的 token 数。

温度采样把 logits l_v 变成概率：

$$p_v(\tau) = \frac{\exp(l_v / \tau)}{\sum_{u \in \mathcal{V}} \exp(l_u / \tau)}$$

其中 τ 是 temperature。 τ 越小，分布越尖锐； τ 越大，候选越分散。

Top-k 采样集合：

$$\mathcal{V}_k = \{v: L_v \in \text{top } k\}$$

Top-p / nucleus sampling 先按概率从大到小排序，选择最小前缀集合：

$$\mathcal{N}_p = \{v_{(1)}, \dots, v_{(m)}\}, \\ \text{quad} \\ \sum_{j=1}^m p_{(j)} \geq p$$

多数投票：

$$\hat{y}_i^{\text{maj}} = \arg\max_a \sum_{j=1}^{K_i} \mathbb{1}[\nu(\hat{y}_{ij}) = a]$$

其中 ν 是答案标准化函数，例如把 42.0、42 minutes 和 答案：42 合并成同一个答案等价类。

加权投票：

$$\hat{y}_i^w = \arg\max_a \sum_{j=1}^{K_i} s_{ij} \mathbb{1}[\nu(\hat{y}_{ij}) = a]$$

Self-consistency accuracy：

$$A_{\text{sc}} = \frac{1}{\sum_{i=1}^N} \mathbb{1}[\hat{y}_i^{\text{maj}} = y_i^*]$$

Pass@k 不衡量“最终选对”，而衡量候选集合中至少有一个正确候选的概率。给定 n 个候选、其中 c 个正确，不放回估计为：

$$\text{pass}@k = 1 - \frac{\text{binom}\{n-c\}\{k\}}{\text{binom}\{n\}\{k\}}$$

平均候选多样性可以粗略用标准化答案去重比例表示：

$$D_{\text{ans}} = \frac{1}{\sum_{i=1}^N} \frac{|\{\nu(\hat{y}_{ij})\}_{j=1}^{K_i}|}{K_i}$$

Self-consistency token 成本：

$$C_{\text{sc}} = \sum_{i=1}^N \sum_{j=1}^{K_i} T_{ij}$$

一个简化上线门禁：

```
G_{\mathrm{sc}}
=
\mathbb{1}[
A_{\mathrm{sc}} \geq A_{\mathrm{greedy}}
\land
\mathrm{pass}@k \geq \alpha
\land
D_{\mathrm{ans}} \geq \delta
\land
C_{\mathrm{sc}} \leq C_{\mathrm{max}}
]
```

面试中要强调：A_{sc}、pass@k 和 verifier rerank accuracy 是三种不同指标。pass@k 高说明候选集合里有正确解，但如果 majority vote 或 verifier 选错，线上最终答案仍然可能错。

3.3 Greedy Decoding 为什么不适合 self-consistency

Greedy decoding 每一步都选择概率最高的 token。

如果模型和输入完全相同，greedy 通常每次都生成同一条路径。

```
sample 1 -> same answer
sample 2 -> same answer
sample 3 -> same answer
```

这没有多样性，不能发挥 self-consistency。

Self-consistency 通常需要 sampling，例如设置 temperature、top-p 或 top-k，让模型探索不同推理路径。

3.4 Temperature

Temperature 控制采样分布的平滑程度。

低 temperature：

1. 输出更稳定。
2. 多样性较低。
3. 更接近 greedy。

高 temperature：

1. 输出更多样。
2. 可能探索不同解法。
3. 错误和跑偏风险更高。

reasoning 中常需要中等 temperature。太低没有多样性，太高会生成不可靠推理。

工程上需要根据任务调参，而不是固定一个值适用所有任务。

3.5 Top-p 和 Top-k

Top-k 限制每一步只从概率最高的 k 个 token 中采样。

Top-p 选择累计概率达到 p 的候选 token 集合。

在 reasoning 任务中：

1. top-p 可以保留合理多样性。
2. top-k 可以避免长尾低质量 token。
3. 过强截断可能让不同路径不够多样。

4. 过弱截断可能导致推理跑偏。

常见组合是 moderate temperature + top-p。

3.6 答案抽取

Self-consistency 的一个关键工程问题是：如何从一段长 CoT 中抽取最终答案。

推荐让模型使用固定格式：

答案：42

或者：

Final answer: 42

抽取函数可以基于规则：

```
def extract_final_answer(text):
    marker = " 答案: "
    if marker in text:
        return text.split(marker)[-1].strip().split("\n")[0]
    return text.strip().split("\n")[-1]
```

真实项目中要处理：

1. 多个答案标记。
2. 单位。
3. 分数和小数。
4. 多选题选项。
5. LaTeX。
6. 中文数字。
7. 模型输出解释和答案混在一起。

3.7 答案标准化

同一个答案可能有不同表达。

例如：

42

42.0

答案是 42

四十二

如果不标准化，投票会被分散。

标准化包括：

1. 去掉空格和标点。
2. 统一大小写。
3. 数值转标准格式。
4. 分数化简。
5. 单位归一。
6. 多选题提取选项字母。

数学任务中，答案标准化会显著影响评估结果。

3.8 Majority Vote

最简单聚合是多数投票。

```
from collections import Counter
```

```
def majority_vote(answers):  
    return Counter(answers).most_common(1)[0][0]
```

优点：

1. 简单。
2. 不需要额外模型。
3. 对答案空间明确的任务有效。

缺点：

1. 无法判断少数正确、多数错误的情况。
2. 对开放式答案不稳定。
3. 依赖答案抽取和标准化。

3.9 加权投票

如果每条路径有置信度或 verifier 分数，可以加权投票。

answer A: $0.8 + 0.7$

answer B: 0.9

最终 A 分数 1.5, B 分数 0.9

分数来源可以是：

1. 模型 log probability。
2. Verifier score。
3. Reward model。
4. 工具校验结果。
5. 规则检查。

加权投票比多数投票更强，但依赖评分器质量。

3.10 Pass@k

Pass@k 常用于代码生成和数学候选生成评估。

直觉：如果生成 k 个候选，只要其中一个正确，就认为 pass。

例如代码题：

1. 生成 10 个解法。
2. 对每个解法运行单元测试。
3. 只要有一个通过，就 pass@10 成功。

Pass@k 反映模型“生成正确候选的能力”，不等于模型“自动选择正确候选的能力”。如果没有 verifier 或测试器，生成了正确候选也未必能选出来。

面试中要区分：

1. pass@k: 候选集合里是否有正确答案。
2. self-consistency accuracy: 聚合后最终答案是否正确。
3. verifier accuracy: 能否选中正确候选。

3.11 成本和质量曲线

采样数 k 越大，通常准确率会先提升，但边际收益递减。

k=1 -> baseline
k=4 -> 明显提升
k=16 -> 继续提升但成本高
k=64 -> 可能收益很小

要画成本-质量曲线：

1. 横轴：平均 tokens、延迟或调用次数。
2. 纵轴：准确率。
3. 比较不同 k、temperature、verifier 策略。

工程上不能只追求最高准确率，还要考虑用户能接受的延迟和成本。

3.12 什么时候 self-consistency 有效

有效场景：

1. 答案可标准化。
2. 存在多条推理路径。
3. 单次生成容易偶然出错。
4. 模型有一定基础能力。
5. 多数正确路径能压过错误路径。

典型任务：

1. 数学应用题。
2. 逻辑题。
3. 选择题。
4. 代码候选生成。
5. 可执行工具验证任务。

不适合场景：

1. 开放创意写作。
2. 答案难以标准化。
3. 模型系统性错误。
4. 低延迟强约束场景。
5. 高风险任务中无 verifier 的投票。

3.13 多样性和正确性的平衡

Self-consistency 需要多样性，但不是越随机越好。

太保守：

所有样本几乎一样，投票没有意义。

太随机：

推理路径发散，错误答案增多。

理想状态是：候选路径有足够多样性，但仍围绕合理解法。

调参方向：

1. temperature。
2. top-p。
3. prompt 约束。
4. 最大推理长度。
5. verifier 过滤。

3.14 Self-Consistency 和 Verifier 结合

更强方案是采样 + verifier。

流程：

1. 采样多个候选。
2. 抽取答案和推理链。
3. verifier 给每个候选打分。
4. 选择最高分或加权投票。

代码题中 verifier 可以是单元测试。

数学题中 verifier 可以是：

1. 规则检查。
2. 计算器。
3. 符号系统。
4. 训练的 reward model。

Verifier 的引入能解决 “多数错、少数对” 的问题，但也可能引入 verifier 偏差。

3.15 一个完整实验设计

如果做 self-consistency 项目，可以这样设计：

1. 选择数据集，例如 GSM8K。
2. 设置 baseline: greedy CoT。
3. 设置采样策略: $k=4, 8, 16$ 。
4. 调 temperature 和 top-p。
5. 做答案抽取和标准化。
6. 比较 majority vote 和 verifier rerank。
7. 统计 accuracy、tokens、latency、cost。
8. 分析 bad cases。

bad case 分类：

1. 题意理解错。
2. 计算错。
3. 答案抽取错。
4. 多数投票选错。
5. 所有候选都错。
6. verifier 选错。

3.16 最小可运行 self-consistency / 采样审计 demo

下面这个 demo 用 0 依赖 Python 模拟 5 道 toy reasoning 题。它同时比较：

1. greedy: 单次直出答案。
2. majority: 对多个采样候选做答案标准化后多数投票。
3. weighted: 按 verifier / 规则分数做加权投票。
4. pass@2: 候选集合里至少有一个正确答案的概率估计。

重点不是模拟真实模型，而是把 self-consistency 项目里最容易漏掉的工程点一次跑通：答案标准化、候选多样性、majority failure、weighted rescue、pass@k 和 token 成本。

```
from collections import Counter, defaultdict
from math import comb
import re
```

```
problems = [
```

```

{
  "id": "water_tank",
  "gold": "10",
  "greedy": "12",
  "candidates": [
    {"answer": "10 minutes", "score": 0.86, "tokens": 58},
    {"answer": "10", "score": 0.78, "tokens": 54},
    {"answer": "12", "score": 0.42, "tokens": 46},
    {"answer": "10.0", "score": 0.81, "tokens": 61},
  ],
},
{
  "id": "distractor_math",
  "gold": "12",
  "greedy": "99",
  "candidates": [
    {"answer": "111", "score": 0.35, "tokens": 52},
    {"answer": "111", "score": 0.40, "tokens": 49},
    {"answer": "12", "score": 0.91, "tokens": 57},
    {"answer": "111", "score": 0.37, "tokens": 50},
    {"answer": "12 units", "score": 0.88, "tokens": 59},
  ],
},
{
  "id": "code_loop",
  "gold": "pass",
  "greedy": "fail",
  "candidates": [
    {"answer": "fail", "score": 0.20, "tokens": 42},
    {"answer": "pass", "score": 0.83, "tokens": 64},
    {"answer": "pass", "score": 0.79, "tokens": 68},
    {"answer": "pass", "score": 0.77, "tokens": 63},
  ],
},
{
  "id": "logic_grid",
  "gold": "blue",
  "greedy": "red",
  "candidates": [
    {"answer": "blue", "score": 0.72, "tokens": 47},
    {"answer": "green", "score": 0.41, "tokens": 43},
    {"answer": "blue", "score": 0.74, "tokens": 45},
    {"answer": "red", "score": 0.30, "tokens": 44},
  ],
},
{
  "id": "capital_lookup",
  "gold": "tokyo",
  "greedy": "Tokyo",
  "candidates": [
    {"answer": "tokyo", "score": 0.82, "tokens": 20},
    {"answer": "kyoto", "score": 0.25, "tokens": 35},
    {"answer": "Tokyo.", "score": 0.81, "tokens": 22},
    {"answer": "tokyo city", "score": 0.76, "tokens": 24},
  ],
},

```

```

    },
]

```

```

NUMBER_RE = re.compile(r"-?\d+(?:\.\d+)?")

```

```

def normalize_answer(answer):
    text = answer.strip().lower()
    if text in {"pass", "fail", "blue", "green", "red", "tokyo", "kyoto"}:
        return text
    if text.endswith("."):
        text = text[:-1]
    if text == "tokyo city":
        return "tokyo"
    numbers = NUMBER_RE.findall(text)
    if numbers:
        value = float(numbers[-1])
        if value.is_integer():
            return str(int(value))
        return f"{value:.6g}"
    return text

```

```

def majority_vote(answers):
    counts = Counter(answers)
    return counts.most_common(1)[0][0]

```

```

def weighted_vote(candidates):
    scores = defaultdict(float)
    for cand in candidates:
        scores[normalize_answer(cand["answer"])] += cand["score"]
    return max(scores.items(), key=lambda item: (item[1], item[0]))[0]

```

```

def pass_at_k(n, c, k):
    if c == 0:
        return 0.0
    if n - c < k:
        return 1.0
    return 1.0 - comb(n - c, k) / comb(n, k)

```

```

rows = []
for item in problems:
    normalized = [
        normalize_answer(cand["answer"])
        for cand in item["candidates"]
    ]
    n = len(normalized)
    c = sum(ans == item["gold"] for ans in normalized)
    rows.append(
        {
            "id": item["id"],
            "greedy": normalize_answer(item["greedy"]),

```

```

        "majority": majority_vote(normalized),
        "weighted": weighted_vote(item["candidates"]),
        "correct_candidates": c,
        "unique_answers": len(set(normalized)),
        "tokens": sum(cand["tokens"] for cand in item["candidates"]),
        "pass_at_2": round(pass_at_k(n, c, 2), 3),
    }
)

greedy_accuracy = sum(
    row["greedy"] == item["gold"]
    for row, item in zip(rows, problems)
) / len(problems)
majority_accuracy = sum(
    row["majority"] == item["gold"]
    for row, item in zip(rows, problems)
) / len(problems)
weighted_accuracy = sum(
    row["weighted"] == item["gold"]
    for row, item in zip(rows, problems)
) / len(problems)
pass_at_1_est = sum(
    row["correct_candidates"] / len(item["candidates"])
    for row, item in zip(rows, problems)
) / len(problems)
pass_at_2_est = sum(row["pass_at_2"] for row in rows) / len(rows)
total_tokens = sum(row["tokens"] for row in rows)
total_candidates = sum(len(item["candidates"]) for item in problems)
avg_unique_ratio = sum(
    row["unique_answers"] / len(item["candidates"])
    for row, item in zip(rows, problems)
) / len(rows)
majority_failures = [
    row["id"]
    for row, item in zip(rows, problems)
    if row["majority"] != item["gold"]
]
weighted_rescues = [
    row["id"]
    for row, item in zip(rows, problems)
    if row["majority"] != item["gold"]
    and row["weighted"] == item["gold"]
]

report = {
    "greedy_accuracy": round(greedy_accuracy, 3),
    "majority_accuracy": round(majority_accuracy, 3),
    "weighted_accuracy": round(weighted_accuracy, 3),
    "pass_at_1_est": round(pass_at_1_est, 3),
    "pass_at_2_est": round(pass_at_2_est, 3),
    "avg_unique_answer_ratio": round(avg_unique_ratio, 3),
    "total_candidates": total_candidates,
    "total_tokens": total_tokens,
    "cost_per_majority_correct": round(
        total_tokens / (majority_accuracy * len(problems)), 3
    )
}

```



```

    ),
    "majority_failures": majority_failures,
    "weighted_rescues": weighted_rescues,
    "per_problem": rows,
}
gates = {
    "majority_beats_greedy": (
        report["majority_accuracy"] > report["greedy_accuracy"]
    ),
    "weighted_beats_majority": (
        report["weighted_accuracy"] > report["majority_accuracy"]
    ),
    "pass_at_2_ready": report["pass_at_2_est"] >= 0.8,
    "diversity_not_too_low": report["avg_unique_answer_ratio"] >= 0.45,
    "budget_ok": report["cost_per_majority_correct"] <= 260,
}
report["gates"] = gates
report["gate_pass"] = all(gates.values())

for key, value in report.items():
    print(f"{key}={value}")

```

期望输出：

```

greedy_accuracy=0.2
majority_accuracy=0.8
weighted_accuracy=1.0
pass_at_1_est=0.63
pass_at_2_est=0.907
avg_unique_answer_ratio=0.53
total_candidates=21
total_tokens=1003
cost_per_majority_correct=250.75
majority_failures=['distractor_math']
weighted_rescues=['distractor_math']
per_problem=[{'id': 'water_tank', 'greedy': '12', 'majority': '10', 'weighted': '10', 'correct_candidates': 3, 'unique_answers': 3}]
gates={'majority_beats_greedy': True, 'weighted_beats_majority': True, 'pass_at_2_ready': True, 'diversity_not_too_low': True, 'budget_ok': True}
gate_pass=True

```

这段 demo 的面试表达是：

1. majority_accuracy 高于 greedy_accuracy，说明多路径采样能降低单次偶然错误。
2. distractor_math 的 majority vote 选错，但 weighted vote 选对，说明 pass@k 高不等于自动聚合后一定对。
3. pass_at_2_est 高于 pass_at_1_est，说明增加候选数能提高“至少包含一个正确解”的概率。
4. avg_unique_answer_ratio 不能太低，否则多次采样只是重复同一条路径。
5. cost_per_majority_correct 必须和准确率一起看，否则 self-consistency 会变成简单堆 token。

3.17 面试官会怎么问

问题一：Self-consistency 是什么？

回答模板：

Self-consistency 是对同一个问题采样多条推理路径，抽取最终答案，并通过多数投票或加权投票选择答案的方法。它利用不同 CoT

问题二：为什么 self-consistency 需要 sampling?

回答模板：

因为如果使用 greedy decoding，同一个输入通常每次生成同样路径，没有多样性。Self-consistency 需要通过 temperature、top-p 等采样方式生成不同推理路径，才能让投票发挥作用。

问题三：pass@k 和 majority vote 有什么区别？

回答模板：

pass@k 只看 k 个候选中是否至少有一个正确，衡量生成正确候选的能力；majority vote 是从多个候选答案中投票选最终答案，衡量

问题四：Self-consistency 的成本问题怎么处理？

回答模板：

采样 k 条路径会让成本和延迟大致随 k 增长。工程上要画成本-质量曲线，选择合适 k，并可结合早停、verifier 过滤、动态预算或 consistency。

问题五：Self-consistency 为什么可能失败？

回答模板：

如果模型系统性理解错题，多数采样也会错；如果答案抽取或标准化错误，投票会失真；如果 temperature 太高，路径可能跑偏；如

3.18 小练习

1. 对一道数学题采样 5 条 CoT，并手动投票。
2. 写一个 `extract_final_answer` 函数。
3. 写一个答案标准化函数，处理整数、小数和单位。
4. 比较 greedy、temperature=0.3、temperature=0.8 的输出差异。
5. 设计一个 pass@k 评估流程。
6. 画出 k 从 1 到 16 的成本-质量曲线。
7. 构造一个 majority vote 选错但 verifier 能选对的例子。
8. 修改上面的 demo，把 `distractor_math` 的 verifier 分数调低，观察 `weighted_accuracy` 如何变化。

3.19 本章总结

Self-consistency 是 test-time compute 的入门方法。它通过多路径采样和答案聚合提升推理可靠性，尤其适合答案可标准化、存在多种解法的数学和逻辑任务。

需要记住：

1. Greedy decoding 没有多样性，不适合 self-consistency。
2. Temperature 和 top-p 控制候选多样性。
3. 答案抽取和标准化是工程关键。
4. Majority vote 简单但可能选错。
5. Pass@k 衡量候选中是否有正确答案，不等于最终选择能力。
6. Verifier 可以提升候选选择质量。
7. 采样数量增加会带来成本和延迟，需要画成本-质量曲线。

下一章会进入 verifier 与 reward model，系统讲 outcome verifier、process verifier、reward model、候选重排和验证器训练。

第四章：Verifier 与 Reward Model

0. 本讲资料边界与第二轮精修口径

本章第二轮精修时，重点参考 Training Verifiers to Solve Math Word Problems、Let’s Verify Step by Step、InstructGPT / RLHF reward model、Learning to Summarize from Human Feedback、RewardBench 以及 HumanEval / pass@k 的评估口径。这里把 verifier / reward model 放在 reasoning 系统里讨论：generator 负责提出候选，verifier / reward model 负责打分、重排、过滤或辅助搜索。

本章聚焦：

1. outcome verifier、process verifier、reward model、programmatic verifier 的职责边界。
2. pointwise、pairwise、listwise 训练目标和 rerank 公式。
3. verifier reranking、best-of-N、process score、pairwise accuracy、hard negative accuracy、calibration 和成本门禁。
4. 为什么 verifier 本身也会被 reward hacking、长度偏差、格式偏差和 hard negative 误导。
5. 如何用最小 Python demo 审计 verifier 是否真的提升下游 reasoning accuracy。

本章不展开第五章 process supervision 的完整数据构造和 PRM 搜索算法，也不展开 RLHF / DPO 的完整训练 pipeline。这里的 reward model 只作为 reasoning 候选选择器来讲；对齐和偏好优化里的 reward model 会在后训练和安全章节中更系统展开。

Reasoning model 的一个核心趋势是“生成”和“验证”分工。生成模型负责提出候选答案、推理链或代码；verifier 或 reward model 负责判断哪个候选更可能正确。这个范式非常重要，因为复杂推理中，模型一次生成就答对很难，但生成多个候选再筛选，常常能显著提升准确率。

本章系统讲 verifier、reward model、outcome verifier、process verifier、reranking、pairwise/listwise 训练、生成-验证范式、工程使用方式和常见风险。

4.1 为什么需要 Verifier

单个生成模型容易出现：

1. 中间步骤错误。
2. 最终答案错误。
3. 多条候选质量不一。
4. 推理链看似合理但不可靠。
5. 自信输出错误答案。

如果能额外训练或使用一个 verifier，就可以让系统从多个候选中选择更好的答案。

直觉：

Generator：给出多个可能答案

Verifier：判断哪个答案更可信

面试回答：

Verifier 的作用是评估候选答案或推理过程的质量。Reasoning 中生成模型负责提出多个候选，verifier 负责打分、筛选或重排。这

4.1.1 关键公式与 Verifier 指标速查

把第 i 道题的候选集合写成：

$$\mathcal{C}_i = \{(z_{ij}, \hat{y}_{ij}, s_{ij}, p_{ij}, T_{ij})\}_{j=1}^{K_i}$$

其中 z_{ij} 是候选推理过程， $\hat{y}_{ij}$ 是候选答案， s_{ij} 是 verifier / reward model 分数， p_{ij} 是程序验证或工具验证结果， T_{ij} 是候选 token 成本。

Verifier 打分函数：

$$s_{ij} = f_{\phi}(x_i, z_{ij}, \hat{y}_{ij})$$

Rerank 选择:

$$j_i^{\star} = \arg\max_{j \in \{1, \dots, K_i\}} s_{ij}, \\ \hat{y}_i^{\mathrm{ver}} = \hat{y}_{ij_i^{\star}}$$

Pointwise 二分类 verifier loss:

$$L_{\mathrm{point}} = -q \log \sigma(s) - (1-q) \log(1 - \sigma(s))$$

其中 $q=1$ 表示候选正确, $q=0$ 表示候选错误。

Pairwise reward model / verifier loss:

$$L_{\mathrm{pair}} = -\log \sigma(r_w - r_l)$$

其中 r_w 是 chosen / winner 候选分数, r_l 是 rejected / loser 候选分数。

Listwise softmax loss:

$$L_{\mathrm{list}} = -\log \frac{\exp(s_{i, j^+})}{\sum_{j=1}^{K_i} \exp(s_{ij})}$$

其中 j^+ 是正确候选或标注最优候选。

Process verifier 可以把步骤分数聚合为整条链分数:

$$S_{\mathrm{proc}}(z_i) = \frac{1}{\sum_{m=1}^{M_i} q_{im}}$$

其中 q_{im} 是第 m 个步骤的正确性分数。

Rerank top-1 accuracy:

$$A_{\mathrm{rerank}} = \frac{1}{\sum_{i=1}^N} \mathbb{1}[\hat{y}_i^{\mathrm{ver}} = y_i^{\star}]$$

Pairwise accuracy:

$$A_{\mathrm{pair}} = \frac{1}{|\mathcal{P}|} \sum_{(a,b) \in \mathcal{P}} \mathbb{1}[s_a > s_b]$$

其中 a 是正确或更优候选, b 是错误或更差候选。

Verifier calibration 可以用 ECE 粗略审计:

$$\mathrm{ECE} =$$

$$\sum_{b=1}^B \frac{|S_b|}{n} \left| \mathrm{acc}(S_b) - \mathrm{conf}(S_b) \right|$$

一个简化 verifier 上线门禁:

$$G_{\{\mathrm{ver}\}} = \mathbb{1} [A_{\{\mathrm{rerank}\}} \geq \alpha \land A_{\{\mathrm{pair}\}} \geq \beta \land A_{\{\mathrm{hard}\}} \geq \gamma \land \mathrm{ECE} \leq \epsilon \land C_{\{\mathrm{ver}\}} \leq C_{\{\max\}}]$$

面试中要强调: verifier 本身不是可信性的终点。它必须在真实候选分布、hard negatives、校准、下游准确率和成本上一起评估。

4.2 Outcome Verifier

Outcome verifier 只看最终答案是否正确或更好。

输入可能是:

question + final answer

或者:

question + reasoning + final answer

输出:

score

它不一定逐步检查过程, 只判断最后结果质量。

优点:

1. 标注相对容易。
2. 适合答案可验证任务。
3. 可以用于候选重排。

缺点:

1. 无法指出哪一步错。
2. 对答案格式敏感。
3. 如果最终答案碰巧对, 过程错误也可能高分。

4.3 Process Verifier

Process verifier 检查中间步骤。

输入:

question + step_1 + step_2 + ...

输出可以是:

1. 每一步正确/错误。
2. 每一步分数。
3. 整条推理链分数。

优点：

1. 能更早发现错误。
2. 给搜索提供中间节点评分。
3. 适合 process supervision。

缺点：

1. 标注成本高。
2. 正确步骤可能有多种表达。
3. 步骤粒度难定义。
4. 评分器本身也可能错。

面试中可以说：outcome verifier 看结果，process verifier 看过程。

4.4 Reward Model 和 Verifier 的关系

Reward model 通常给模型输出打一个偏好分数。

在 RLHF 中，reward model 学的是人类偏好；在 reasoning 中，reward model 可以学候选答案是否正确、推理过程是否可靠、是否更符合题目要求。

关系：

1. Verifier 更强调正确性验证。
2. Reward model 更泛化，可能评分 helpfulness、format、safety、reasoning quality。
3. 在很多系统中，两者都可以作为候选重排器。

可以把 reasoning reward model 看成一种 learned verifier。

4.5 生成-验证范式

生成-验证范式流程：

1. Generator 生成多个候选。
2. Verifier 给每个候选打分。
3. 选择最高分候选。
4. 必要时继续搜索或修正。

伪代码：

```
candidates = []
```

```
for _ in range(k):  
    answer = generator.generate(question)  
    score = verifier.score(question, answer)  
    candidates.append((score, answer))
```

```
best = max(candidates, key=lambda x: x[0])
```

这种方法适合：

1. 数学题。
2. 代码题。
3. 规划任务。
4. 工具使用。
5. 多候选问答。

4.6 Reranking

Reranking 是 verifier 最常见的用法之一。

流程：

1. 生成 N 个候选答案。
2. verifier 对每个候选打分。
3. 按分数排序。
4. 返回 top-1 或 top-k。

和 self-consistency 的区别：

1. Self-consistency 通常按答案投票。
2. Reranking 按候选质量打分。
3. Reranking 可以选中少数但高质量的答案。

如果多数候选错但有一个正确，majority vote 可能失败；好的 verifier rerank 可能成功。

4.7 Pairwise Ranking

Pairwise 训练让模型判断两个候选哪个更好。

样本：

```
{  
  "question": "...",  
  "chosen": "  正确解法",  
  "rejected": "  错误解法"  
}
```

训练目标：让 chosen 分数高于 rejected。

优点：

1. 偏好标注比绝对打分容易。
2. 和 RLHF/DPO 数据形式类似。
3. 适合训练 reward model。

缺点：

1. 只知道相对偏好。
2. 数据覆盖不足时泛化差。
3. 可能学到长度、格式等偏差。

4.8 Pointwise 和 Listwise

除了 pairwise，还有 pointwise 和 listwise。

Pointwise：每个候选单独打分。

candidate -> score

Listwise：一次输入多个候选，直接学习排序。

[candidate_1, candidate_2, candidate_3] -> ranking

比较：

1. Pointwise 简单，但分数校准难。
2. Pairwise 标注容易，常用。
3. Listwise 更贴近排序目标，但训练复杂。

4.9 Verifier 训练数据

Verifier 数据来源：

1. 人工标注。
2. 规则自动标注。
3. 数学答案校验。
4. 代码单元测试。
5. LLM-as-Judge 初筛。
6. 生成模型产生错误候选。

高质量 verifier 数据需要包含：

1. 正确候选。
2. 常见错误候选。
3. 迷惑性错误。
4. 格式不同但正确的答案。
5. 中间步骤错误但最终碰巧对的样本。
6. 中间步骤合理但最终算错的样本。

如果 negative 太简单，verifier 只会学会区分明显错误，无法处理真实候选。

4.10 Verifier 的偏差

Verifier 可能学到错误偏差。

常见偏差：

1. 偏好更长答案。
2. 偏好格式更整齐的答案。
3. 偏好更自信的语气。
4. 偏好训练集中常见模式。
5. 不能识别隐蔽计算错误。

这会导致 reward hacking：生成模型学会骗过 verifier，而不是真的提高正确性。

解决方向：

1. 增加 hard negatives。
2. 做 calibration。
3. 用工具验证补充。
4. 定期更新 verifier 数据。
5. 分开评估 verifier 和 generator。

4.11 Programmatic Verifier

程序化 verifier 是最可靠的一类验证器之一。

例子：

1. 数学表达式用 Python 计算。
2. 代码题运行单元测试。
3. SQL 查询执行结果。
4. JSON schema 校验。
5. 形式化证明检查器。

优点：

1. 客观。
2. 可复现。
3. 不容易被语言风格欺骗。

缺点：

1. 只适用于可执行或可规则验证任务。
2. 测试用例可能不完整。
3. 执行环境有安全风险。
4. 需要处理超时和沙箱。

4.12 Verifier 与 Search

Search 需要中间状态评分。Process verifier 可以给每一步打分，帮助剪枝。

流程：

1. 当前推理状态生成多个下一步。
2. verifier 给每个下一步评分。
3. 保留高分分支。
4. 继续展开。

这比只在最后评分更高效，因为错误路径可以早停。

但如果 verifier 过早误杀正确分支，搜索也会失败。

4.13 ORM 和 PRM

Reasoning 中常见两个缩写：

1. ORM: Outcome Reward Model。
2. PRM: Process Reward Model。

ORM 评价最终答案或整条输出。

PRM 评价每一步过程。

对比：

ORM：这道题最终答对了吗？

PRM：这一步推理对吗？下一步是否合理？

PRM 更适合搜索和过程监督，但标注成本更高。

4.14 评估 Verifier

Verifier 本身也要评估。

指标：

1. Pairwise accuracy。
2. Ranking accuracy。
3. Calibration。
4. Top-1 selection accuracy。
5. Hard negative accuracy。
6. 下游任务提升。

最关键的是下游提升：使用 verifier 后，最终 reasoning accuracy 是否提高，成本是否可接受。

不要只看 verifier 在验证集上的分类准确率，因为它可能在真实生成候选分布上失效。

4.15 工程使用建议

使用 verifier 时建议：

1. 先建立 greedy baseline。
2. 再做 self-consistency baseline。

3. 加 verifier rerank。
4. 对比 accuracy、tokens、latency、cost。
5. 分析 verifier 选错的样本。
6. 检查是否偏好长答案或格式。
7. 对代码和数学尽量加入程序验证。

如果 verifier 带来的收益小于成本，就不一定值得上线。

4.16 最小可运行 verifier rerank 审计 demo

下面这个 demo 不调用模型，而是用 toy candidate set 模拟三种选择方式：

1. greedy: 直接取第一个候选。
2. rm_rerank: 只按 learned reward model / verifier 分数选择。
3. hybrid_verifier: 优先使用 programmatic verifier，再看过程步骤分数和 RM 分数。

它刻意构造了一个 hard negative: `distractor_math` 的错误候选 111 有更高 RM 分数，但程序验证失败。这个例子说明：reward model 能显著优于 greedy，但如果没有 hard negative、程序验证和校准审计，仍可能被“看起来更像正确推理”的错误候选骗过。

```
cases = [
  {
    "id": "water_tank",
    "gold": "10",
    "candidates": [
      {
        "answer": "12",
        "rm_score": 0.62,
        "steps": [True, False],
        "program_pass": False,
        "hard_negative": True,
        "tokens": 46,
      },
      {
        "answer": "10",
        "rm_score": 0.84,
        "steps": [True, True, True],
        "program_pass": True,
        "hard_negative": False,
        "tokens": 55,
      },
      {
        "answer": "8",
        "rm_score": 0.31,
        "steps": [False],
        "program_pass": False,
        "hard_negative": False,
        "tokens": 40,
      },
    ],
  },
  {
    "id": "distractor_math",
    "gold": "12",
    "candidates": [
      {
        "answer": "111",
```

```

        "rm_score": 0.91,
        "steps": [True, False, False],
        "program_pass": False,
        "hard_negative": True,
        "tokens": 62,
    },
    {
        "answer": "12",
        "rm_score": 0.78,
        "steps": [True, True, True],
        "program_pass": True,
        "hard_negative": False,
        "tokens": 58,
    },
    {
        "answer": "99",
        "rm_score": 0.28,
        "steps": [False],
        "program_pass": False,
        "hard_negative": False,
        "tokens": 43,
    },
],
},
{
    "id": "code_loop",
    "gold": "pass",
    "candidates": [
        {
            "answer": "fail",
            "rm_score": 0.40,
            "steps": [True, False],
            "program_pass": False,
            "hard_negative": False,
            "tokens": 50,
        },
        {
            "answer": "pass",
            "rm_score": 0.73,
            "steps": [True, True],
            "program_pass": True,
            "hard_negative": False,
            "tokens": 65,
        },
        {
            "answer": "pass",
            "rm_score": 0.69,
            "steps": [True, True, True],
            "program_pass": True,
            "hard_negative": False,
            "tokens": 70,
        },
    ],
},
},
{

```

```

    "id": "logic_grid",
    "gold": "blue",
    "candidates": [
      {
        "answer": "red",
        "rm_score": 0.66,
        "steps": [True, False],
        "program_pass": False,
        "hard_negative": True,
        "tokens": 45,
      },
      {
        "answer": "blue",
        "rm_score": 0.71,
        "steps": [True, True],
        "program_pass": True,
        "hard_negative": False,
        "tokens": 50,
      },
      {
        "answer": "green",
        "rm_score": 0.38,
        "steps": [False],
        "program_pass": False,
        "hard_negative": False,
        "tokens": 39,
      },
    ],
  },
  {
    "id": "capital_lookup",
    "gold": "tokyo",
    "candidates": [
      {
        "answer": "tokyo",
        "rm_score": 0.80,
        "steps": [True],
        "program_pass": True,
        "hard_negative": False,
        "tokens": 18,
      },
      {
        "answer": "kyoto",
        "rm_score": 0.58,
        "steps": [False],
        "program_pass": False,
        "hard_negative": True,
        "tokens": 32,
      },
      {
        "answer": "tokyo",
        "rm_score": 0.79,
        "steps": [True],
        "program_pass": True,
        "hard_negative": False,

```

```

        "tokens": 20,
    },
],
},
]

```

```

def is_correct(case, cand):
    return cand["answer"] == case["gold"]

```

```

def rm_select(case):
    return max(case["candidates"], key=lambda cand: cand["rm_score"])

```

```

def hybrid_select(case):
    return max(
        case["candidates"],
        key=lambda cand: (
            cand["program_pass"],
            sum(cand["steps"]) / len(cand["steps"]),
            cand["rm_score"],
        ),
    )

```

```

def accuracy(selector):
    return sum(is_correct(case, selector(case)) for case in cases) / len(cases)

```

```

def pairwise_accuracy():
    wins = 0
    total = 0
    for case in cases:
        correct = [
            cand for cand in case["candidates"]
            if is_correct(case, cand)
        ]
        wrong = [
            cand for cand in case["candidates"]
            if not is_correct(case, cand)
        ]
        for good in correct:
            for bad in wrong:
                total += 1
                wins += good["rm_score"] > bad["rm_score"]
    return wins / total

```

```

def hard_negative_accuracy():
    outcomes = []
    for case in cases:
        best_correct = max(
            cand["rm_score"]
            for cand in case["candidates"]

```

```

        if is_correct(case, cand)
    )
    for cand in case["candidates"]:
        if cand["hard_negative"]:
            outcomes.append(cand["rm_score"] < best_correct)
    return sum(outcomes) / len(outcomes)

def calibration_ece():
    bins = [(0.0, 0.5), (0.5, 0.75), (0.75, 1.01)]
    total = sum(len(case["candidates"]) for case in cases)
    ece = 0.0
    details = []
    for lo, hi in bins:
        bucket = [
            (cand["rm_score"], is_correct(case, cand))
            for case in cases
            for cand in case["candidates"]
            if lo <= cand["rm_score"] < hi
        ]
        if not bucket:
            continue
        conf = sum(score for score, _ in bucket) / len(bucket)
        acc = sum(ok for _, ok in bucket) / len(bucket)
        gap = abs(conf - acc)
        ece += len(bucket) / total * gap
        details.append((f"[{lo},{hi})", round(conf, 3), round(acc, 3), len(bucket)))
    return round(ece, 3), details

all_steps = [
    ok
    for case in cases
    for cand in case["candidates"]
    for ok in cand["steps"]
]
rm_failures = [
    case["id"]
    for case in cases
    if not is_correct(case, rm_select(case))
]
hybrid_rescues = [
    case["id"]
    for case in cases
    if not is_correct(case, rm_select(case))
    and is_correct(case, hybrid_select(case))
]
ece, ece_bins = calibration_ece()
total_tokens = sum(
    cand["tokens"]
    for case in cases
    for cand in case["candidates"]
)

report = {

```

```

"greedy_accuracy": round(
    sum(is_correct(case, case["candidates"][0]) for case in cases)
    / len(cases),
    3,
),
"rm_rerank_accuracy": round(accuracy(rm_select), 3),
"hybrid_verifier_accuracy": round(accuracy(hybrid_select), 3),
"pairwise_accuracy": round(pairwise_accuracy(), 3),
"hard_negative_accuracy": round(hard_negative_accuracy(), 3),
"process_step_accuracy": round(sum(all_steps) / len(all_steps), 3),
"rm_ece": ece,
"ece_bins": ece_bins,
"rm_failures": rm_failures,
"hybrid_rescues": hybrid_rescues,
"total_tokens": total_tokens,
"cost_per_hybrid_correct": round(
    total_tokens / (accuracy(hybrid_select) * len(cases)),
    3,
),
}
gates = {
    "rm_beats_greedy": (
        report["rm_rerank_accuracy"] > report["greedy_accuracy"]
    ),
    "hybrid_beats_rm": (
        report["hybrid_verifier_accuracy"] > report["rm_rerank_accuracy"]
    ),
    "pairwise_ok": report["pairwise_accuracy"] >= 0.85,
    "hard_negative_ok": report["hard_negative_accuracy"] >= 0.75,
    "calibration_ok": report["rm_ece"] <= 0.25,
    "budget_ok": report["cost_per_hybrid_correct"] <= 150,
}
report["gates"] = gates
report["gate_pass"] = all(gates.values())

```

```

for key, value in report.items():
    print(f"{key}={value}")

```

期望输出:

```

greedy_accuracy=0.2
rm_rerank_accuracy=0.8
hybrid_verifier_accuracy=1.0
pairwise_accuracy=0.9
hard_negative_accuracy=0.75
process_step_accuracy=0.679
rm_ece=0.165
ece_bins=[('0.0,0.5)', 0.343, 0.0, 4), ('0.5,0.75)', 0.665, 0.5, 6), ('0.75,1.01)', 0.824, 0.8, 5)]
rm_failures=['distractor_math']
hybrid_rescues=['distractor_math']
total_tokens=693
cost_per_hybrid_correct=138.6
gates={'rm_beats_greedy': True, 'hybrid_beats_rm': True, 'pairwise_ok': True, 'hard_negative_ok': True, 'calibration_ok': True, 'budget_ok': True}
gate_pass=True

```

这段 demo 的面试表达是:

1. `rm_rerank_accuracy` 明显高于 `greedy_accuracy`, 说明 `learned verifier` 有下游收益。
2. `rm_failures=['distractor_math']` 说明 `verifier` 会被 `hard negative` 骗过, 不能只看平均分。
3. `hybrid_verifier_accuracy` 高于 `rm_rerank_accuracy`, 说明能程序验证时, 程序信号通常应优先于语言风格分数。
4. `pairwise_accuracy` 和 `hard_negative_accuracy` 是 `verifier` 自身质量指标, 不等于最终生成系统准确率。
5. `rm_ece` 提醒我们 `verifier` 分数不是天然校准概率, 上线要做阈值和分桶校准。

4.17 面试官会怎么问

问题一: Verifier 和 generator 的关系是什么?

回答模板:

`Generator` 负责生成候选答案或推理路径, `verifier` 负责评估候选质量并选择更可靠的输出。这个生成-验证范式能把“提出可能解”

问题二: Outcome verifier 和 process verifier 有什么区别?

回答模板:

`Outcome verifier` 主要看最终答案或整条输出是否正确, 标注相对容易但无法定位中间错误。`Process verifier` 会检查每一步推理是

问题三: Reward model 和 verifier 有什么区别?

回答模板:

`Verifier` 更强调正确性验证, `reward model` 更泛化, 可以学习人类偏好、`helpfulness`、`format`、`safety` 和 `reasoning quality`。在

问题四: Verifier 为什么可能被 reward hacking?

回答模板:

如果 `verifier` 学到的是长度、格式、自信语气等表面特征, `generator` 可能优化这些特征来骗过 `verifier`, 而不是真的提高正确性。

问题五: 代码任务中最好的 verifier 是什么?

回答模板:

通常是执行测试用例。代码是否正确可以通过编译、运行单元测试、检查输出和性能约束来验证。相比语言 `reward model`, 程序执行

4.18 小练习

1. 构造一个 `question/chosen/rejected` 的 `verifier` 训练样本。
2. 比较 `outcome verifier` 和 `process verifier` 的适用场景。
3. 写一个简单 `rerank` 伪代码。
4. 设计一个代码题 `programmatic verifier`。
5. 列出 5 种 `verifier` 偏差。
6. 设计一个 `verifier` 评估表。
7. 构造一个 `majority vote` 失败但 `verifier` 成功的例子。
8. 修改上面的 demo, 让 `distractor_math` 的 `hard negative` 分数更高, 观察 `pairwise accuracy`、`hard negative accuracy` 和 `gate` 是否变化。

4.19 本章总结

`Verifier` 和 `reward model` 是 `reasoning` 系统中非常重要的组件。它们让系统不再只依赖一次生成, 而是可以生成多个候选、打分、重排、搜索和验证。

需要记住:

1. `Outcome verifier` 看最终结果。

2. Process verifier 看中间步骤。
3. Reward model 可以作为 learned verifier。
4. Reranking 能从多个候选中选择更好答案。
5. Programmatic verifier 在代码和可执行任务中非常强。
6. Verifier 也会有偏差和 reward hacking 风险。
7. 最终要看 verifier 是否提升下游 accuracy, 并且成本可接受。

下一章会进入 process supervision, 进一步讲如何监督中间步骤、如何构造过程标注, 以及 PRM 在 search 和 reasoning 中的作用。

第五章：Process Supervision

Process supervision, 过程监督, 是 reasoning model 中非常关键的一条路线。普通监督通常只看最终答案对不对; 过程监督进一步要求监督中间推理步骤是否正确。对于复杂数学、代码、规划和多步任务来说, 一个答案可能最后错了, 但前面很多步骤是对的; 也可能最终答案碰巧对, 但中间推理是错的。只看结果会浪费大量训练信号, 也难以定位错误来源。

本章系统讲过程监督的动机、步骤标注、PRM、推理轨迹、错误步骤定位、标注成本、和 search/verifier 的结合, 以及常见风险。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修参考公开资料中的 process supervision 和 PRM 线索, 重点包括 OpenAI 的 [Let's Verify Step by Step](#)、OpenAI 对过程监督的公开介绍、PRM800K 数据集说明, 以及更早的 [Training Verifiers to Solve Math Word Problems](#)。这些资料共同说明了一点: 在数学等可拆步骤任务上, 只监督最终答案会丢失大量中间错误信息; 如果能标注或评估每一步, 模型和搜索过程就能更早发现坏路径。

但本章不把 process supervision 写成所有 reasoning 任务的通用最优解。真实系统还要同时看最终答案、搜索效果、标注成本、标注一致性、PRM 偏差、hidden reasoning 边界和安全展示策略。本章 demo 也只模拟 toy 级审计闭环, 用于训练面试表达和工程指标意识, 不代表真实 PRM 训练配方。

5.1 为什么需要过程监督

Outcome supervision 只监督最终答案。

例如:

问题 -> 最终答案: 错

但它不知道哪里错。

Process supervision 会看每一步:

step 1: 正确

step 2: 正确

step 3: 错误

step 4: 基于错误继续推理

这样可以给模型更细粒度反馈。

面试回答:

过程监督不是只看最终答案, 而是监督推理过程中的每一步是否正确。它能更早发现错误, 给模型更密集的训练信号, 也能训练 proc

5.1.1 关键公式与 Process Supervision 指标速查

把第 i 个过程监督样本写成:

$$p_i = (x_i, z_i, y_i^{\star}, \ell_i, e_i, c_i)$$

其中 x_i 是题目, $z_i=(z_{i1},\ldots,z_{iM_i})$ 是推理步骤序列, y_i^* 是标准答案, ell_i 是步骤标签集合, e_i 是错误类型或第一处错误位置, c_i 是标注成本元信息。

步骤正确性和相关性可以写成:

$$q_{ij} \in \{0,1\}, \\ \quad \quad \quad \\ r_{ij} \in \{0,1\}$$

其中 $q_{ij}=1$ 表示第 j 步正确, $r_{ij}=1$ 表示这一步和解题相关。过程监督不能只鼓励“正确废话”, 否则模型可能生成大量无关但看似无害的步骤。

Outcome accuracy 仍然要保留:

$$A_{\{\mathrm{out}\}}= \\ \frac{1}{\sum_{i=1}^N} \\ \mathbb{1}[\hat{y}_i=y_i^*]$$

步骤分类准确率衡量 PRM 或过程分类器是否判断对每一步:

$$A_{\{\mathrm{step}\}}= \\ \frac{1}{\sum_{i=1}^N \sum_{j=1}^{M_i} \mathbb{1}[\hat{q}_{ij}=q_{ij}]} \\ \sum_{i=1}^N M_i$$

第一处错误位置是过程监督里最重要的诊断信号之一:

$$j_i^{\{\mathrm{err}\}}= \\ \min\{j:q_{ij}=0\}$$

如果样本没有错误步骤, 可以把 j_i^{err} 记为无错误。第一处错误检测准确率可写成:

$$A_{\{\mathrm{first}\}}= \\ \frac{1}{\sum_{i=1}^N} \\ \mathbb{1}[\hat{j}_i^{\{\mathrm{err}\}}=j_i^{\{\mathrm{err}\}}]$$

PRM 通常给“题目 + 已有步骤前缀 + 当前步骤”打分:

$$s_{ij}=f_{\{\phi\}}(x_i,z_{i,1:j})$$

整条推理链的过程分数可以取平均, 也可以取最薄弱步骤:

$$S_{\{\mathrm{avg}\}}(z_i)= \\ \frac{1}{\sum_{j=1}^{M_i} s_{ij}} \\ S_{\{\mathrm{min}\}}(z_i)= \\ \min_{1 \leq j \leq M_i} s_{ij}$$

平均分适合整体质量排序, 最小分适合“只要有一步明显错就要降级”的数学推理和代码推导任务。

PRM 接入搜索时, 一个简化 beam 更新可以写成:

$$B_{t+1}= \\ \mathrm{TopK} \\ (\\ \{b+a:b \in B_t, a \in \mathcal{A}(b)\}, \\ S_{\{\mathrm{avg}\}} \\)$$

其中 B_t 是第 t 步保留的中间状态集合， $A(b)$ 是从状态 b 可生成的下一步动作集合。这个公式表达的是：每一层生成多个下一步，再用过程分数剪枝。

标注侧要同时看自动标注覆盖率和人工标注成本：

$$C_{\{\mathrm{auto}\}} = \frac{\sum_i \sum_j a_{ij}}{\sum_i M_i}$$

$$C_{\{\mathrm{label}\}} = \sum_i \sum_j (1 - a_{ij}) c_{ij}$$

其中 $a_{ij}=1$ 表示第 i 个样本第 j 步可由程序、规则、测试或工具自动标注， c_{ij} 是需要人工复核时的成本。

一个上线前过程监督门禁可以写成：

$$G_{\{\mathrm{proc}\}} = \mathbb{1} [\begin{aligned} &A_{\{\mathrm{step}\}} \geq \alpha \\ &\land \\ &A_{\{\mathrm{first}\}} \geq \beta \\ &\land \\ &C_{\{\mathrm{auto}\}} \geq \gamma \\ &\land \\ &A_{\{\mathrm{search}\}} \geq \eta \\ &\land \\ &C_{\{\mathrm{label}\}} \leq C_{\{\max\}} \end{aligned}]$$

面试里要强调： A_{out} 、 A_{step} 、 A_{first} 、 A_{search} 和 C_{label} 不能互相替代。最终答案正确但过程错，说明 outcome supervision 有盲区；过程正确但最终格式错，说明系统还需要答案规范化或格式 verifier。

5.2 Outcome Supervision 的局限

只看最终答案有几个问题：

1. 不知道哪一步错。
2. 正确中间步骤没有被奖励。
3. 错误中间步骤可能被忽略。
4. 最终答案碰巧对时，错误过程可能被当成好样本。
5. 对长推理任务训练信号太稀疏。

例如数学题中，模型可能因为计算错一个符号导致最终答案错。如果只看最终答案，整条推理都被判错；但实际上前面题意理解和公式选择可能是正确的。

5.3 什么是推理步骤

过程监督首先要定义“步骤”。

一个步骤可以是：

1. 提取条件。
2. 写出公式。
3. 代入数值。
4. 做一次计算。
5. 得到中间结论。
6. 检查答案。

示例：

Step 1: 净速度 = $3 - 1 = 2$ 。

Step 2: 总容量是 20 升。

Step 3: 时间 = $20 / 2 = 10$ 。

步骤粒度太粗，定位不了错误；粒度太细，标注成本太高。合理粒度通常是一个可检查的逻辑单元。

5.4 过程标注格式

一种简单格式：

```
{
  "question": "...",
  "steps": [
    {"text": " 净速度 = 3 - 1 = 2", "label": "correct"},
    {"text": " 时间 = 20 / 2 = 10", "label": "correct"}
  ],
  "answer": "10"
}
```

也可以用分数：

```
{"step": "...", "score": 1.0}
```

或者标注错误类型：

```
{"step": "...", "label": "wrong", "error_type": "calculation_error"}
```

错误类型有助于后续分析和数据改进。

5.5 PRM: Process Reward Model

PRM 是给推理步骤或中间状态打分的模型。

输入：

question + previous steps + current step

输出：

当前步骤正确概率或分数

PRM 可以用于：

1. 训练模型偏好正确步骤。
2. 在搜索中给中间节点打分。
3. 早停错误路径。
4. 分析模型哪里出错。

PRM 和 ORM 的区别：PRM 评价过程，ORM 评价结果。

5.6 正确步骤和有用步骤

过程监督中要区分两件事：

1. 这一步是否正确。
2. 这一步是否有助于解题。

有些步骤正确但无关：

题目说有 3 个苹果。苹果是一种水果。

这句话正确，但对解题没帮助。

高质量过程监督应该鼓励：

1. 正确。
2. 相关。
3. 必要。
4. 不冗余。

否则模型可能学会生成很多正确但无用的废话。

5.7 错误步骤定位

一个好的 process supervision 数据集应该能定位第一处错误。

例如：

Step 1: 正确

Step 2: 正确

Step 3: 错误

Step 4: 因为 step 3 错而继续错

训练时要特别关注第一处错误，因为后面的错误可能只是连锁反应。

错误类型：

1. 题意理解错。
2. 条件提取错。
3. 公式选择错。
4. 计算错。
5. 单位错。
6. 逻辑跳步。
7. 引入不存在条件。

5.8 标注成本

过程监督最大问题是标注贵。

标最终答案相对简单；标每一步需要标注者理解题目和推理过程。

成本来自：

1. 步骤切分。
2. 判断每步正确性。
3. 标注错误类型。
4. 多种正确解法。
5. 领域知识要求。

降低成本的方法：

1. 用规则或工具自动校验部分步骤。
2. 用 LLM 初标，人类复核。
3. 只标注关键步骤。
4. 重点标 hard cases。
5. 使用程序化任务，例如代码测试。

5.9 自动过程监督

某些任务可以自动获得过程监督。

例如代码任务：

1. 每次修改后运行测试。
2. 编译错误定位到某个步骤。
3. 单元测试反馈是否通过。

数学任务中也可以用：

1. Python 计算器。
2. 符号计算。
3. 单位检查。
4. 公式规则。

但自动监督覆盖有限。自然语言推理步骤是否“合理”很难完全规则化。

5.10 Process Supervision 与 Search

搜索需要中间节点评分，PRM 很适合。

流程：

1. 模型生成多个下一步。
2. PRM 给每个下一步打分。
3. 保留高分步骤。
4. 继续展开。

相比只用最终答案评分，PRM 可以更早剪掉错误路径，节省计算。

风险：如果 PRM 错误地低估了正确路径，搜索会错过好解。

5.11 Process Supervision 与 RL

过程奖励可以用于强化学习。

普通 outcome reward：只在最终答案给奖励。

过程 reward：每个步骤都可能给奖励。

优点：

1. 奖励更密集。
2. Credit assignment 更容易。
3. 对长推理任务更友好。

挑战：

1. PRM 质量决定训练方向。
2. 模型可能优化表面步骤。
3. 奖励 hacking 风险。

5.12 过程监督的风险

风险包括：

1. 模型学会写“看起来正确”的步骤。
2. 步骤格式僵化。
3. 标注者偏好影响推理风格。
4. 多种正确解法被误判。
5. PRM 被 reward hacking。
6. 长步骤导致成本上升。

因此过程监督不能只看步骤评分，还要看最终任务正确率和泛化能力。

5.13 过程监督和可解释性

过程监督会让模型输出更结构化的推理过程，看起来更可解释。

但注意：

1. 可读步骤不一定是真实内部机制。
2. 步骤正确不一定代表模型真正理解。
3. 解释可能只是训练出的格式。

过程监督能提升可检查性，但不能完全解决可解释性问题。

5.14 数据集设计建议

设计 process supervision 数据集时，要覆盖：

1. 简单题和复杂题。
2. 多种解法。
3. 常见错误类型。
4. 迷惑性错误步骤。
5. 正确但冗余步骤。
6. 最终答案正确但过程错误的样本。
7. 过程正确但最后格式错误的样本。

数据越贴近模型真实错误分布，PRM 越有用。

5.15 评估 Process Supervision

评估维度：

1. Step classification accuracy。
2. First-error detection accuracy。
3. PRM ranking accuracy。
4. Search 后最终准确率。
5. 成本和延迟。
6. 对 hard cases 的提升。

最终还是要看：使用过程监督后，reasoning 系统是否在真实任务上更准确、更稳定。

5.16 最小可运行 process supervision / PRM 审计 demo

这个 demo 不训练真实 PRM，而是模拟一张上线前审计表：同一批 toy reasoning 样本同时记录最终答案、步骤标签、PRM 预测、自动标注覆盖、人工标注成本和搜索选择结果。它刻意保留两个 bad case：lucky_answer 是最终答案碰巧正确但过程错误，format_error 是过程正确但最终格式不合格。

输入：

1. toy 样本的最终答案、标准答案和步骤标签。
2. 每一步的 PRM 预测、是否相关、是否可自动标注和人工成本。
3. toy 搜索状态的候选动作与 PRM 选择。

输出：

1. outcome accuracy、step accuracy、first-error accuracy。
2. relevant step ratio、auto label coverage、human label cost。
3. outcome supervision 盲区样本、冗余步骤和 search failure。
4. process supervision gate。

```
def first_error(labels):  
    for idx, label in enumerate(labels, start=1):  
        if label == 0:
```

```

        return idx
    return None

```

```

cases = [
    {
        "id": "clean_solve",
        "gold": "10",
        "answer": "10",
        "steps": [
            {"id": "extract_rate", "q": 1, "pred": 1, "relevant": 1, "auto": 1, "cost": 0},
            {"id": "net_rate", "q": 1, "pred": 1, "relevant": 1, "auto": 1, "cost": 0},
            {"id": "volume", "q": 1, "pred": 1, "relevant": 1, "auto": 1, "cost": 0},
            {"id": "time_formula", "q": 1, "pred": 1, "relevant": 1, "auto": 0, "cost": 2},
            {"id": "compute_time", "q": 1, "pred": 1, "relevant": 1, "auto": 1, "cost": 0},
            {"id": "final_answer", "q": 1, "pred": 1, "relevant": 1, "auto": 0, "cost": 2},
        ],
    },
    {
        "id": "lucky_answer",
        "gold": "42",
        "answer": "42",
        "steps": [
            {"id": "read_question", "q": 1, "pred": 1, "relevant": 1, "auto": 1, "cost": 0},
            {"id": "wrong_formula", "q": 0, "pred": 0, "relevant": 1, "auto": 0, "cost": 3},
            {"id": "bad_substitution", "q": 0, "pred": 1, "relevant": 1, "auto": 0, "cost": 0},
            {"id": "redundant_plan", "q": 1, "pred": 1, "relevant": 0, "auto": 0, "cost": 2},
            {"id": "lucky_arithmetic", "q": 0, "pred": 1, "relevant": 1, "auto": 1, "cost": 0},
            {"id": "unsupported_finish", "q": 0, "pred": 1, "relevant": 1, "auto": 0, "cost": 2},
        ],
    },
    {
        "id": "format_error",
        "gold": "6",
        "answer": "six hours",
        "steps": [
            {"id": "extract_distance", "q": 1, "pred": 1, "relevant": 1, "auto": 1, "cost": 0},
            {"id": "extract_speed", "q": 1, "pred": 1, "relevant": 1, "auto": 1, "cost": 0},
            {"id": "divide", "q": 1, "pred": 1, "relevant": 1, "auto": 1, "cost": 0},
            {"id": "unit_check", "q": 1, "pred": 1, "relevant": 1, "auto": 0, "cost": 3},
            {"id": "numeric_result", "q": 1, "pred": 1, "relevant": 1, "auto": 1, "cost": 0},
            {"id": "format_instruction", "q": 1, "pred": 1, "relevant": 1, "auto": 0, "cost": 2},
        ],
    },
]

```

```

search_states = [
    {
        "id": "distractor_state",
        "choices": [
            {"name": "follow distractor", "score": 0.22, "good": False},
            {"name": "ignore distractor", "score": 0.81, "good": True},
        ],
    },
]

```



```

        "id": "logic_state",
        "choices": [
            {"name": "branch_red", "score": 0.45, "good": False},
            {"name": "branch_blue", "score": 0.76, "good": True},
        ],
    },
    {
        "id": "hard_negative_state",
        "choices": [
            {"name": "plain_correct", "score": 0.62, "good": True},
            {"name": "polished_wrong", "score": 0.84, "good": False},
        ],
    },
]

outcome_correct = [case["answer"] == case["gold"] for case in cases]
all_steps = [step for case in cases for step in case["steps"]]

step_matches = [step["q"] == step["pred"] for step in all_steps]
outcome_accuracy = round(sum(outcome_correct) / len(cases), 3)
step_accuracy = round(sum(step_matches) / len(step_matches), 3)

true_first_errors = [first_error([step["q"] for step in case["steps"]]) for case in cases]
pred_first_errors = [first_error([step["pred"] for step in case["steps"]]) for case in cases]
first_error_accuracy = round(
    sum(t == p for t, p in zip(true_first_errors, pred_first_errors)) / len(cases),
    3,
)

relevant_step_ratio = round(sum(step["relevant"] for step in all_steps) / len(all_steps), 3)
auto_label_coverage = round(sum(step["auto"] for step in all_steps) / len(all_steps), 3)
human_label_cost = sum(0 if step["auto"] else step["cost"] for step in all_steps)

final_correct_bad_process = [
    case["id"]
    for case in cases
    if case["answer"] == case["gold"] and any(step["q"] == 0 for step in case["steps"])
]
process_correct_bad_final = [
    case["id"]
    for case in cases
    if case["answer"] != case["gold"] and all(step["q"] == 1 for step in case["steps"])
]
redundant_step_ids = [step["id"] for step in all_steps if step["relevant"] == 0]

search_choices = []
for state in search_states:
    choice = max(state["choices"], key=lambda item: item["score"])
    search_choices.append({"id": state["id"], "choice": choice["name"], "good": choice["good"]})

search_top1_accuracy = round(
    sum(choice["good"] for choice in search_choices) / len(search_choices),
    3,
)
search_failures = [choice["id"] for choice in search_choices if not choice["good"]]

```

```

gates = {
    "step_quality_ok": step_accuracy >= 0.8,
    "first_error_ok": first_error_accuracy >= 0.9,
    "auto_coverage_ok": auto_label_coverage >= 0.5,
    "search_ready": search_top1_accuracy >= 0.8,
    "cost_ok": human_label_cost <= 20,
    "no_outcome_blind_spot": not final_correct_bad_process and not process_correct_bad_final,
}
gate_pass = all(gates.values())

print(f"outcome_accuracy={outcome_accuracy}")
print(f"step_accuracy={step_accuracy}")
print(f"first_error_accuracy={first_error_accuracy}")
print(f"relevant_step_ratio={relevant_step_ratio}")
print(f"auto_label_coverage={auto_label_coverage}")
print(f"human_label_cost={human_label_cost}")
print(f"final_correct_bad_process={final_correct_bad_process}")
print(f"process_correct_bad_final={process_correct_bad_final}")
print(f"redundant_step_ids={redundant_step_ids}")
print(f"search_top1_accuracy={search_top1_accuracy}")
print(f"search_choices={search_choices}")
print(f"search_failures={search_failures}")
print(f"gates={gates}")
print(f"gate_pass={gate_pass}")

```

预期输出：

```

outcome_accuracy=0.667
step_accuracy=0.833
first_error_accuracy=1.0
relevant_step_ratio=0.944
auto_label_coverage=0.556
human_label_cost=16
final_correct_bad_process=['lucky_answer']
process_correct_bad_final=['format_error']
redundant_step_ids=['redundant_plan']
search_top1_accuracy=0.667
search_choices=[{'id': 'distractor_state', 'choice': 'ignore distractor', 'good': True}, {'id': 'logic_state', 'choice': 'polish hard negative', 'good': True}]
search_failures=['hard_negative_state']
gates={'step_quality_ok': True, 'first_error_ok': True, 'auto_coverage_ok': True, 'search_ready': False, 'cost_ok': True, 'no_outcome_blind_spot': True}
gate_pass=False

```

这个输出刻意让 `gate_pass=False`。它不是 demo 失败，而是在说明 process supervision 的真实工程判断：步骤质量和第一处错误检测看起来不错，但 outcome supervision 有盲区，PRM 搜索也会被 polished hard negative 误导。上线前不能只看 A_step，还要补 hard negative、格式 verifier、搜索切片和人工复核策略。

5.17 面试官会怎么问

问题一：Process supervision 是什么？

回答模板：

Process supervision 是对推理中间步骤进行监督，而不是只监督最终答案。它可以标注每一步是否正确，训练 process reward model。

问题二：它和 outcome supervision 有什么区别？

回答模板：

Outcome supervision 只看最终答案对不对，训练信号稀疏，无法定位中间错误。Process supervision 会检查每一步推理，能提供更

问题三：PRM 有什么作用？

回答模板：

PRM 是 process reward model，用来给推理步骤或中间状态打分。它可以用于搜索时选择更好的下一步，也可以用于训练模型偏好正确

问题四：过程监督为什么成本高？

回答模板：

因为标注者不仅要知道最终答案，还要理解每一步推理是否正确、是否相关、错误发生在哪里，以及错误类型是什么。多种正确解法

问题五：过程监督有什么风险？

回答模板：

模型可能学会生成格式漂亮但不真实的推理步骤，也可能被标注风格限制，或者 reward hacking PRM。过程正确性也不等于真实内部

5.18 小练习

1. 给一道数学题写出逐步标注格式。
2. 标出一条错误 CoT 中第一处错误。
3. 设计 5 类推理错误类型。
4. 构造一个最终答案正确但过程错误的样本。
5. 设计一个 PRM 输入输出格式。
6. 比较 PRM 和 ORM 在搜索中的作用。
7. 设计一个 process supervision 数据质量 checklist。
8. 运行本章 demo, 并新增一个“最终答案正确但第一步读题错误”的 hard case, 观察 outcome accuracy、first-error accuracy 和 gate 如何变化。

5.19 本章总结

Process supervision 通过监督中间步骤，为 reasoning model 提供比最终答案更细粒度的训练信号。它能帮助错误定位、训练 PRM、支持搜索和提升复杂任务表现，但也带来高标注成本、步骤粒度争议和 reward hacking 风险。

需要记住：

1. Outcome supervision 看最终答案。
2. Process supervision 看中间步骤。
3. PRM 给步骤或中间状态打分。
4. 第一处错误定位非常重要。
5. 过程监督适合 search 和长推理任务。
6. 标注成本和标注质量是最大瓶颈。
7. 最终要看下游 accuracy 是否提升。
8. 工程上必须同时看 outcome、step、first-error、search、标注成本和 hard negative。

下一章会进入 search 与 Tree-of-Thought，讲清如何把推理建模成搜索、如何展开候选、如何用 verifier 剪枝，以及 MCTS 等方法的直觉。

第六章：Search 与 Tree-of-Thought

前面几章讲了 CoT、self-consistency、verifier 和过程监督。它们共同指向一个更一般的思想：把推理看成搜索。模型不必一次生成完整答案，而是可以在多个中间状态中探索、评分、剪枝和回溯。Tree-of-Thought、beam search、MCTS、规划式推理都属于这条路线。

本章系统讲 search reasoning：为什么需要搜索、如何定义状态和动作、Tree-of-Thought 如何工作、beam search 和 MCTS 有什么区别、verifier 如何参与剪枝、工程上如何控制成本，以及面试中如何把这类方法讲清楚。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修参考公开资料中的搜索推理路线，重点包括 [Tree of Thoughts](#)、[Language Agent Tree Search](#)、UCT / bandit-based Monte Carlo planning 和 MCTS survey。它们共同说明：LLM 不只能线性生成一条推理链，也可以作为 proposal model 生成候选状态，再由 value / verifier / tool feedback 选择、扩展、模拟和回传。

本章不把 ToT、beam search 或 MCTS 写成无条件更强的方法。搜索能提高复杂任务可靠性，前提是状态表示、动作粒度、评分器、停止条件和预算设计都合理。若 verifier 偏好“看起来更漂亮”的错误分支，搜索会更快放大错误；若搜索预算不受控，正确率提升也可能被延迟和成本吞掉。本章 demo 只做 toy 级审计，目标是帮助面试时讲清公式、指标、失败模式和工程门禁。

6.1 为什么需要 Search

普通 CoT 是一条路径：

question -> step 1 -> step 2 -> step 3 -> answer

复杂任务的问题在于，某一步选错后，后续推理可能全部建立在错误基础上。Search 的思想是不要只走一条路，而是保留多个候选路径，逐步探索、评分和筛选。

question

-> thought A -> thought A1 -> answer
-> thought B -> thought B1 -> answer
-> thought C -> pruned

面试回答：

Search 把推理过程看成状态空间探索。模型从当前状态生成多个候选下一步，用 verifier、reward model、规则或工具反馈评分，保

6.1.1 关键公式与 Search / ToT 指标速查

把搜索推理过程写成一个状态空间问题：

$s_t = (x, z_{1:t}, m_t, b_t)$

其中 x 是原始问题， $z_1, \dots, z_t$ 是已经生成的 thought 或步骤， m_t 是工具结果、测试结果、约束状态等元信息， b_t 是当前已消耗的预算。

动作集合表示当前状态下可选的下一步：

$a_t \in \mathcal{A}(s_t)$

状态转移表示执行动作后得到新状态：

$s_{t+1} = T(s_t, a_t)$

如果每层平均生成 B 个分支、最大深度为 D ，不剪枝的完整搜索树节点数近似为：

$$\begin{aligned} N_{\{\mathrm{full}\}} &= \sum_{d=0}^D B^d \\ &= \frac{B^{D+1} - 1}{B - 1} \end{aligned}$$

这就是 search reasoning 成本容易爆炸的根源。工程上通常使用 beam、阈值、去重、缓存和预算门禁限制实际节点数。

一个通用节点分数可以写成：

$$S(n) = \lambda_p S_{\mathrm{proc}}(n) + \lambda_o S_{\mathrm{out}}(n) + \lambda_u S_{\mathrm{tool}}(n) - \lambda_c C(n)$$

其中 S_{proc} 是过程分数, S_{out} 是终局或 outcome 分数, S_{tool} 是工具反馈分数, $C(n)$ 是 token、延迟、工具调用或风险成本。不同任务的权重不同: 代码题更依赖测试, 数学题更依赖过程和答案校验, 开放问答更依赖证据和人工评估。

Beam search 的层级更新可以写成:

$$F_{t+1} = \mathrm{TopK} \left(\{T(s, a) : s \in F_t, a \in \mathcal{A}(s)\}, S, K \right)$$

其中 F_t 是第 t 层 frontier, K 是 beam size。

MCTS / UCT 常用 “价值 + 探索奖励” 的选择分数:

$$U(v) = Q(v) + c \sqrt{\frac{\log(N_p + 1)}{N_v + 1}}$$

其中 $Q(v)$ 是节点平均价值, N_p 是父节点访问次数, N_v 是当前节点访问次数, c 控制探索强度。它的直觉是: 价值高的节点值得继续利用, 访问少但仍有潜力的节点也要给探索机会。

Search accuracy 衡量最终被搜索策略选出的答案是否正确:

$$A_{\mathrm{search}} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\hat{y}_i^{\mathrm{search}} = y_i^{\star}]$$

搜索成本可以按节点数、token 和工具调用分别统计:

$$C_{\mathrm{node}} = \sum_{i=1}^N \sum_{t=0}^{\infty} |D_i| F_{it}$$

$$C_{\mathrm{tok}} = \sum_{i=1}^N \sum_{n \in \mathcal{N}} T_n$$

如果搜索策略把正确路径提前剪掉, 记为 prune false negative:

$$R_{\mathrm{prune}} = \frac{\sum_i \mathbb{1}[\mathrm{correct\ path\ pruned}_i]}{N}$$

一个简化上线门禁可以写成:

$$G_{\mathrm{search}} = \mathbb{1}[A_{\mathrm{search}} \geq \alpha]$$

```

\land
R_{\mathrm{prune}}\leq \rho
\land
C_{\mathrm{tok}}\leq C_{\max}
\land
D_{\mathrm{cand}}\geq \delta
]

```

其中 D_{cand} 是候选多样性指标。面试里要强调：搜索不是“多想一会儿”这么简单，而是状态空间、评分函数、剪枝风险和预算约束共同决定效果。

6.2 状态、动作和转移

把推理建模为搜索，至少要定义五件事：

1. State：当前推理状态。
2. Action：下一步可选操作。
3. Transition：执行 action 后如何得到新状态。
4. Score：状态或动作的质量如何评估。
5. Termination：什么时候停止。

在数学题中：

```

state = 当前已写出的推理步骤
action = 下一步数学变换或子结论
transition = 把下一步加入推理链
score = verifier 对当前路径的评分
termination = 得到最终答案或达到深度上限

```

在代码题中：

```

state = 当前代码版本和测试结果
action = 修改代码或新增测试
transition = 应用修改并运行检查
score = 测试通过率、错误减少程度、代码质量评分
termination = 全部测试通过或预算耗尽

```

搜索能否有效，通常不是取决于“会不会展开树”，而是取决于状态表示是否完整、动作粒度是否合适、评分信号是否可靠。

6.3 Tree-of-Thought

Tree-of-Thought，简称 ToT，是把推理过程组织成一棵树。

普通 CoT 只有一条链，ToT 有多条分支。基本流程是：

1. 从原始问题出发。
2. 生成多个候选 thought。
3. 对每个 thought 或路径打分。
4. 选择若干分支继续展开。
5. 重复直到得到答案或达到预算上限。

这里的 thought 不一定是一句话，也不一定是一整段答案。它可以是一个中间结论、一个子问题分解、一个计划步骤、一段代码修改、一个数学变换。关键是它必须可评估、可继续展开。

粒度太小，搜索树会爆炸；粒度太大，无法定位和纠正错误。工程上常用“一个自然推理步骤”作为 thought 粒度。

6.4 Beam Search

Beam search 是最常见的搜索方法之一。它的做法是每一层只保留分数最高的 `beam_size` 个候选。

```
start
generate candidates
keep top B
expand each candidate
keep top B
repeat
```

优点：

1. 简单。
2. 成本可控。
3. 容易和 verifier 结合。
4. 适合分层展开的推理任务。

缺点：

1. 可能过早丢掉潜在正确路径。
2. 结果高度依赖评分函数。
3. 候选容易同质化。
4. 如果 beam 太大，成本仍然很高。

在 reasoning 场景中，beam search 常用于“生成多个下一步，再用 verifier 或 process reward model 选择前几个”。

6.5 Best-First Search

Best-first search 每次优先展开当前评分最高的节点。

它的直觉是：

哪个路径当前最有希望，就先探索哪个路径

当 verifier 分数比较可靠时，best-first search 可以快速深入高质量路径。但如果评分器偏差很大，它也可能一直沿着看似合理但实际错误的方向走。

Beam search 更像按层推进；best-first search 更像从全局候选池中挑最优节点推进。

6.6 MCTS 的直觉

MCTS, Monte Carlo Tree Search, 常用于游戏和规划。它通常包含四步：

1. Selection：选择要展开的节点。
2. Expansion：扩展新节点。
3. Simulation：模拟后续结果。
4. Backpropagation：把结果回传更新节点价值。

在 LLM reasoning 中，可以把模型生成的 thought 当成 action，把 verifier 分数、工具验证结果或最终答案正确性当成 reward。

MCTS 的优势是能在探索新路径和利用高分路径之间做平衡。它适合规划、博弈、代码修复、复杂数学推理等任务。但它的实现和调参比 beam search 更复杂，并且需要可靠 reward，否则回传的价值估计会被噪声污染。

6.7 Verifier 在搜索中的作用

Search 离不开评分。常见评分来源包括：

1. LLM 自评。
2. Outcome verifier。
3. Process verifier。
4. Programmatic verifier。
5. 规则 heuristic。

6. 工具执行反馈。

Process verifier 特别适合中间节点评分，因为它能判断当前步骤是否合理。Outcome verifier 更适合终局评分。Programmatic verifier 最可靠，但只适用于答案可自动检查的任务，例如代码测试、数学计算、格式校验。好的 verifier 可以显著减少无效搜索；差的 verifier 会误导搜索，把正确路径剪掉，把错误路径留下。

6.8 剪枝

剪枝是搜索中控制成本的关键。常见策略包括：

1. 分数低于阈值就丢弃。
2. 每层只保留 top-k。
3. 重复状态合并。
4. 超过最大深度停止。
5. 发现明显错误立即停止。
6. 工具验证失败则剪枝。
7. 达到 token 或延迟预算后停止。

剪枝太弱，成本爆炸；剪枝太强，可能丢掉正确路径。一个实用原则是：早期保留多样性，后期加强筛选。

6.9 搜索成本

如果每一步生成 b 个分支，深度是 d ，最坏情况下节点数量会随 b^d 增长。因此 search reasoning 不能无节制开启。

工程上通常限制：

1. 最大深度。
2. 每步候选数。
3. beam size。
4. 总 token budget。
5. verifier 调用次数。
6. 工具执行次数。
7. 总延迟。

Search 更适合高价值复杂任务，例如数学竞赛题、代码修复、规划、严肃问答和需要高可靠性的决策任务。对普通闲聊或简单问答，搜索的收益通常抵不过成本。

6.10 Search 和 Self-Consistency 的区别

Self-consistency 是独立生成多条完整推理路径，最后投票。中间过程不干预。

Search 是分步生成、分步评分、分步剪枝，可以回溯和继续探索。

self-consistency = 多条完整链 + 最后聚合

search = 多步展开 + 中间筛选 + 动态探索

Self-consistency 更简单，适合快速提升稳定性；search 更强，但工程复杂度和成本更高。

6.11 Search 和工具使用

工具可以作为搜索环境的一部分。

代码任务：

state = 当前代码

action = 修改代码

tool = 运行测试

score = 通过测试数量和错误变化

数学任务：

state = 当前推导

action = 写出下一步表达式

tool = Python 或符号计算器

score = 是否和约束一致

工具反馈让 search 更可靠，因为它提供外部校验。但也会带来超时、沙箱、安全、环境不一致和反馈稀疏等问题。

6.12 常见失败模式

1. 分支太多，成本爆炸。
2. Verifier 误剪正确路径。
3. LLM 生成的候选缺乏多样性。
4. 搜索陷入局部最优。
5. 状态表示不完整，导致评分失真。
6. 评分函数偏好格式而不是正确性。
7. 工具反馈不充分或不可用。
8. 最终答案整合错误。

很多失败不是搜索算法本身的问题，而是状态、动作、评分和预算设计的问题。

6.13 一个简单 ToT 伪代码

```
frontier = [(0.0, initial_state)]
```

```
for depth in range(max_depth):
    candidates = []
    for _, state in frontier:
        thoughts = generator.generate_next_thoughts(state, n=num_branches)
        for thought in thoughts:
            new_state = state.add(thought)
            score = verifier.score(new_state)
            candidates.append((score, new_state))

    candidates.sort(key=lambda item: item[0], reverse=True)
    frontier = candidates[:beam_size]

    for score, state in frontier:
        if state.is_finished():
            return state.final_answer()
```

```
return select_best_answer(frontier)
```

这个伪代码体现了展开、评分、剪枝和停止。真实系统还需要去重、缓存、失败重试、预算控制、工具调用和日志追踪。

6.14 什么时候值得用 Search

适合使用 search 的场景：

1. 解空间很大。
2. 存在多种解法。
3. 中间步骤可以被评分。
4. 错误路径能被较早发现。
5. 任务价值足以覆盖额外成本。

不适合的场景：

1. 简单问答。
2. 低延迟强约束任务。
3. 无法评估中间状态的开放生成。
4. Verifier 不可靠且没有工具反馈。
5. 用户只需要快速草稿。

6.15 最小可运行 search / ToT / MCTS 审计 demo

这个 demo 不实现真实 LLM 调用，而是模拟一批 toy reasoning 任务：每道题都有 greedy 答案、搜索分支、verifier 分数、MCTS 价值估计、访问次数和 token 成本。它展示三件事：

1. Search 能显著提高 greedy 之外的正确率。
2. Beam search 会被 high-score hard negative 误导。
3. MCTS 通过价值和探索项可能救回 beam 剪掉的正确路径，但仍要看成本和剪枝风险。

```
from math import log, sqrt
```

```
cases = [
    {
        "id": "water_jug",
        "gold": "4",
        "greedy": "4",
        "branches": [
            {"name": "wrong_volume", "answer": "5", "score": 0.45, "value": 0.20, "visits": 7, "tokens": 70, "correct": False},
            {"name": "state_search", "answer": "4", "score": 0.82, "value": 0.92, "visits": 6, "tokens": 110, "correct": True},
            {"name": "irrelevant_fact", "answer": "5", "score": 0.18, "value": 0.10, "visits": 3, "tokens": 40, "correct": False},
        ],
    },
    {
        "id": "logic_grid",
        "gold": "blue",
        "greedy": "red",
        "branches": [
            {"name": "row_elimination", "answer": "blue", "score": 0.79, "value": 0.86, "visits": 5, "tokens": 80, "correct": True},
            {"name": "column_guess", "answer": "blue", "score": 0.61, "value": 0.55, "visits": 4, "tokens": 95, "correct": False},
            {"name": "shortcut_guess", "answer": "red", "score": 0.35, "value": 0.22, "visits": 8, "tokens": 50, "correct": False},
        ],
    },
    {
        "id": "code_patch",
        "gold": "pass",
        "greedy": "fail",
        "branches": [
            {"name": "patch_edge_case", "answer": "pass", "score": 0.74, "value": 0.88, "visits": 7, "tokens": 90, "correct": True},
            {"name": "rewrite_all", "answer": "fail", "score": 0.41, "value": 0.35, "visits": 5, "tokens": 125, "correct": False},
            {"name": "add_sleep", "answer": "timeout", "score": 0.12, "value": 0.05, "visits": 2, "tokens": 60, "correct": False},
        ],
    },
    {
        "id": "hard_negative",
        "gold": "40",
        "greedy": "42",
        "branches": [
            {"name": "polished_wrong", "answer": "42", "score": 0.88, "value": 0.38, "visits": 10, "tokens": 100, "correct": False},
            {"name": "plain_correct", "answer": "40", "score": 0.62, "value": 0.78, "visits": 3, "tokens": 85, "correct": True},
        ],
    },
]
```

```

        {"name": "unit_checked", "answer": "40", "score": 0.59, "value": 0.71, "visits": 2, "tokens": 75, "correct": True},
    ],
},
]

```

```

def uct_score(branch, parent_visits, exploration=0.45):
    bonus = exploration * sqrt(log(parent_visits + 1) / (branch["visits"] + 1))
    return round(branch["value"] + bonus, 3)

```

```

def accuracy(flags):
    return round(sum(flags) / len(flags), 3)

```

```

beam_choices = []
mcts_choices = []
pruned_correct_paths = []
unique_ratios = []

```

```

for case in cases:
    parent_visits = sum(branch["visits"] for branch in case["branches"])
    beam_choice = max(case["branches"], key=lambda branch: branch["score"])
    mcts_choice = max(case["branches"], key=lambda branch: uct_score(branch, parent_visits))
    beam_choices.append({"id": case["id"], "choice": beam_choice["name"], "correct": beam_choice["correct"]})
    mcts_choices.append({"id": case["id"], "choice": mcts_choice["name"], "correct": mcts_choice["correct"]})
    if (not beam_choice["correct"]) and any(branch["correct"] for branch in case["branches"]):
        pruned_correct_paths.append(case["id"])
    unique_ratios.append(len({branch["answer"] for branch in case["branches"]}) / len(case["branches"]))

```

```

greedy_accuracy = accuracy([case["greedy"] == case["gold"] for case in cases])
beam_accuracy = accuracy([choice["correct"] for choice in beam_choices])
mcts_accuracy = accuracy([choice["correct"] for choice in mcts_choices])
avg_unique_answer_ratio = round(sum(unique_ratios) / len(unique_ratios), 3)
total_nodes_expanded = sum(1 + len(case["branches"]) for case in cases)
total_tokens = sum(branch["tokens"] for case in cases for branch in case["branches"])
cost_per_beam_correct = round(
    total_tokens / max(1, sum(choice["correct"] for choice in beam_choices)),
    3,
)

```

```

mcts_rescues = [
    beam["id"]
    for beam, mcts in zip(beam_choices, mcts_choices)
    if (not beam["correct"]) and mcts["correct"]
]

```

```

gates = {
    "beam_accuracy_ok": beam_accuracy >= 0.75,
    "mcts_accuracy_ok": mcts_accuracy >= 0.9,
    "diversity_ok": avg_unique_answer_ratio >= 0.7,
    "budget_ok": total_tokens <= 1000,
    "no_pruned_correct_path": len(pruned_correct_paths) == 0,
}
gate_pass = all(gates.values())

```

```

print(f"greedy_accuracy={greedy_accuracy}")
print(f"beam_accuracy={beam_accuracy}")
print(f"mcts_accuracy={mcts_accuracy}")
print(f"avg_unique_answer_ratio={avg_unique_answer_ratio}")
print(f"total_nodes_expanded={total_nodes_expanded}")
print(f"total_tokens={total_tokens}")
print(f"cost_per_beam_correct={cost_per_beam_correct}")
print(f"beam_choices={beam_choices}")
print(f"mcts_choices={mcts_choices}")
print(f"pruned_correct_paths={pruned_correct_paths}")
print(f"mcts_rescues={mcts_rescues}")
print(f"gates={gates}")
print(f"gate_pass={gate_pass}")

```

预期输出：

```

greedy_accuracy=0.25
beam_accuracy=0.75
mcts_accuracy=1.0
avg_unique_answer_ratio=0.75
total_nodes_expanded=16
total_tokens=980
cost_per_beam_correct=326.667
beam_choices=[{'id': 'water_jug', 'choice': 'state_search', 'correct': True}, {'id': 'logic_grid', 'choice': 'row_elimination', 'correct': True}]
mcts_choices=[{'id': 'water_jug', 'choice': 'state_search', 'correct': True}, {'id': 'logic_grid', 'choice': 'row_elimination', 'correct': True}]
pruned_correct_paths=['hard_negative']
mcts_rescues=['hard_negative']
gates={'beam_accuracy_ok': True, 'mcts_accuracy_ok': True, 'diversity_ok': True, 'budget_ok': True, 'no_pruned_correct_paths': True}
gate_pass=False

```

这个 `gate_pass=False` 是刻意设计的：beam 的平均准确率已经明显高于 greedy，但在 `hard_negative` 上把正确路径剪掉了。面试里可以借这个 demo 说明，搜索系统不能只报最终准确率，还要报告被剪掉的正确路径、候选多样性、MCTS 是否救回、token 成本和 `hard negative` 切片。

6.16 面试题：Tree-of-Thought 相比 CoT 的优势是什么

回答要点：

CoT 通常是一条推理链，容易受早期错误影响。Tree-of-Thought 把推理组织成树，允许模型生成多个中间 thought，并用评分器或规则进行筛选。

6.17 面试题：Beam Search 和 MCTS 的区别

回答要点：

Beam search 通常按层展开，每层保留 top-k 候选，简单、稳定、成本容易控制，但容易早剪正确路径。MCTS 会通过 selection、expansion、simulation 和 backup 进行探索。

6.18 小练习

1. 给一个数学题设计 state、action、transition、score 和 termination。
2. 写出 beam search 每层保留 top-k 的伪代码，并说明 beam size 对成本和错误剪枝的影响。
3. 用自己的话解释 UCT 公式中 value 和 exploration bonus 分别解决什么问题。
4. 给本章 demo 增加一个“beam 找到正确答案但 MCTS 因过度探索选错”的样本，观察 gate 如何变化。
5. 设计一张 search reasoning 上线审计表，至少包含 accuracy、pruned correct path、candidate diversity、token cost、tool calls 和 hard negative slice。

6.19 本章小结

Search reasoning 的核心是把推理从“一次生成一条答案”改成“在状态空间中探索多个候选路径”。Tree-of-Thought 提供了直观框架，beam search 提供了简单可控的实现，MCTS 提供了更强的规划能力，verifier 和工具反馈提供了剪枝依据。

真正落地时，最重要的不是把搜索树画出来，而是设计好状态、动作、评分、剪枝和预算。下一章会继续讨论 test-time compute scaling，也就是如何在推理阶段用更多计算换取更高质量答案。

第七章：Test-Time Compute Scaling

前面几章讲了 CoT、self-consistency、verifier、process supervision 和 search。它们背后都有一个共同现象：模型在推理阶段花更多计算，往往能得到更好的答案。Test-time compute scaling，推理时计算扩展，就是研究如何在推理阶段分配更多计算预算，并把这些预算转化为更高准确率、更强鲁棒性或更好可验证性的技术路线。

本章重点讲推理时计算扩展的动机、常见方式、预算分配、自适应计算、延迟和成本权衡、工程系统设计，以及面试中如何回答这类问题。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修参考公开资料中的 inference-time / test-time compute scaling 线索，重点包括 [Scaling LLM Test-Time Compute Optimally](#)、[Large Language Monkeys](#)、Tree of Thoughts、self-consistency、verifier reranking 和 OpenAI o1 system card 中关于推理时计算、reasoning 与安全评估的公开表述。这些资料共同说明：模型参数固定后，推理阶段的采样数、候选数、搜索深度、verifier 调用和工具反馈仍然可以显著改变任务表现。

本章不把“更多 test-time compute”写成无条件更强。推理时计算扩展必须同时看准确率、边际收益、延迟、成本、候选相关性、verifier 偏差、安全风险和任务价值。尤其要避免两种误区：第一，把长 CoT 或更多采样等同于真实推理能力；第二，只汇报高预算准确率，而不汇报单位正确样本成本、P95 延迟和低价值请求的预算浪费。

7.1 什么是 Test-Time Compute Scaling

传统 scaling law 主要关注训练阶段：参数更多、数据更多、训练 FLOPs 更多，模型能力通常更强。

Test-time compute scaling 关注的是另一件事：模型已经训练好了，在推理时能不能通过更多计算得到更好的答案。

例如：

普通推理：生成 1 条答案

更多推理计算：生成 32 条候选 + verifier 选择

搜索推理：逐步展开 + 剪枝 + 工具验证

长思考：允许模型生成更长 reasoning trace

面试回答：

Test-time compute scaling 指在模型参数固定的情况下，通过增加推理阶段计算来提升输出质量，例如更长思考、多样本采样、自一

7.1.1 关键公式与 TTC Scaling 指标速查

对第 i 个请求，把推理时预算写成一个向量：

$$\mathbf{b}_i = (K_i, L_i, D_i, V_i, U_i, T_i)$$

其中 K_i 是候选数或采样数， L_i 是允许的推理长度， D_i 是搜索深度， V_i 是 verifier 调用数， U_i 是工具调用数， T_i 是总 token 数。Test-time compute scaling 的核心不是让所有分量都变大，而是按任务价值和难度分配这些预算。

给定一种推理策略 m ，第 i 个请求的抽象成本可以写成：

$$C_i(m) = c_{\{\mathrm{tok}\}}T_i + c_{\{\mathrm{ver}\}}V_i + c_{\{\mathrm{tool}\}}U_i + c_{\{\mathrm{lat}\}}R_i$$

其中 R_i 是用户可感知延迟或服务占用时间。不同业务可以调整各项权重，例如交互式产品更重视 R_i ，批处理评测更重视总 token 和 verifier 调用。

固定预算下的准确率曲线可以写成：

$$A(B) = \frac{1}{\sum_{i=1}^N} [\hat{y}_i(B) = y_i^*]$$

边际收益衡量多花一段预算换来多少准确率提升：

$$g(B_1, B_2) = \frac{A(B_2) - A(B_1)}{C(B_2) - C(B_1)}$$

如果每个候选独立命中正确解的概率粗略为 p_i ，生成 K 个候选时“至少出现一个正确候选”的理想化概率为：

$$P_i(K) = 1 - (1 - p_i)^K$$

这个公式只表达直觉：更多候选提高覆盖正确解的概率，但真实候选高度相关，增长会更快进入边际收益递减。

并行候选生成和串行搜索的延迟口径不同。一个简化并行口径是：

$$R_i^{\{\mathrm{parallel}\}} = \max_{1 \leq k \leq K_i} R_{\{ik\}} + R_i^{\{\mathrm{verify}\}}$$

搜索或工具循环更接近串行：

$$R_i^{\{\mathrm{loop}\}} = \sum_{t=1}^{H_i} (R_{\{it\}}^{\{\mathrm{gen}\}} + R_{\{it\}}^{\{\mathrm{tool}\}} + R_{\{it\}}^{\{\mathrm{verify}\}})$$

因此同样的 token 数，在用户体验上可能完全不同。

自适应计算可以写成一个路由函数：

$$m_i = \pi(x_i, d_i, v_i, u_i)$$

其中 d_i 是难度估计， v_i 是任务价值， u_i 是可验证性或工具可用性。路由函数决定该请求走 direct、self-consistency、verifier、search 还是 tool loop。

一个简化上线门禁：

$$G_{\{\mathrm{ttc}\}} = \mathbb{1} [A_{\{\mathrm{adapt}\}} \geq \alpha \wedge C_{\{\mathrm{adapt}\}} \leq C_{\{\mathrm{max}\}} \wedge R_{95} \leq R_{\{\mathrm{max}\}} \wedge W_{\{\mathrm{waste}\}} \leq \omega]$$

其中 W_{waste} 表示高预算但仍失败、或者低价值请求消耗高预算的浪费率。面试里要强调：TTC scaling 的目标不是“所有请求都更慢”，而是把额外计算投给低置信度、高价值、可验证的请求。

7.2 为什么推理时计算有用

很多复杂任务不是模型完全不会，而是第一次采样不一定走到正确路径。增加推理计算可以带来几类收益：

1. 提高探索范围。
2. 降低单次采样偶然性。
3. 给 verifier 更多候选可选。
4. 允许模型检查和修正错误。
5. 让工具反馈参与决策。
6. 对困难样本投入更多预算。

一个直观例子是数学题。模型一次生成可能犯错，但生成多条推理路径后，其中可能有一条正确。如果有可靠 verifier 或最终答案投票，就能选出更好的答案。

7.3 常见推理时扩展方式

常见方法包括：

1. Long CoT：允许模型生成更长推理。
2. Self-consistency：采样多条推理链并聚合。
3. Best-of-N：生成 N 个候选，用评分器选择。
4. Verifier reranking：用 verifier 重排候选。
5. Tree-of-Thought：逐步展开推理树。
6. MCTS：用树搜索估计路径价值。
7. Reflection：先回答，再自查并修正。
8. Tool-use loops：生成、执行工具、观察反馈、继续推理。

这些方法本质上都是用更多推理计算换更高答案质量，只是计算花在不同位置。

7.4 Best-of-N

Best-of-N 是最简单的 test-time scaling 方法。

流程：

输入问题

生成 N 个候选答案

给每个候选打分

选择最高分答案

它的优点是实现简单、并行友好、容易扩展。缺点是需要评分器。如果评分器不可靠，N 越大可能越容易选到“看起来好但实际错”的答案。

Best-of-N 的效果通常取决于两个因素：候选中是否存在正确答案，以及评分器是否能识别它。

7.5 Self-Consistency

Self-consistency 也是一种推理时计算扩展。它生成多条推理路径，然后对最终答案投票。

适用条件：

1. 最终答案形式比较明确。
2. 多条路径可以独立采样。
3. 正确答案在采样中出现概率不低。
4. 错误答案分散，正确答案更容易聚集。

它不需要训练额外 verifier，但对开放式问答、创意写作、主观任务不一定适用。

7.6 Verifier Reranking

Verifier reranking 是 reasoning model 中非常常见的推理时扩展方法。

基本流程：

generator 生成多个候选
verifier 给候选评分
系统返回分数最高的候选

如果 verifier 是 outcome verifier，它主要看最终答案是否正确。如果 verifier 是 process verifier，它还可以看中间步骤是否合理。

Reranking 的关键不是生成更多文本，而是让评分器参与选择。很多时候，generator 已经能生成正确答案，但单次采样不稳定；reranking 可以把正确候选挑出来。

7.7 Adaptive Compute

所有问题都用同样推理预算并不合理。简单问题一次回答即可，困难问题才需要更多计算。Adaptive compute 的目标是根据样本难度动态分配预算。

常见策略：

1. 先用低成本模式回答。
2. 如果置信度低，再增加采样。
3. 如果候选分歧大，调用 verifier。
4. 如果 verifier 仍不确定，启动搜索。
5. 如果工具可验证，调用工具。
6. 达到预算上限后停止。

一个实用流程：

easy query -> direct answer
medium query -> self-consistency
hard query -> search + verifier + tools

Adaptive compute 的难点是如何判断“这题难不难”和“现在是否应该继续花钱”。

7.8 置信度估计

自适应计算依赖置信度估计。常见信号包括：

1. 模型 log probability。
2. 多样本答案一致性。
3. Verifier 分数。
4. 候选之间的分歧程度。
5. 工具验证结果。
6. 历史错误模式。
7. 输入任务类型。

但这些信号都不完美。模型可能高置信度地犯错，verifier 也可能被表面合理的答案欺骗。因此工程系统通常会组合多个信号，而不是只依赖单一置信度。

7.9 Latency Trade-off

推理时计算扩展最直接的代价是延迟。

如果生成一个答案需要 2 秒，生成 16 个候选即使并行，也会带来更高排队、显存、吞吐和 verifier 成本。如果还要搜索和工具调用，延迟可能进一步上升。

工程上要区分：

1. 用户可感知延迟。
2. 后台总计算成本。
3. GPU 吞吐损失。
4. 工具调用等待时间。
5. verifier 服务延迟。
6. 超时和重试成本。

高质量模式可以慢，但不能不可控。系统必须设置硬预算和降级策略。

7.10 Cost Trade-off

推理计算扩展也会显著增加成本。

成本来源：

1. 更多输出 token。
2. 多次模型调用。
3. Verifier 调用。
4. 工具执行。
5. 搜索树展开。
6. 缓存和日志存储。

业务上需要回答：额外准确率是否值得额外成本。对于医疗、法律、代码修复、金融分析等高价值任务，更多推理计算可能值得；对于简单客服问答，可能不值得。

7.11 Budget Allocation

预算分配比“预算越多越好”更重要。

常见分配方式：

1. 增加候选数量。
2. 增加每条推理长度。
3. 增加搜索深度。
4. 增加 verifier 精度。
5. 增加工具调用。
6. 增加反思轮数。

不同任务适合不同分配方式。数学题可能更适合多路径采样和 verifier；代码题可能更适合工具执行和迭代修复；规划任务可能更适合搜索和约束检查。

7.12 Diminishing Returns

推理时计算通常存在边际收益递减。N 从 1 增加到 8，提升可能明显；N 从 64 增加到 128，提升可能很小。

原因包括：

1. 候选开始重复。
2. Verifier 成为瓶颈。
3. 模型能力上限限制。
4. 错误模式高度相关。
5. 任务本身缺少可验证信号。

因此系统应该测量不同预算下的质量曲线，而不是盲目增加推理次数。

7.13 工程系统设计

一个支持 test-time compute scaling 的系统通常包含：

1. Router：判断任务类型和难度。

2. Generator: 生成候选或下一步。
3. Verifier: 评估候选质量。
4. Search Controller: 控制展开、剪枝和停止。
5. Tool Executor: 执行外部工具。
6. Budget Manager: 管理 token、时间、调用次数。
7. Aggregator: 聚合候选并输出最终答案。
8. Logger: 记录路径、评分和失败原因。

这类系统的核心不是单个 prompt，而是一个受预算约束的推理控制器。

7.14 常见失败模式

1. 花了更多计算，但候选高度重复。
2. Verifier 选错，把错误答案排第一。
3. 搜索过深，延迟不可控。
4. 低价值请求消耗高预算。
5. 置信度估计错误，难题被当成简单题。
6. 反思轮数增加，但模型只是在重复原错误。
7. 工具调用失败后系统没有降级。
8. 日志不完整，无法分析预算浪费在哪里。

推理时计算扩展不是“多生成几次”这么简单，而是要把额外计算用在能提升正确率的位置。

7.15 最小可运行 TTC scaling / 动态预算审计 demo

这个 demo 模拟 6 个 toy 请求，在四种固定策略和一个 adaptive routing 策略之间比较：

1. direct: 低预算直接回答。
2. self_consistency: 多样本采样与聚合。
3. verifier: 候选生成加 verifier。
4. search: 搜索加 verifier / tool feedback。
5. adaptive: 按难度、任务价值和可验证性路由。

它刻意保留 adversarial_math: adaptive 给它分配了高于 direct 的预算，但仍然失败，所以 gate_pass=False。这个失败用来提醒读者：动态预算系统不能只看平均准确率，也要看高算力浪费和 hard slice。

```
CASES = [
    {
        "id": "easy_lookup",
        "difficulty": 0.15,
        "value": 0.20,
        "verifiable": False,
        "modes": {
            "direct": {"correct": True, "tokens": 32, "verifier_calls": 0, "tool_calls": 0, "latency_ms": 120},
            "self_consistency": {"correct": True, "tokens": 180, "verifier_calls": 0, "tool_calls": 0, "latency_ms": 120},
            "verifier": {"correct": True, "tokens": 240, "verifier_calls": 3, "tool_calls": 0, "latency_ms": 430},
            "search": {"correct": True, "tokens": 520, "verifier_calls": 5, "tool_calls": 0, "latency_ms": 900},
        },
    },
    {
        "id": "factual_short",
        "difficulty": 0.25,
        "value": 0.30,
        "verifiable": False,
        "modes": {
            "direct": {"correct": True, "tokens": 36, "verifier_calls": 0, "tool_calls": 0, "latency_ms": 130},
            "self_consistency": {"correct": True, "tokens": 210, "verifier_calls": 0, "tool_calls": 0, "latency_ms": 130},
            "verifier": {"correct": True, "tokens": 260, "verifier_calls": 3, "tool_calls": 0, "latency_ms": 460},
        },
    },
]
```

```

        "search": {"correct": True, "tokens": 540, "verifier_calls": 5, "tool_calls": 0, "latency_ms": 920},
    },
    {
        "id": "algebra_hard",
        "difficulty": 0.62,
        "value": 0.80,
        "verifiable": True,
        "modes": {
            "direct": {"correct": False, "tokens": 90, "verifier_calls": 0, "tool_calls": 0, "latency_ms": 220},
            "self_consistency": {"correct": True, "tokens": 520, "verifier_calls": 0, "tool_calls": 0, "latency_ms": 220},
            "verifier": {"correct": True, "tokens": 430, "verifier_calls": 4, "tool_calls": 0, "latency_ms": 620},
            "search": {"correct": True, "tokens": 760, "verifier_calls": 6, "tool_calls": 0, "latency_ms": 1180},
        },
    },
    {
        "id": "code_patch",
        "difficulty": 0.90,
        "value": 0.95,
        "verifiable": True,
        "modes": {
            "direct": {"correct": False, "tokens": 120, "verifier_calls": 0, "tool_calls": 0, "latency_ms": 260},
            "self_consistency": {"correct": False, "tokens": 650, "verifier_calls": 0, "tool_calls": 0, "latency_ms": 260},
            "verifier": {"correct": False, "tokens": 610, "verifier_calls": 5, "tool_calls": 0, "latency_ms": 820},
            "search": {"correct": True, "tokens": 940, "verifier_calls": 5, "tool_calls": 2, "latency_ms": 1420},
        },
    },
    {
        "id": "logic_puzzle",
        "difficulty": 0.78,
        "value": 0.70,
        "verifiable": False,
        "modes": {
            "direct": {"correct": False, "tokens": 100, "verifier_calls": 0, "tool_calls": 0, "latency_ms": 240},
            "self_consistency": {"correct": True, "tokens": 560, "verifier_calls": 0, "tool_calls": 0, "latency_ms": 240},
            "verifier": {"correct": True, "tokens": 620, "verifier_calls": 5, "tool_calls": 0, "latency_ms": 860},
            "search": {"correct": True, "tokens": 980, "verifier_calls": 8, "tool_calls": 0, "latency_ms": 1500},
        },
    },
    {
        "id": "adversarial_math",
        "difficulty": 0.82,
        "value": 0.60,
        "verifiable": True,
        "modes": {
            "direct": {"correct": False, "tokens": 110, "verifier_calls": 0, "tool_calls": 0, "latency_ms": 240},
            "self_consistency": {"correct": False, "tokens": 620, "verifier_calls": 0, "tool_calls": 0, "latency_ms": 240},
            "verifier": {"correct": False, "tokens": 590, "verifier_calls": 5, "tool_calls": 0, "latency_ms": 840},
            "search": {"correct": False, "tokens": 900, "verifier_calls": 7, "tool_calls": 0, "latency_ms": 1450},
        },
    },
]

```

```

TOKEN_COST = 1.0
VERIFIER_COST = 45.0

```

```
TOOL_COST = 100.0
```

```
def unit_cost(stats):
    return round(
        TOKEN_COST * stats["tokens"]
        + VERIFIER_COST * stats["verifier_calls"]
        + TOOL_COST * stats["tool_calls"],
        3,
    )

def choose_route(case):
    if case["difficulty"] <= 0.35:
        return "direct"
    if case["verifiable"] and case["value"] >= 0.90:
        return "search"
    if case["verifiable"] and case["difficulty"] <= 0.70:
        return "verifier"
    if case["difficulty"] >= 0.60:
        return "self_consistency"
    return "direct"

def summarize(mode):
    chosen = [case["modes"][mode] for case in CASES]
    correct = sum(item["correct"] for item in chosen)
    cost = sum(unit_cost(item) for item in chosen)
    latencies = sorted(item["latency_ms"] for item in chosen)
    return {
        "accuracy": round(correct / len(chosen), 3),
        "total_cost": round(cost, 3),
        "cost_per_correct": round(cost / max(1, correct), 3),
        "p95_latency_ms": latencies[-1],
    }
```

```
fixed_modes = ["direct", "self_consistency", "verifier", "search"]
fixed_summary = {mode: summarize(mode) for mode in fixed_modes}
```

```
adaptive_records = []
for case in CASES:
    route = choose_route(case)
    stats = case["modes"][route]
    adaptive_records.append(
        {
            "id": case["id"],
            "route": route,
            "correct": stats["correct"],
            "cost": unit_cost(stats),
            "latency_ms": stats["latency_ms"],
        }
    )

adaptive_correct = sum(item["correct"] for item in adaptive_records)
```

```

adaptive_cost = sum(item["cost"] for item in adaptive_records)
adaptive_latencies = sorted(item["latency_ms"] for item in adaptive_records)
adaptive_summary = {
    "accuracy": round(adaptive_correct / len(adaptive_records), 3),
    "total_cost": round(adaptive_cost, 3),
    "cost_per_correct": round(adaptive_cost / max(1, adaptive_correct), 3),
    "p95_latency_ms": adaptive_latencies[-1],
}

marginal = {}
base_acc = fixed_summary["direct"]["accuracy"]
base_cost = fixed_summary["direct"]["total_cost"]
for mode in ["self_consistency", "verifier", "search"]:
    acc_gain = fixed_summary[mode]["accuracy"] - base_acc
    cost_gain = fixed_summary[mode]["total_cost"] - base_cost
    marginal[mode] = round(acc_gain / cost_gain, 6) if cost_gain else 0.0

wasted_high_compute = [
    item["id"]
    for item in adaptive_records
    if item["route"] != "direct" and not item["correct"]
]

gates = {
    "adaptive_accuracy_ok": adaptive_summary["accuracy"] >= 0.8,
    "adaptive_cheaper_than_search": adaptive_summary["total_cost"] < fixed_summary["search"]["total_cost"],
    "latency_ok": adaptive_summary["p95_latency_ms"] <= 1500,
    "wasted_high_compute_ok": len(wasted_high_compute) == 0,
}

print(f"fixed_summary={fixed_summary}")
print(f"adaptive_records={adaptive_records}")
print(f"adaptive_summary={adaptive_summary}")
print(f"marginal_accuracy_per_cost={marginal}")
print(f"wasted_high_compute={wasted_high_compute}")
print(f"gates={gates}")
print(f"gate_pass={all(gates.values())}")

```

预期输出：

```

fixed_summary={'direct': {'accuracy': 0.333, 'total_cost': 488.0, 'cost_per_correct': 244.0, 'p95_latency_ms': 260}, 'search': {'accuracy': 0.833, 'total_cost': 3223.0, 'cost_per_correct': 644.6, 'p95_latency_ms': 1420}}
adaptive_records=[{'id': 'easy_lookup', 'route': 'direct', 'correct': True, 'cost': 32.0, 'latency_ms': 120}, {'id': 'hard_slice_detector', 'route': 'search', 'correct': False, 'cost': 1500.0, 'latency_ms': 1500}]
adaptive_summary={'accuracy': 0.833, 'total_cost': 3223.0, 'cost_per_correct': 644.6, 'p95_latency_ms': 1420}
marginal_accuracy_per_cost={'self_consistency': 0.000148, 'verifier': 9.9e-05, 'search': 8.4e-05}
wasted_high_compute=['adversarial_math']
gates={'adaptive_accuracy_ok': True, 'adaptive_cheaper_than_search': True, 'latency_ok': True, 'wasted_high_compute_ok': False}
gate_pass=False

```

这个输出说明：固定 search 准确率最高，但成本也最高；adaptive routing 用接近一半的成本达到同样的准确率。不过 adversarial_math 暴露了一个上线前必须处理的问题：系统识别到它难，却没有选对有效策略。真实工程里要继续补 hard-slice detector、更强 verifier、工具校验或人工复核，而不是只看平均准确率达标。

7.16 面试题：什么是 Test-Time Compute Scaling

回答要点：

Test-time compute scaling 是指模型参数固定时，通过增加推理阶段计算来提升答案质量。典型方法包括长 CoT、多样本采样、self

consistency、best-of-N、verifier reranking、tree search、工具调用和反思修正。它的关键挑战是预算分配和成本控制，因为更

7.17 面试题：如何设计 Adaptive Compute

回答要点：

我会先设计一个任务路由和置信度估计模块。简单任务直接回答；中等难度任务使用多样本采样或 self-consistency；高难度任务启

7.18 小练习

1. 给一个在线问答产品设计 direct、self-consistency、verifier 和 search 四档预算。
2. 写出你自己的 TTC 成本函数，至少包含 token、verifier 调用、工具调用和延迟。
3. 修改本章 demo，让 adversarial_math 走 search，观察准确率、成本和 gate 如何变化。
4. 设计一个 hard-slice 报告，说明哪些题应该升级到人工复核而不是继续增加采样数。
5. 用 3 句话解释为什么 test-time compute scaling 会有边际收益递减。

7.19 本章小结

Test-time compute scaling 是 reasoning model 的重要方向。它说明模型能力不只来自训练阶段，也来自推理阶段如何使用计算。Long CoT、self-consistency、best-of-N、verifier reranking、search、tools 和 reflection 都是推理时扩展的具体形式。

真正的工程问题是：哪些请求值得更多计算，额外预算应该用于采样、搜索、验证还是工具，什么时候停止，以及如何在质量、成本和延迟之间取得平衡。下一章会进入数学推理训练，讨论 reasoning model 在数学任务上如何构造数据、训练和评估。

第八章：数学推理训练

数学推理是 reasoning model 最重要的训练场景之一。原因很直接：数学题目标明确、步骤可分析、答案常常可验证，适合研究长思维链、过程监督、verifier、搜索和 test-time compute scaling。很多 reasoning 模型能力的提升，都会先在数学基准上体现出来。

本章系统讲数学推理训练：数学数据从哪里来，为什么只看最终答案不够，如何构造步骤数据和合成题，verifier 如何参与训练与筛选，难度 curriculum 如何设计，以及数学推理训练常见风险和面试表达。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修参考公开数学 reasoning 资料，包括 [GSM8K / Training Verifiers to Solve Math Word Problems](#)、[MATH dataset](#)、[Let's Verify Step by Step / PRM800K](#)、[DeepSeekMath](#)、[OpenWebMath](#)、[AlphaGeometry](#) 和 [NuminaMath](#) 等资料。它们共同提示：数学推理能力不是单一 prompt 技巧，而是由数学语料、步骤数据、合成数据、答案验证、过程监督、verifier、curriculum、RL / rejection sampling 和严格评估共同组成的训练闭环。

本章不把数学 benchmark 分数写成通用 reasoning 能力的充分证据。数学题常常更封闭、更可验证，也更容易被公开题库、题解和合成模板污染。第二轮精修重点是把“训练数学推理”拆成可审计对象：样本来源是否干净、答案是否可验证、步骤是否真的正确、错误样本是否能定位第一处错误、合成题是否模板化、难题切片是否改善、训练成本和 test-time compute 是否值得。

8.1 为什么数学适合训练 Reasoning

数学任务有几个天然优势：

1. 答案通常明确。
2. 推理过程有结构。
3. 错误步骤较容易定位。
4. 可以构造大量合成题。

5. 很多题可以自动验证。
6. 难度可以分层。

例如一道代数题，模型不仅要给出最终数值，还要完成变形、代入、化简、检查条件等步骤。这些步骤非常适合训练长链条推理。

面试回答：

数学是 reasoning model 的重要训练场景，因为数学问题有明确答案、推理链条较清晰、部分任务可以程序化验证，还能通过合成题 time search。但数学能力不能完全代表通用推理能力，因为现实任务往往更开放、约束更模糊。

8.1.1 关键公式与数学推理训练指标速查

把第 i 道数学训练样本写成：

$$m_i = (x_i, y_i^{\star}, z_i, q_i, d_i, t_i, s_i, r_i)$$

其中 x_i 是题目, $y_i^{\star}$ 是标准答案, $z_i = (z_{i1}, \dots, z_{iL_i})$ 是解题步骤, $q_i = (q_{i1}, \dots, q_{iL_i})$ 是步骤标签, d_i 是难度, t_i 是题型, s_i 是来源或模板标识, r_i 是污染、重复、验证失败等风险标记。

只做最终答案监督时，简化目标可以写成：

$$L_{\{\mathrm{ans}\}}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log P_{\theta}(y_i^{\star} \mid x_i, z_i)$$

如果训练时只给题目和最终答案，就把条件中的 z_i 去掉。这个目标容易落地，但它不知道哪一步错，也无法区分“正确推理后算错”和“乱推但猜对”。

过程监督把第 j 步是否正确建模为 q_{ij} ：

$$L_{\{\mathrm{proc}\}}(\phi) = -\frac{1}{M} \sum_{i,j} \left[q_{ij} \log \hat{q}_{ij} + (1 - q_{ij}) \log (1 - \hat{q}_{ij}) \right]$$

其中 $\hat{q}_{ij}$ 是 verifier 或 PRM 对第 j 步正确性的预测， M 是被标注的步骤总数。训练时要单独记录自动标注和人工标注比例，因为过程标签很贵。

最终答案准确率：

$$A_{\{\mathrm{ans}\}} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\hat{y}_i = y_i^{\star}]$$

步骤准确率：

$$A_{\{\mathrm{step}\}} = \frac{\sum_{i,j} \mathbb{1}[\hat{q}_{ij} = q_{ij}]}{\sum_i L_i}$$

第一处错误定位准确率：

$$A_{\{\mathrm{first}\}} = \frac{1}{N_{\{\mathrm{err}\}}}$$

$$\sum_{i \in \mathcal{E}} \mathbb{1}[\hat{e}_i = e_i^*]$$

其中 $\mathcal{E}$ 是存在错误步骤的样本集合, e_i^* 是人工或高可信 verifier 标出的第一处错误位置。

自动验证覆盖率:

$$C_{\{\mathrm{verify}\}} = \frac{1}{\sum_{i=1}^N \mathbb{1}[v_i=1]}$$

这里 $v_i=1$ 表示这道题至少可以用答案归一化、数值代入、符号化简、穷举、单元测试或规则检查中的一种方法验证。覆盖率高不代表证明完整, 只表示能拿到一个较硬的自动检查信号。

训练污染率:

$$R_{\{\mathrm{contam}\}} = \frac{1}{\sum_{i=1}^N \mathbb{1}[o_i=1]}$$

其中 $o_i=1$ 表示样本和评测集题面、答案、解析或高度相似变体存在重叠风险。数学推理训练尤其要监控这个指标, 因为公开题库和题解非常容易进入预训练或 SFT 数据。

合成题模板多样性:

$$D_{\{\mathrm{temp}\}} = \frac{|\{s_i\}_{i=1}^N|}{N}$$

如果大量样本来自同一个模板, 模型可能学会模板模式, 而不是稳健推理。模板多样性要和题型覆盖、难度覆盖一起看。

某个难度切片的准确率:

$$A_g = \frac{\sum_{i \in \mathcal{G}_g} \mathbb{1}[\hat{y}_i = y_i^*]}{|\mathcal{G}_g|}$$

其中 $\mathcal{G}_g$ 可以是 easy、medium、hard, 也可以是 algebra、geometry、proof、combinatorics 等题型切片。面试中要强调: 数学训练最怕平均分提升, 但 hard slice、证明题或几何题完全没改善。

一个简化数学推理训练门禁:

$$G_{\{\mathrm{math}\}} = \mathbb{1}[A_{\{\mathrm{ans}\}} \geq \alpha \land A_{\{\mathrm{step}\}} \geq \beta \land A_{\{\mathrm{first}\}} \geq \gamma \land R_{\{\mathrm{contam}\}} \leq \rho \land D_{\{\mathrm{temp}\}} \geq \delta \land A_{\{\mathrm{hard}\}} \geq \eta]$$

这个门禁表达的是训练数据和模型效果的共同要求：答案要对，步骤要可靠，错误要能定位，评测不能污染，合成题不能过度模板化，难题切片不能被平均分掩盖。

8.2 数学数据类型

数学训练数据可以分为几类：

1. 题目和最终答案。
2. 题目、解题步骤和最终答案。
3. 多种解法数据。
4. 错误解法和纠错数据。
5. 难度分级数据。
6. 合成题数据。
7. 工具验证数据。

只包含最终答案的数据可以训练模型输出答案，但对推理路径帮助有限。包含步骤的数据更适合训练模型如何思考。包含错误步骤的数据可以训练模型识别和修正错误。

8.3 最终答案监督的局限

Outcome supervision 只看答案是否正确。

它的问题是：

1. 不知道哪一步错。
2. 无法奖励部分正确推理。
3. 可能奖励碰巧猜对的答案。
4. 难以训练中间步骤质量。
5. 对长链条问题信号稀疏。

例如模型前面推理全对，最后算错一个符号，最终答案错误。只看最终答案会把整条链都当成失败样本。反过来，模型中间推理乱写，但最后猜对答案，也可能被错误奖励。

这就是为什么数学推理训练通常需要 CoT 数据、process supervision 或 verifier。

8.4 解题步骤数据

高质量数学步骤数据通常包含：

1. 明确的问题陈述。
2. 分步推导。
3. 每一步的理由。
4. 最终答案。
5. 必要的条件检查。
6. 可选的替代解法。

一个好的步骤不是越长越好，而是每一步都承担推理功能。

常见坏步骤包括：

1. 跳步严重。
2. 只写结论不写理由。
3. 格式很漂亮但逻辑错误。
4. 中间变量未定义。
5. 最终答案和推导不一致。
6. 过度冗长，堆砌无用解释。

训练 reasoning model 时，步骤质量比步骤数量更重要。

8.5 合成题

合成题是数学推理训练的重要数据来源。它可以通过程序、模板、模型生成或人机协作构造。

优点：

1. 数量大。
2. 难度可控。
3. 答案可自动计算。
4. 可以覆盖稀有题型。
5. 便于构造 curriculum。

风险：

1. 模板痕迹明显。
2. 分布和真实考试题不一致。
3. 题目过于机械。
4. 模型学会题型模式而不是推理。
5. 合成答案或步骤可能有错。

因此合成题通常需要过滤、去重、难度分层和验证。

8.6 难度 Curriculum

Curriculum learning 的思想是从简单到困难训练。

数学推理中可以按以下维度分难度：

1. 步骤长度。
2. 概念数量。
3. 是否需要分类讨论。
4. 是否需要构造辅助变量。
5. 是否需要证明而不是计算。
6. 是否需要跨章节知识。
7. 是否容易被自动验证。

一个简单 curriculum：

基础计算 -> 代数变形 -> 应用题 -> 几何证明 -> 组合计数 -> 奥赛难题

课程学习的价值是让模型先学稳定的基础推理模式，再逐步接触更长、更抽象、更难验证的问题。

8.7 Verifier 在数学训练中的作用

Verifier 可以用于多个环节：

1. 数据清洗：过滤错误答案。
2. 步骤评分：判断中间推理是否合理。
3. 候选选择：从多个解法中选更好的一条。
4. RL reward：作为强化学习奖励信号。
5. 推理时 reranking：选择最终输出。
6. 搜索剪枝：提前丢弃错误路径。

数学 verifier 有几种形态：

1. 规则验证器。
2. 符号计算验证器。
3. 程序执行验证器。
4. LLM verifier。
5. Process reward model。

程序化 verifier 最可靠，但覆盖面有限。LLM verifier 覆盖面广，但可能被表面合理的错误推导欺骗。

8.8 自动验证和符号计算

很多数学问题可以用工具验证，例如：

1. 数值计算。
2. 方程求解。
3. 表达式化简。
4. 代入检查。
5. 穷举验证。
6. 随机测试。

例如代数恒等式可以随机代入多个数检查；组合题可以在小规模上穷举验证；函数题可以用采样检查反例。

但自动验证不能替代完整证明。一个表达式在若干点上成立，不代表恒等成立；一个数值答案正确，不代表推导过程正确。数学训练需要区分“答案验证”和“过程证明”。

8.9 多解法训练

同一道题可能有多种解法。多解法数据有助于模型学习不同思路。

例如：

1. 代数法。
2. 几何法。
3. 构造法。
4. 反证法。
5. 归纳法。
6. 穷举和观察模式。

多解法训练能提升模型的探索能力，也有利于 test-time search。因为搜索时模型需要生成不同方向，而不是多次生成同一种解法。

8.10 错误解法和纠错数据

只训练正确解法不够。Reasoning model 还需要学会发现错误。

错误数据可以包括：

1. 符号错误。
2. 条件遗漏。
3. 除以可能为零的量。
4. 分类讨论不完整。
5. 偷换概念。
6. 证明中使用未证结论。
7. 计算中间步骤错误。

训练形式可以是：

给定题目和错误解法 -> 找出第一处错误 -> 解释原因 -> 给出修正

这类数据对 verifier、process supervision 和反思修正都很有价值。

8.11 RL 在数学推理中的使用

数学任务适合做 RL 的原因是 reward 相对容易定义。最终答案正确可以给 reward，步骤正确也可以给 reward，工具验证通过也可以给 reward。

常见做法：

1. 用 SFT 训练基础解题能力。
2. 采样多条解法。
3. 用 verifier 或答案检查打分。

4. 用 RL 提高高 reward 解法概率。
5. 结合长度、格式和安全约束。

风险是模型可能 reward hacking。例如学会输出 verifier 偏好的格式，或者通过猜测最终答案拿分，而不是形成真实推理能力。

8.12 数学推理评估

评估不能只看一个 benchmark 分数。常见维度包括：

1. 最终答案准确率。
2. 步骤正确率。
3. 第一处错误位置。
4. 不同难度层级表现。
5. 不同题型表现。
6. 多次采样稳定性。
7. 推理长度和成本。
8. 工具验证通过率。
9. 对变体题的泛化能力。

如果一个模型在训练分布题型上很强，但换个表述就失败，说明它可能学到了模式匹配，而不是稳健推理。

8.13 数据泄漏风险

数学 benchmark 常有数据泄漏风险。模型可能在预训练中见过题目、答案或相似解析。

风险来源：

1. 公开题库。
2. 论坛解析。
3. 竞赛题解。
4. 训练集和测试集重叠。
5. 合成题模板重复。

缓解方式：

1. 去重。
2. 使用新题。
3. 改写题面并检查稳定性。
4. 构造参数化变体。
5. 分析错误类型而不只看准确率。

8.14 常见失败模式

1. 只会套模板，不会处理变体。
2. 推理步骤很长，但核心逻辑错误。
3. 答案正确，过程不可靠。
4. 过程看似漂亮，但跳过关键证明。
5. Verifier 被格式欺骗。
6. 合成题分布太窄。
7. Curriculum 太简单，迁移不到难题。
8. 过度优化 benchmark。

数学推理训练的目标不是让模型背题，而是让模型掌握可迁移的推理操作。

8.15 最小可运行数学推理训练数据审计 demo

这个 demo 模拟 6 条 toy 数学训练样本，覆盖答案正确性、步骤标签、自动标注、人审成本、污染、模板重复、难度切片和 verifier 误判。它不是训练模型，而是演示上线一个数学推理训练集前应该先做哪些审计。

```
from collections import Counter, defaultdict
```

```
SAMPLES = [
    {
        "id": "linear_easy",
        "topic": "algebra",
        "difficulty": 1,
        "template": "linear_v1",
        "synthetic": False,
        "train_overlap": False,
        "answer_correct": True,
        "verifier_pass": True,
        "first_error_gold": None,
        "first_error_pred": None,
        "steps": [
            {"ok": True, "relevant": True, "auto": True, "human_cost": 0},
            {"ok": True, "relevant": True, "auto": True, "human_cost": 0},
        ],
    },
    {
        "id": "ratio_word",
        "topic": "word_problem",
        "difficulty": 2,
        "template": "word_v1",
        "synthetic": True,
        "train_overlap": False,
        "answer_correct": True,
        "verifier_pass": True,
        "first_error_gold": None,
        "first_error_pred": None,
        "steps": [
            {"ok": True, "relevant": True, "auto": True, "human_cost": 0},
            {"ok": True, "relevant": False, "auto": True, "human_cost": 0},
            {"ok": True, "relevant": True, "auto": False, "human_cost": 2},
        ],
    },
    {
        "id": "geometry_jump",
        "topic": "geometry",
        "difficulty": 4,
        "template": "geom_v2",
        "synthetic": False,
        "train_overlap": False,
        "answer_correct": False,
        "verifier_pass": False,
        "first_error_gold": 1,
        "first_error_pred": 1,
        "steps": [
            {"ok": True, "relevant": True, "auto": True, "human_cost": 0},
            {"ok": False, "relevant": True, "auto": False, "human_cost": 3},
            {"ok": False, "relevant": True, "auto": False, "human_cost": 3},
        ],
    },
]
```

```

        "id": "counting_template",
        "topic": "combinatorics",
        "difficulty": 3,
        "template": "count_v1",
        "synthetic": True,
        "train_overlap": True,
        "answer_correct": True,
        "verifier_pass": True,
        "first_error_gold": None,
        "first_error_pred": None,
        "steps": [
            {"ok": True, "relevant": True, "auto": True, "human_cost": 0},
            {"ok": True, "relevant": True, "auto": True, "human_cost": 0},
            {"ok": True, "relevant": True, "auto": True, "human_cost": 0},
        ],
    },
    {
        "id": "proof_polished_wrong",
        "topic": "proof",
        "difficulty": 5,
        "template": "proof_v1",
        "synthetic": False,
        "train_overlap": False,
        "answer_correct": False,
        "verifier_pass": True,
        "first_error_gold": 1,
        "first_error_pred": 2,
        "steps": [
            {"ok": True, "relevant": True, "auto": False, "human_cost": 5},
            {"ok": False, "relevant": True, "auto": False, "human_cost": 5},
            {"ok": True, "relevant": True, "auto": False, "human_cost": 5},
        ],
    },
    {
        "id": "algebra_duplicate",
        "topic": "algebra",
        "difficulty": 2,
        "template": "linear_v1",
        "synthetic": True,
        "train_overlap": False,
        "answer_correct": True,
        "verifier_pass": True,
        "first_error_gold": None,
        "first_error_pred": None,
        "steps": [
            {"ok": True, "relevant": True, "auto": True, "human_cost": 0},
            {"ok": True, "relevant": True, "auto": True, "human_cost": 0},
        ],
    },
]

TARGET_TOPICS = {"algebra", "word_problem", "geometry", "combinatorics", "proof"}

```

```
def ratio(numerator, denominator):
```

```

    return round(numerator / denominator, 3) if denominator else 0.0

def bucket(difficulty):
    if difficulty <= 2:
        return "easy"
    if difficulty <= 4:
        return "medium"
    return "hard"

total = len(SAMPLES)
answer_correct = sum(sample["answer_correct"] for sample in SAMPLES)
verified = sum(sample["verifier_pass"] for sample in SAMPLES)
contaminated = [sample["id"] for sample in SAMPLES if sample["train_overlap"]]
templates = Counter(sample["template"] for sample in SAMPLES)
repeated_templates = sorted(name for name, count in templates.items() if count > 1)
covered_topics = {sample["topic"] for sample in SAMPLES}

all_steps = [step for sample in SAMPLES for step in sample["steps"]]
step_correct = sum(step["ok"] for step in all_steps)
step_relevant = sum(step["relevant"] for step in all_steps)
auto_labeled = sum(step["auto"] for step in all_steps)
human_cost = sum(step["human_cost"] for step in all_steps)

error_cases = [
    sample for sample in SAMPLES
    if sample["first_error_gold"] is not None
]
first_error_hits = sum(
    sample["first_error_gold"] == sample["first_error_pred"]
    for sample in error_cases
)

by_bucket = defaultdict(lambda: [0, 0])
by_topic = defaultdict(lambda: [0, 0])
for sample in SAMPLES:
    group = bucket(sample["difficulty"])
    by_bucket[group][0] += int(sample["answer_correct"])
    by_bucket[group][1] += 1
    by_topic[sample["topic"]][0] += int(sample["answer_correct"])
    by_topic[sample["topic"]][1] += 1

accepted = [
    sample for sample in SAMPLES
    if sample["answer_correct"]
    and sample["verifier_pass"]
    and not sample["train_overlap"]
]
accepted = sorted(accepted, key=lambda item: (item["difficulty"], item["id"]))
accepted_topics = Counter(sample["topic"] for sample in accepted)
training_mix = {
    topic: ratio(count, len(accepted))
    for topic, count in sorted(accepted_topics.items())
}

```

```

summary = {
    "answer_accuracy": ratio(answer_correct, total),
    "verifier_pass_rate": ratio(verified, total),
    "contamination_rate": ratio(len(contaminated), total),
    "unique_template_ratio": ratio(len(templates), total),
    "topic_coverage": ratio(len(covered_topics & TARGET_TOPICS), len(TARGET_TOPICS)),
}
process_metrics = {
    "step_accuracy": ratio(step_correct, len(all_steps)),
    "relevant_step_ratio": ratio(step_relevant, len(all_steps)),
    "auto_label_coverage": ratio(auto_labeled, len(all_steps)),
    "human_label_cost": human_cost,
    "first_error_accuracy": ratio(first_error_hits, len(error_cases)),
}
slice_accuracy = {
    "difficulty": {
        name: ratio(values[0], values[1])
        for name, values in sorted(by_bucket.items())
    },
    "topic": {
        name: ratio(values[0], values[1])
        for name, values in sorted(by_topic.items())
    },
}
risks = {
    "contaminated": contaminated,
    "verified_wrong": [
        sample["id"] for sample in SAMPLES
        if sample["verifier_pass"] and not sample["answer_correct"]
    ],
    "hard_failures": [
        sample["id"] for sample in SAMPLES
        if bucket(sample["difficulty"]) == "hard" and not sample["answer_correct"]
    ],
    "repeated_templates": repeated_templates,
}
gates = {
    "answer_quality": summary["answer_accuracy"] >= 0.75,
    "process_quality": (
        process_metrics["step_accuracy"] >= 0.8
        and process_metrics["first_error_accuracy"] >= 0.8
    ),
    "contamination_ok": summary["contamination_rate"] == 0.0,
    "coverage_ok": summary["topic_coverage"] >= 0.8,
    "hard_slice_ok": slice_accuracy["difficulty"].get("hard", 0.0) >= 0.5,
}

print(f"data_summary={summary}")
print(f"process_metrics={process_metrics}")
print(f"slice_accuracy={slice_accuracy}")
print(f"curriculum_order=[sample['id'] for sample in accepted]")
print(f"training_mix={training_mix}")
print(f"risks={risks}")
print(f"gates={gates}")

```



```
print(f"gate_pass={all(gates.values())}")
```

预期输出：

```
data_summary={'answer_accuracy': 0.667, 'verifier_pass_rate': 0.833, 'contamination_rate': 0.167, 'unique_template_ratio': 0.833, 'process_metrics': {'step_accuracy': 0.812, 'relevant_step_ratio': 0.938, 'auto_label_coverage': 0.625, 'human_label_cost': 0.375}, 'slice_accuracy': {'difficulty': {'easy': 1.0, 'hard': 0.0, 'medium': 0.5}, 'topic': {'algebra': 1.0, 'combinatorics': 1.0, 'geometry': 0.5, 'proof': 0.5}}, 'curriculum_order': ['linear_easy', 'algebra_duplicate', 'ratio_word'], 'training_mix': {'algebra': 0.667, 'word_problem': 0.333}, 'risks': {'contaminated': ['counting_template'], 'verified_wrong': ['proof_polished_wrong'], 'hard_failures': ['proof_polished_wrong']}, 'gates': {'answer_quality': False, 'process_quality': False, 'contamination_ok': False, 'coverage_ok': True, 'hard_slice_ok': False}, 'gate_pass=False
```

这个输出故意让 `gate_pass=False`。原因有四个：平均答案准确率不够，第一处错误定位太弱，存在评测污染样本，`hard proof` 切片完全失败。真实数学训练中，如果只保留 `curriculum_order` 里通过答案和 verifier 检查的样本，训练集会偏向 `easy algebra` 和 `word problem`，反而掩盖 `geometry`、`proof` 和 `hard slice` 的短板。

8.16 面试题：如何构造数学推理训练数据

回答要点：

我会把数学数据分成题目答案数据、步骤解法数据、多解法数据、错误纠错数据和合成题数据。数据构造时需要保证答案正确、步骤清晰、解法多样、错误定位准确、合成题难度可控。

8.17 面试题：数学 verifier 有什么用

回答要点：

数学 verifier 可以用于数据清洗、候选 reranking、过程评分、RL reward 和推理时搜索剪枝。程序化 verifier 在可计算任务上更有优势。

8.18 小练习

1. 设计一个数学训练样本 schema，至少包含题目、答案、步骤、步骤标签、题型、难度、来源和污染标记。
2. 修改本章 demo，让 `proof_polished_wrong` 的 verifier 不通过，观察 `verified_wrong` 和 `gate` 如何变化。
3. 给 `algebra`、`geometry`、`proof` 三个切片各设计 2 个可自动验证信号，并说明哪些信号不能替代人工证明审查。
4. 解释为什么合成题需要同时看模板多样性、题型覆盖、难度覆盖和真实评测迁移效果。
5. 设计一个数学 PRM 标注任务，要求标注员只标第一处错误，而不是重写完整解法。

8.19 本章小结

数学推理训练是 reasoning model 的核心训练场景。它提供了明确目标、结构化步骤、可验证答案和可控难度，非常适合训练 CoT、process supervision、verifier、search 和 test-time compute scaling。

但数学训练也有风险：合成题可能模板化，benchmark 可能泄漏，模型可能学会格式而不是推理，verifier 可能被欺骗。真正可靠的数学 reasoning 需要高质量步骤数据、多解法、错误纠错、自动验证、难度 curriculum 和严格评估共同支撑。下一章会进入代码推理与执行反馈，讨论代码任务中如何利用测试、编译器和运行时反馈训练 reasoning model。

第九章：代码推理与执行反馈

代码任务是 reasoning model 的另一个核心训练场景。和数学一样，代码有明确目标、结构化过程和可验证结果；但代码比数学更工程化，因为模型可以运行程序、读取报错、修改代码、再运行测试。这种“生成 -> 执行 -> 反馈 -> 修正”的闭环，是训练和评估可验证推理能力的重要方式。

本章系统讲代码推理与执行反馈：代码生成为什么适合 reasoning，单元测试和解释器反馈如何提供 reward，self-debug 如何工作，程序合成和搜索如何结合，训练数据如何构造，以及工程上如何处理沙箱、安全和成本。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修参考公开代码 reasoning 资料，包括 [Codex / HumanEval](#)、[AlphaCode](#)、[CodeRL](#)、[Self-Debugging](#)、APPS、MBPP 和 SWE-bench 等代码生成、程序合成、执行反馈与真实仓库修复评测资料。这些资料共同提示：代码 reasoning 不能只看代码文本相似度，而要看 functional correctness、测试覆盖、pass@k、公开 / 隐藏测试差距、修复闭环、执行环境和安全沙箱。

本章不提供绕过沙箱、逃逸执行环境或访问真实系统资源的技巧。所有代码示例只用于防御性审计和教学，强调静态拦截、资源限制、测试隔离和执行日志。真实系统不能直接运行模型生成的任意代码；必须用隔离容器、最小权限、超时、内存限制、网络隔离、文件系统隔离和审计日志。

9.1 为什么代码适合训练 Reasoning

代码任务有几个特点：

1. 输入目标通常明确。
2. 输出可以被编译或运行。
3. 测试可以判断行为是否正确。
4. 错误信息提供中间反馈。
5. 可以迭代修复。
6. 代码结构天然适合分解。

数学题通常验证最终答案，代码题可以验证行为。例如一个函数是否正确，不只看文本是否像代码，而是看它能不能通过测试。

面试回答：

代码任务适合 reasoning model，因为代码可以执行，编译器、解释器和单元测试能提供可验证反馈。模型可以生成代码、运行测试、debug、tool-use reasoning、verifier 和执行反馈驱动的强化学习。

9.1.1 关键公式与代码执行反馈指标速查

把第 i 个代码任务写成：

$$c_i = (x_i, \sigma_i, T_i^{\{\mathrm{pub}\}}, T_i^{\{\mathrm{hid}\}}, \mathcal{P}_i, e_i, b_i)$$

其中 x_i 是需求， σ_i 是函数签名或接口约束， $T_i^{\{\mathrm{pub}\}}$ 是公开测试， $T_i^{\{\mathrm{hid}\}}$ 是隐藏测试， $\mathcal{P}_i$ 是候选程序集合， e_i 是执行环境， b_i 是时间、内存、候选数和迭代轮数预算。

候选程序 p_{ij} 在测试集合 T 上的通过率：

$$S(p_{ij}, T) = \frac{1}{|T|} \sum_{\tau \in T} \mathbb{1}[\mathrm{run}(p_{ij}, \tau) = 1]$$

公开测试通过率和隐藏测试通过率分别是 $S(p, T_{\mathrm{pub}})$ 和 $S(p, T_{\mathrm{hid}})$ 。公开测试可以用于反馈和筛选，隐藏测试只能用于最终评估或离线审计。

代码生成常用 pass@k 衡量“生成 k 个候选时至少有一个正确候选”的概率。给定 n 个候选，其中 c 个通过隐藏测试，不放回估计为：

$$\mathrm{pass}@k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

注意 pass@k 不等于线上单次调用可靠性。它更像候选池潜力，仍然需要选择器、测试覆盖和预算控制。

公开 / 隐藏泛化差距：

$$\Delta_{\{\mathrm{pub-hid}\}} = A_{\{\mathrm{pub}\}} - A_{\{\mathrm{hid}\}}$$

如果 A_{pub} 很高但 A_{hid} 很低，说明模型或 reranker 可能过拟合公开样例、边界条件覆盖不足，或者候选选择器偏向“样例投机”代码。

一次 self-debug 轨迹可以写成：

$$\tau_i = (p_i^{\{0\}}, f_i^{\{0\}}, p_i^{\{1\}}, f_i^{\{1\}}, \dots, p_i^{\{R_i\}}, f_i^{\{R_i\}})$$

其中 $p_i^{\{r\}}$ 是第 r 轮代码， $f_i^{\{r\}}$ 是编译器、解释器、单元测试、类型检查或静态分析反馈。

修复成功率：

$$A_{\{\mathrm{repair}\}} = \frac{1}{N_{\{\mathrm{fail}\}}} \sum_{i \in \mathcal{F}} \mathbb{1}[S(p_i^{\{R_i\}}, T_i^{\{\text{hid}\}}) = 1]$$

其中 $\mathcal{F}$ 是初始候选没有通过公开测试或验证反馈明确失败的任务集合。

平均首次通过轮数：

$$R_{\{\mathrm{first}\}} = \frac{1}{N} \sum_{i=1}^N \min\{r: S(p_i^{\{r\}}, T_i^{\{\text{pub}\}}) = 1\}$$

如果很多任务需要多轮才能过公开测试，说明初始生成质量或需求理解可能不足；如果公开测试很快过但隐藏测试差，说明测试覆盖和泛化有问题。

沙箱违规率：

$$R_{\{\mathrm{sandbox}\}} = \frac{1}{M} \sum_{i,j} \mathbb{1}[u_{ij} = 1]$$

其中 $u_{ij}=1$ 表示候选代码触发导入禁用模块、文件访问、网络访问、无限循环、子进程、危险调用或资源超限等拦截规则。这个指标不是为了教模型如何绕过限制，而是为了证明执行系统没有把危险候选放进真实运行环境。

一个简化代码 reasoning 上线门禁：

$$G_{\{\mathrm{code}\}} = \mathbb{1}[\begin{aligned} &A_{\{\mathrm{hid}\}} \geq \alpha \\ &\land \\ &\Delta_{\{\mathrm{pub-hid}\}} \leq \delta \\ &\land \\ &A_{\{\mathrm{repair}\}} \geq \beta \\ &\land \\ &R_{\{\mathrm{sandbox}\}} \leq \rho \\ &\land \\ &C_{\{\mathrm{exec}\}} \leq C_{\{\max\}} \end{aligned}]$$

其中 C_{exec} 是候选生成、执行、测试和修复迭代的总成本。面试里要强调：代码 reasoning 的可靠性来自可执行反馈闭环，但上线门禁必须同时看隐藏泛化、修复能力、沙箱安全和成本。

9.2 代码推理的基本闭环

一个典型代码推理闭环是：

理解需求 -> 设计方案 -> 生成代码 -> 执行测试 -> 读取反馈 -> 修改代码 -> 再测试

这个闭环比一次性代码生成更可靠。因为模型第一次生成的代码可能有语法错误、边界条件遗漏、性能问题或接口不匹配。执行反馈可以把这些错误暴露出来。

Reasoning model 的价值在于，它不只是写代码，还能根据反馈定位错误并制定修复策略。

9.3 执行反馈有哪些类型

常见执行反馈包括：

1. 语法错误。
2. 编译错误。
3. 运行时异常。
4. 单元测试失败。
5. 超时。
6. 内存超限。
7. 静态检查错误。
8. 类型检查错误。
9. 覆盖率不足。

不同反馈提供不同信号。语法错误通常容易修；测试失败更难，因为它只说明行为不符合预期；超时和内存超限则常常意味着算法复杂度不合适。

9.4 单元测试作为 Verifier

在代码任务中，单元测试是最常见的 verifier。

它可以用于：

1. 判断候选代码是否正确。
2. 给不同候选排序。
3. 作为 RL reward。
4. 作为搜索剪枝依据。
5. 作为 self-debug 的反馈。

例如：

candidate A: 通过 3/10 tests
candidate B: 通过 8/10 tests
candidate C: 通过 10/10 tests

系统可以优先保留 candidate C，或者让模型分析 candidate B 的失败用例继续修复。

测试的优势是客观、可执行、自动化。局限是测试覆盖不一定完整，通过测试不等于完全正确。

9.5 Self-Debug

Self-debug 指模型根据错误反馈自我修复。

基本流程：

生成代码
运行测试
读取报错
解释失败原因
修改代码
再次运行

一个好的 self-debug 过程不是盲目改代码，而是先定位失败类型。

例如：

1. 是接口错误还是算法错误。

2. 是边界条件遗漏还是复杂度不够。
3. 是数据结构选错还是状态更新错。
4. 是测试理解错误还是代码实现错误。

训练 self-debug 时，可以构造 “错误代码 + 报错信息 + 修复后代码” 的数据。

9.6 程序合成

程序合成是从规格或示例中生成满足要求的程序。它和 search reasoning 关系密切，因为一个程序可以通过多种方式构造。

程序合成常见输入：

1. 自然语言需求。
2. 函数签名。
3. 输入输出样例。
4. 隐藏测试。
5. 类型约束。
6. 性能约束。

程序合成中的搜索空间非常大，因此需要约束、测试和剪枝。LLM 可以生成候选程序，测试系统提供反馈，搜索控制器决定继续修改哪个候选。

9.7 搜索和代码修复

代码修复可以看成搜索问题。

state = 当前代码和测试结果

action = 一次代码修改

transition = 应用修改并运行测试

score = 测试通过率、错误减少程度、代码质量

termination = 测试通过或预算耗尽

常见策略：

1. 生成多个候选补丁。
2. 运行测试评分。
3. 保留通过更多测试的补丁。
4. 针对失败测试继续修复。
5. 达到最大轮数后返回最佳候选。

这种方法适合 bug fixing、竞赛题、API 使用修复和小型代码生成任务。

9.8 训练数据构造

代码推理训练数据可以包括：

1. 需求到代码。
2. 需求到解题思路再到代码。
3. 错误代码到修复代码。
4. 测试失败信息到修复解释。
5. 多候选代码和测试结果。
6. 代码审查和缺陷定位。
7. 性能优化过程。
8. 工具调用轨迹。

高质量数据应该包含任务、环境、依赖、测试、错误反馈和最终修复，而不只是孤立代码片段。

9.9 执行反馈作为 Reward

代码任务适合 RL，因为 reward 可以来自执行结果。

常见 reward：

1. 编译通过。
2. 单元测试通过数量。
3. 隐藏测试通过率。
4. 无超时。
5. 无内存超限。
6. 代码风格或安全检查通过。

但 reward 设计要小心。只奖励公开测试通过，模型可能过拟合测试；只奖励短代码，模型可能牺牲可读性；只奖励最终通过，训练信号可能过于稀疏。

更稳健的做法是结合过程反馈、测试覆盖、静态检查和最终结果。

9.10 隐藏测试和泛化

公开测试容易被过拟合。隐藏测试用于评估泛化能力。

一个模型可能通过所有公开样例，但在隐藏边界条件上失败。例如：

1. 空输入。
2. 单元素输入。
3. 极大输入。
4. 重复元素。
5. 负数。
6. Unicode 或特殊字符。
7. 并发或状态污染。

训练和评估时，要关注模型是否真的理解规格，而不是只拟合样例。

9.11 安全沙箱

执行模型生成的代码有安全风险。必须使用沙箱。

需要限制：

1. 文件系统访问。
2. 网络访问。
3. CPU 时间。
4. 内存。
5. 子进程。
6. 系统调用。
7. 环境变量和密钥。

工程系统中，代码执行服务通常要隔离运行，并设置超时、资源限制和日志审计。不能直接在生产主机上无保护地运行模型生成代码。

9.12 多语言和环境问题

代码任务的难点还包括环境一致性。

常见问题：

1. Python 版本不同。
2. 依赖缺失。
3. 编译器版本不同。
4. 操作系统差异。

5. 随机数和时间依赖。
6. 测试顺序依赖。
7. 文件路径差异。

如果环境不稳定，执行反馈会变成噪声。训练和评估系统必须记录语言、版本、依赖和运行配置。

9.13 常见失败模式

1. 代码看起来合理，但没有通过测试。
2. 模型误解需求，修错方向。
3. 只修公开测试，泛化失败。
4. 反复修改同一处，无法跳出局部错误。
5. 报错信息很长，模型抓不到关键点。
6. 执行环境不一致导致反馈不稳定。
7. 生成不安全代码。
8. 通过测试但代码不可维护。

代码 reasoning 的关键不是“会写代码”，而是能把需求、反馈、测试和修复策略连接起来。

9.14 评估指标

代码推理评估可以看：

1. Pass@1。
2. Pass@k。
3. 修复成功率。
4. 平均迭代轮数。
5. 测试通过率提升。
6. 隐藏测试通过率。
7. 超时率。
8. 安全违规率。
9. 代码可读性和维护性。

Pass@k 衡量多样本生成能力；修复成功率衡量执行反馈利用能力；隐藏测试通过率衡量泛化能力。

9.15 最小可运行代码推理 / 执行反馈审计 demo

这个 demo 用 4 个 toy 函数任务模拟代码生成、公开测试筛选、隐藏测试评估、self-debug 修复和静态安全拦截。它只执行正文里硬编码的候选代码，并用简化 AST guard 拦截 import、危险调用和无限循环；这只是教学 demo，不是生产级沙箱。

```
import ast
import math
```

```
TASKS = [
    {
        "id": "count_positive",
        "public": [([1, -2, 3],), 2], ([[1, 5],), 1],
        "hidden": [([0, 1, -1],), 1], ([[0, 0],), 0],
        "candidates": [
            "def solve(nums):\n    return sum(1 for x in nums if x >= 0)",
            "def solve(nums):\n    return sum(1 for x in nums if x > 0)",
            "import os\ndef solve(nums):\n    return len(nums)",
        ],
    },
    {
```

```

        "id": "left_middle",
        "public": [((("abc",), "b"), (("hello",), "l"))],
        "hidden": [((("abcd",), "b"), (("xy",), "x"))],
        "candidates": [
            "def solve(s):\n    return s[len(s) // 2]",
            "def solve(s):\n    return s[(len(s) - 1) // 2]",
        ],
    },
    {
        "id": "clamp_values",
        "public": [((-2, 0, 3), 0, 5), (0, 0, 3)], ((1, 2), 0, 5), [1, 2]],
        "hidden": [((9, -1, 4), 0, 5), [5, 0, 4]],
        "candidates": [
            "def solve(nums, lo, hi):\n    return [min(max(x, lo), hi) for x in nums]",
            "def solve(nums, lo, hi):\n    return [max(x, lo) for x in nums]",
        ],
    },
    {
        "id": "parse_ints",
        "public": [((("1 2 -3",), [1, 2, -3])),
        "hidden": [((("4, 5 -6",), [4, 5, -6])),
        "candidates": [
            "def solve(text):\n    return [int(x) for x in text.split(',')]",
            "def solve(text):\n    return [int(x) for x in text.replace(',', ' ').split()]",
            "def solve(text):\n    while True:\n        pass",
        ],
    },
]

```

```

ALLOWED_BUILTINS = {

```

```

    "sum": sum,
    "len": len,
    "int": int,
    "str": str,
    "range": range,
    "list": list,
    "set": set,
    "sorted": sorted,
    "enumerate": enumerate,
    "min": min,
    "max": max,

```

```

}

```

```

BANNED_CALLS = {"open", "eval", "exec", "compile", "input", "__import__"}

```

```

def guard_code(src):
    try:
        tree = ast.parse(src)
    except SyntaxError as exc:
        return False, f"syntax:{exc.__class__.__name__}"
    for node in ast.walk(tree):
        if isinstance(node, (ast.Import, ast.ImportFrom)):
            return False, "blocked_import"
        if isinstance(node, ast.While):
            return False, "blocked_loop"

```



```

        if isinstance(node, ast.Attribute) and node.attr.startswith("__"):
            return False, "blocked_dunder"
        if isinstance(node, ast.Name) and node.id.startswith("__"):
            return False, "blocked_dunder"
        if isinstance(node, ast.Call) and isinstance(node.func, ast.Name):
            if node.func.id in BANNED_CALLS:
                return False, f"blocked_call:{node.func.id}"
    return True, "ok"

def load_solve(src):
    allowed, reason = guard_code(src)
    if not allowed:
        return None, reason
    env = {"__builtins__": ALLOWED_BUILTINS}
    local = {}
    exec(compile(src, "<candidate>", "exec"), env, local)
    fn = local.get("solve") or env.get("solve")
    if not callable(fn):
        return None, "missing_solve"
    return fn, "ok"

def run_tests(fn, tests):
    passed = 0
    errors = []
    for args, expected in tests:
        try:
            got = fn(*args)
            ok = got == expected
        except Exception as exc:
            got = f"error:{exc.__class__.__name__}"
            ok = False
        passed += int(ok)
        if not ok:
            errors.append({"args": args, "expected": expected, "got": got})
    return passed, len(tests), errors

def pass_at_k(n, correct, k):
    if correct <= 0:
        return 0.0
    if n - correct < k:
        return 1.0
    return round(1 - math.comb(n - correct, k) / math.comb(n, k), 3)

reports_by_task = {}
blocked = []
selected = []
public_fail_repairs = []
pass1_values = []
pass2_values = []
attempts_to_public = []
all_reports = []

```

```

for task in TASKS:
    task_reports = []
    for idx, src in enumerate(task["candidates"]):
        fn, status = load_solve(src)
        report_id = f"{task['id']}:{c{idx}}"
        if fn is None:
            report = {
                "id": report_id,
                "status": status,
                "public_pass": 0,
                "public_total": len(task["public"]),
                "hidden_pass": 0,
                "hidden_total": len(task["hidden"]),
            }
            blocked.append(report_id)
        else:
            pub_pass, pub_total, pub_errors = run_tests(fn, task["public"])
            hid_pass, hid_total, hid_errors = run_tests(fn, task["hidden"])
            report = {
                "id": report_id,
                "status": "ok",
                "public_pass": pub_pass,
                "public_total": pub_total,
                "hidden_pass": hid_pass,
                "hidden_total": hid_total,
                "first_public_error": pub_errors[:1],
                "first_hidden_error": hid_errors[:1],
            }
            task_reports.append(report)
            all_reports.append(report)

reports_by_task[task["id"]] = task_reports
executable = [report for report in task_reports if report["status"] == "ok"]
hidden_correct = sum(
    report["hidden_pass"] == report["hidden_total"]
    for report in executable
)
pass1_values.append(pass_at_k(len(executable), hidden_correct, 1))
pass2_values.append(pass_at_k(len(executable), hidden_correct, min(2, len(executable))))

best = max(
    executable,
    key=lambda report: (
        report["public_pass"] / max(1, report["public_total"]),
        -int(report["id"].split(":c")[-1]),
    ),
)
selected.append(best)

first_success = None
for pos, report in enumerate(task_reports, start=1):
    if report["status"] == "ok" and report["public_pass"] == report["public_total"]:
        first_success = pos
        break

```

```

attempts_to_public.append(first_success or len(task_reports) + 1)

greedy = task_reports[0]
if greedy["status"] == "ok" and greedy["public_pass"] < greedy["public_total"]:
    public_fail_repairs.append(
        {
            "task": task["id"],
            "repaired": best["hidden_pass"] == best["hidden_total"],
            "chosen": best["id"],
        }
    )

greedy_accuracy = sum(
    reports_by_task[task["id"]][0]["status"] == "ok"
    and reports_by_task[task["id"]][0]["hidden_pass"] == reports_by_task[task["id"]][0]["hidden_total"]
    for task in TASKS
) / len(TASKS)
selected_public_accuracy = sum(
    report["public_pass"] == report["public_total"]
    for report in selected
) / len(selected)
selected_hidden_accuracy = sum(
    report["hidden_pass"] == report["hidden_total"]
    for report in selected
) / len(selected)
repair_success_rate = (
    sum(item["repaired"] for item in public_fail_repairs) / len(public_fail_repairs)
    if public_fail_repairs else 0.0
)

summary = {
    "greedy_accuracy": round(greedy_accuracy, 3),
    "public_rerank_accuracy": round(selected_hidden_accuracy, 3),
    "selected_public_pass_rate": round(selected_public_accuracy, 3),
    "public_hidden_gap": round(selected_public_accuracy - selected_hidden_accuracy, 3),
    "pass_at_1_est": round(sum(pass1_values) / len(pass1_values), 3),
    "pass_at_2_est": round(sum(pass2_values) / len(pass2_values), 3),
    "repair_success_rate": round(repair_success_rate, 3),
    "avg_attempts_to_public_success": round(sum(attempts_to_public) / len(attempts_to_public), 3),
    "blocked_candidate_rate": round(len(blocked) / len(all_reports), 3),
}

hidden_failures_after_public = [
    report["id"].split(":c")[0]
    for report in selected
    if report["public_pass"] == report["public_total"]
    and report["hidden_pass"] < report["hidden_total"]
]

gates = {
    "public_signal_ok": summary["selected_public_pass_rate"] >= 0.9,
    "hidden_generalization_ok": summary["public_rerank_accuracy"] >= 0.75,
    "gap_ok": summary["public_hidden_gap"] <= 0.25,
    "repair_ok": summary["repair_success_rate"] >= 0.8,
    "sandbox_blocks_unsafe": len(blocked) == 2,
}

```

```

print(f"summary={summary}")
print(f"selected=[[report['id'] for report in selected]])")
print(f"hidden_failures_after_public={hidden_failures_after_public}")
print(f"public_fail_repairs={public_fail_repairs}")
print(f"blocked={blocked}")
print(f"gates={gates}")
print(f"gate_pass={all(gates.values())}")

```

预期输出：

```

summary={'greedy_accuracy': 0.25, 'public_rerank_accuracy': 0.5, 'selected_public_pass_rate': 1.0, 'public_hidden_gap': 0.5}
selected=[{'count_positive': c0, 'left_middle': c0, 'clamp_values': c0, 'parse_ints': c1}]
hidden_failures_after_public=[{'count_positive': c0, 'left_middle': c0}]
public_fail_repairs=[{'task': 'parse_ints', 'repaired': True, 'chosen': 'parse_ints:c1'}]
blocked=[{'count_positive': c2, 'parse_ints': c2}]
gates={'public_signal_ok': True, 'hidden_generalization_ok': False, 'gap_ok': False, 'repair_ok': True, 'sandbox_blocks': True}
gate_pass=False

```

这个输出说明三件事。第一，`pass_at_2_est=1.0` 表示候选池里每题都有正确解，但公开测试 rerank 只能拿到 0.5 的隐藏准确率，说明选择器和测试覆盖还不够。第二，`parse_ints` 能通过执行反馈修复，说明 self-debug 对公开失败有价值。第三，`count_positive` 和 `left_middle` 公开测试全过但隐藏失败，说明只看公开测试会高估能力。`gate_pass=False` 是故意保留的上线阻断信号。

9.16 面试题：执行反馈为什么重要

回答要点：

执行反馈把代码任务从纯文本生成变成可验证闭环。模型生成代码后，可以通过编译器、解释器、单元测试、类型检查和静态分析得到反馈。

9.17 面试题：如何设计代码 self-debug 系统

回答要点：

我会让系统先生成初始代码，然后在沙箱中运行测试，收集编译错误、运行异常和失败用例。模型根据反馈解释失败原因并生成补丁，迭代直到通过测试。

9.18 小练习

1. 给一个代码生成任务设计公开测试和隐藏测试，并说明哪些边界条件不能出现在公开样例里。
2. 修改本章 demo，让 `count_positive` 的公开测试包含 0，观察 `public / hidden gap` 如何变化。
3. 设计一个 self-debug 轨迹 schema，至少包含初始代码、失败测试、错误类型、补丁、修复后测试结果和执行预算。
4. 写出生成级代码执行沙箱至少需要限制的 5 类资源或权限。
5. 解释为什么 `pass@k` 高不代表线上一次调用可靠。

9.19 本章小结

代码推理与执行反馈是 reasoning model 落地能力的重要来源。代码可以运行，测试可以验证，错误信息可以指导修复，这使得代码任务天然适合 tool-use reasoning、self-debug、search、verifier 和 RL。

但代码任务也有工程复杂性：测试覆盖不完整、隐藏用例难、环境差异大、安全沙箱必须可靠、模型可能过拟合公开测试。真正强的代码 reasoning 系统，不只是生成代码，而是能在受控环境中根据反馈持续修正，并在质量、成本、安全和泛化之间取得平衡。下一章会讨论 reasoning 评估，系统梳理如何判断一个模型是否真的具备推理能力。

第十章：Reasoning 评估

Reasoning model 的评估比普通问答评估更难。因为我们不只关心最终答案对不对，还关心推理过程是否可靠、模型是否真的泛化、是否只是背过 benchmark、是否能在更难变体上保持稳定，以及提升是否来自更多推理计算而不是模型本身能力变化。

本章系统讲 reasoning 评估：数学和代码 benchmark、最终答案评估、过程评估、污染检测、泛化评估、鲁棒性、推理时计算成本、人工评估和常见陷阱。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修参考公开评估资料，包括 [HELM](#)、[BIG-bench](#)、[MMLU](#)、[GPQA](#)、GSM8K、MATH、HumanEval、LiveCodeBench、SWE-bench、OpenAI Evals 和 EleutherAI lm-evaluation-harness 等资料。它们共同提示：reasoning 评估不能只看单一 benchmark 平均分，而要同时记录任务版本、prompt、采样参数、候选数、verifier、工具、污染检查、切片表现、统计不确定性和成本。

本章不把评估写成“刷榜技巧”。第二轮精修重点是建立可复现、可审计的评估报告结构：同一批样本上比较 baseline 和 candidate；明确 original、variant、contaminated、hard slice；同时看最终答案、过程步骤、第一处错误、public / hidden test、test-time compute、预算归一化指标和 paired significance。面试中要能说明：分数提升是否来自模型能力、推理预算、候选选择器、污染、prompt 调参，还是统计噪声。

10.1 Reasoning 评估为什么难

普通分类任务可以直接看准确率。Reasoning 任务更复杂，因为：

1. 最终答案可能对，但过程错误。
2. 过程看起来合理，但答案错误。
3. 开放题没有唯一答案。
4. 模型可能见过测试题。
5. 多次采样结果不稳定。
6. 推理成本不同，分数不可直接比较。
7. Verifier 本身可能有偏差。

面试回答：

Reasoning 评估不能只看最终准确率。需要同时评估最终答案、推理过程、泛化能力、鲁棒性、推理成本和数据污染。尤其是当模型作

10.1.1 关键公式与 Reasoning 评估指标速查

把第 i 个 reasoning 评估样本写成：

$$e_i=(x_i,y_i^{\star},V_i,P_i,z_i,q_i,g_i,r_i,b_i)$$

其中 x_i 是原题， $y_i^{\star}$ 是标准答案或评估 oracle， V_i 是扰动 / 变体集合， P_i 是候选输出集合， z_i 是被评推理过程， q_i 是步骤标签， g_i 是题型、难度、语言、领域等切片标签， r_i 是污染、泄漏、格式异常等风险标记， b_i 是推理预算。

最终答案准确率：

$$A_{\{\mathrm{ans}\}}=\frac{1}{\sum_{i=1}^N}\mathbb{1}[\hat{y}_i=y_i^{\star}]$$

切片准确率：

$$A_g=\frac{1}{\sum_{i:g_i=g}}\mathbb{1}[\hat{y}_i=y_i^{\star}]$$

$$\begin{aligned} & \{ \\ & \sum_{i:g_i=g} 1 \\ & \} \end{aligned}$$

切片指标用于发现平均分掩盖的问题，例如 proof、geometry、hard code、long context 或中文题表现很差。

过程步骤准确率：

$$\begin{aligned} A_{\{\mathrm{step}\}} = & \\ \frac{ & \\ \sum_{i,j} \mathbb{1}[\hat{q}_{ij}=q_{ij}] & \\ \{ & \\ \sum_i L_i & \\ \} & \end{aligned}$$

第一处错误定位准确率：

$$\begin{aligned} A_{\{\mathrm{first}\}} = & \\ \frac{1}{N_{\{\mathrm{err}\}}} & \\ \sum_{i \in \mathcal{E}} & \\ \mathbb{1}[\hat{e}_i=e_i^*] & \end{aligned}$$

扰动鲁棒性下降：

$$\begin{aligned} D_{\{\mathrm{robust}\}} = & \\ A_{\{\mathrm{orig}\}} - A_{\{\mathrm{var}\}} & \end{aligned}$$

如果原题正确率高、变体正确率低，常见原因是记忆原题、依赖表面模板、对无关信息敏感或没有真正理解约束。

污染率：

$$\begin{aligned} R_{\{\mathrm{contam}\}} = & \\ \frac{1}{N} & \\ \sum_{i=1}^N & \\ \mathbb{1}[r_i^{\{\mathrm{contam}\}}=1] & \end{aligned}$$

污染检查要覆盖题面、答案、解析、代码签名、测试样例、参数化模板和近似改写，而不是只做字符串 exact match。

test-time compute 成本：

$$\begin{aligned} C_i = & \\ c_{\{\mathrm{tok}\}} T_i & \\ + c_{\{\mathrm{cand}\}} K_i & \\ + c_{\{\mathrm{ver}\}} V_i & \\ + c_{\{\mathrm{tool}\}} U_i & \\ + c_{\{\mathrm{lat}\}} R_i & \end{aligned}$$

其中 T_i 是 token 数， K_i 是候选数， V_i 是 verifier 调用， U_i 是工具调用， R_i 是延迟。不同模型或策略比较时，必须报告预算，否则 1 次 greedy 和 64 次采样 rerank 的准确率不可直接比较。

单位成本正确率：

$$\begin{aligned} E_{\{\mathrm{cost}\}} = & \\ \frac{ & \\ \sum_i \mathbb{1}[\hat{y}_i=y_i^*] & \\ \{ & \\ \sum_i C_i & \\ \} & \end{aligned}$$

成对提升：

$$\begin{aligned} \Delta_{\{\mathrm{pair}\}} = & \\ \frac{1}{N} & \end{aligned}$$

$$\sum_{i=1}^N (\mathbb{1}[\hat{y}_i^{\text{new}} = y_i^{\text{star}}] - \mathbb{1}[\hat{y}_i^{\text{base}} = y_i^{\text{star}}])$$

同一批样本上比较新旧模型时，优先使用 paired evaluation，因为它能消除样本难度差异。小样本下还要给 bootstrap confidence interval 或 McNemar 类检验，而不是只报一个提升点。

一个简化 reasoning 评估门禁：

$$G_{\{\mathrm{eval}\}} = \mathbb{1} [A_{\{\mathrm{ans}\}} \geq \alpha \land A_{\{\mathrm{step}\}} \geq \beta \land A_{\{\mathrm{first}\}} \geq \gamma \land R_{\{\mathrm{contam}\}} \leq \rho \land D_{\{\mathrm{robust}\}} \leq \delta \land E_{\{\mathrm{cost}\}} \geq \eta]$$

这个门禁表达的是评估可信度，而不是单纯模型强弱：答案、过程、错误定位、污染、鲁棒性和成本都过线，评估结论才更值得相信。

10.2 数学 Benchmark

数学 benchmark 常用于评估长链推理能力。

常见类型：

1. 小学和初中应用题。
2. 高中数学题。
3. 竞赛数学题。
4. 证明题。
5. 符号计算题。
6. 合成数学题。

GSM 类任务偏自然语言应用题，MATH 类任务通常更难，包含代数、数论、组合、几何等多种题型。

数学 benchmark 的优势是答案明确、便于自动评测。局限是公开题容易污染，且最终答案准确率不能完全反映推理过程质量。

10.3 代码 Benchmark

代码 benchmark 常评估程序生成和问题求解能力。

常见指标：

1. Pass@1。
2. Pass@k。
3. 编译通过率。
4. 单元测试通过率。
5. 隐藏测试通过率。
6. 平均修复轮数。
7. 超时率。
8. 安全违规率。

代码评估的优势是可以执行验证。缺点是测试覆盖不完整，通过测试不等于完全正确，且不同语言、依赖和运行环境会影响结果。

代码 benchmark 还要区分一次生成能力和执行反馈利用能力。一个模型 Pass@1 高，说明直接生成强；一个模型 self-debug 后提升大，说明利用反馈能力强。

10.4 最终答案评估

最终答案评估是最基础的层面。

数学中可以看：

1. 数值是否相等。
2. 表达式是否等价。
3. 单位是否正确。
4. 格式是否符合要求。
5. 是否满足题目约束。

代码中可以看：

1. 是否通过测试。
2. 是否超时。
3. 是否内存超限。
4. 是否符合接口。
5. 是否产生副作用。

最终答案评估简单清晰，但无法说明模型为什么对或为什么错。

10.5 过程评估

过程评估关注中间推理步骤。

常见维度：

1. 每一步是否正确。
2. 步骤之间是否连贯。
3. 是否存在跳步。
4. 是否遗漏条件。
5. 是否第一处错误很早出现。
6. 最终答案是否由过程支持。

过程评估可以由人工、LLM judge、process verifier 或规则系统完成。它比最终答案评估更细，但成本更高，也更容易受到评估器偏差影响。

一个模型如果答案正确但过程胡编，在高风险场景中不能被视为可靠。

10.6 First Error Evaluation

First error evaluation 关注推理链中第一处错误在哪里。

它比“整条链对不对”更有诊断价值。

例如：

```
step 1: correct
step 2: correct
step 3: wrong
step 4: follows from wrong step
```

如果只看最终答案，模型是错的；如果看第一处错误，可以知道问题出在 step 3。这对训练 process verifier、构造纠错数据和分析模型短板很有帮助。

10.7 污染检测

数据污染是 reasoning 评估的大问题。模型可能在预训练或微调中见过测试题、答案或解析。

污染来源：

1. 公开 benchmark。
2. 题解网站。
3. 教材和论坛。
4. 代码仓库。
5. 合成模板重复。
6. 训练集和测试集去重不充分。

检测方式：

1. 字符串和语义去重。
2. 检查题目近似重复。
3. 使用新构造测试集。
4. 参数化改写题目。
5. 比较原题和变体题表现。
6. 分析模型是否输出标准题解措辞。

污染不能完全消除，但必须被报告和控制。

10.8 泛化评估

真正的 reasoning 能力应该能泛化到新题、变体题和更难题。

泛化评估可以包括：

1. 题面改写。
2. 数值替换。
3. 条件扰动。
4. 跨题型组合。
5. 更长推理链。
6. 未见过的工具环境。
7. 新 benchmark。

如果模型只在原题上高分，在轻微改写后大幅下降，说明可能依赖记忆或模板匹配。

10.9 鲁棒性评估

Reasoning 模型还要评估鲁棒性。

常见扰动：

1. 无关信息干扰。
2. 题目表述顺序变化。
3. 多余条件。
4. 缺失条件。
5. 对抗性提示。
6. 单位变化。
7. 格式变化。
8. 长上下文干扰。

鲁棒模型应该能识别哪些信息重要，哪些信息无关，而不是被表面文本带偏。

10.10 推理成本评估

Reasoning 评估必须报告推理成本。

需要记录：

1. 采样数量。
2. 输出 token 数。
3. 搜索深度。
4. verifier 调用次数。
5. 工具调用次数。
6. 平均延迟。
7. GPU 成本。
8. 超时率。

两个模型如果一个是单次采样，一个是 64 次采样加 reranking，直接比较准确率是不公平的。应同时报告 quality-cost curve，也就是不同预算下的质量曲线。

10.11 多次采样稳定性

Reasoning 模型输出常有随机性。因此需要评估稳定性。

常见方法：

1. 多次运行同一题。
2. 计算答案一致率。
3. 计算正确率方差。
4. 比较不同温度下表现。
5. 分析错误答案分布。

稳定性很重要。一个模型平均分高但波动巨大，在实际系统中可能不如一个平均分稍低但稳定的模型。

10.12 LLM Judge 的使用

LLM judge 可以评估开放式 reasoning，但要小心。

优点：

1. 覆盖开放题。
2. 成本低于大量人工评审。
3. 能给解释。
4. 可用于过程评分。

风险：

1. 偏好长答案。
2. 偏好格式漂亮的答案。
3. 被错误但流畅的推理欺骗。
4. 与被评模型同源导致偏差。
5. 评判标准不稳定。

使用 LLM judge 时，最好结合标准答案、rubric、多评委一致性和人工抽检。

10.13 人工评估

人工评估适合高难度、开放式和过程质量评估。

人工评估要注意：

1. 明确评分 rubric。
2. 标注者需要领域能力。
3. 进行多标注者一致性检查。
4. 区分答案正确和过程正确。
5. 抽样覆盖不同难度和题型。
6. 记录不确定案例。

人工评估成本高，但对发现 benchmark 盲区非常有价值。

10.14 常见评估陷阱

1. 只报最高分，不报预算。
2. 只看最终答案，不看过程。
3. 公开 benchmark 污染严重。
4. 使用不可靠 judge。
5. 题型覆盖太窄。
6. 忽略失败案例分析。
7. 忽略延迟和成本。
8. 把格式更长误认为推理更强。
9. 忽略多次运行波动。

Reasoning 评估的目标不是得到一个漂亮分数，而是理解模型在哪些任务上真的会推理，在哪些任务上只是看起来会推理。

10.15 一个完整评估报告应该包含什么

一个合理的 reasoning 评估报告至少包含：

1. Benchmark 名称和版本。
2. 数据去重和污染检查说明。
3. Prompt 和解码参数。
4. 是否使用 CoT、self-consistency、search、verifier 或工具。
5. 推理预算。
6. 最终答案指标。
7. 过程质量指标。
8. 分题型和分难度结果。
9. 成本和延迟。
10. 失败案例分析。

如果没有这些信息，评估结果很难复现，也很难判断改进来自哪里。

10.16 最小可运行 reasoning 评估审计 demo

这个 demo 模拟 6 个 toy reasoning 样本，对比 baseline 和 candidate。它同时记录原题、变体、过程步骤、第一处错误、污染、切片、成本和 paired bootstrap 区间。它故意让 `gate_pass=False`，因为候选模型虽然平均准确率更高，但统计区间不稳、过程质量不足、存在污染样本、变体鲁棒性下降明显。

```
from random import Random
```

```
SAMPLES = [
    {
        "id": "math_easy",
        "slice": "math",
        "contaminated": False,
        "baseline_correct": True,
        "candidate_correct": True,
        "variant_correct": True,
        "steps": [True, True, True],
        "first_error_gold": None,
        "first_error_pred": None,
        "cost": 120,
    },
    {
```

```

    "id": "math_contam",
    "slice": "math",
    "contaminated": True,
    "baseline_correct": False,
    "candidate_correct": True,
    "variant_correct": False,
    "steps": [True, True],
    "first_error_gold": None,
    "first_error_pred": None,
    "cost": 160,
  },
  {
    "id": "logic_distractor",
    "slice": "logic",
    "contaminated": False,
    "baseline_correct": False,
    "candidate_correct": True,
    "variant_correct": False,
    "steps": [True, False, True],
    "first_error_gold": 1,
    "first_error_pred": 1,
    "cost": 260,
  },
  {
    "id": "code_hidden",
    "slice": "code",
    "contaminated": False,
    "baseline_correct": False,
    "candidate_correct": True,
    "variant_correct": True,
    "steps": [True, True, True, True],
    "first_error_gold": None,
    "first_error_pred": None,
    "cost": 260,
  },
  {
    "id": "proof_hard",
    "slice": "proof",
    "contaminated": False,
    "baseline_correct": False,
    "candidate_correct": False,
    "variant_correct": False,
    "steps": [True, False, False, False],
    "first_error_gold": 1,
    "first_error_pred": 2,
    "cost": 420,
  },
  {
    "id": "simple_regression",
    "slice": "simple",
    "contaminated": False,
    "baseline_correct": True,
    "candidate_correct": False,
    "variant_correct": False,
    "steps": [True, False],

```

```

        "first_error_gold": 1,
        "first_error_pred": 1,
        "cost": 170,
    },
]

def ratio(numerator, denominator):
    return round(numerator / denominator, 3) if denominator else 0.0

def accuracy(key):
    return ratio(sum(sample[key] for sample in SAMPLES), len(SAMPLES))

def paired_bootstrap_ci(diffs, rounds=2000, seed=7):
    rng = Random(seed)
    n = len(diffs)
    values = []
    for _ in range(rounds):
        values.append(sum(diffs[rng.randrange(n)] for _ in range(n)) / n)
    values.sort()
    return (
        round(values[int(0.025 * rounds)], 3),
        round(values[int(0.975 * rounds)], 3),
    )

baseline_accuracy = accuracy("baseline_correct")
candidate_accuracy = accuracy("candidate_correct")
variant_accuracy = accuracy("variant_correct")
diffs = [
    int(sample["candidate_correct"]) - int(sample["baseline_correct"])
    for sample in SAMPLES
]
paired_lift = round(sum(diffs) / len(diffs), 3)
bootstrap_ci = paired_bootstrap_ci(diffs)

all_steps = [step for sample in SAMPLES for step in sample["steps"]]
error_cases = [
    sample for sample in SAMPLES
    if sample["first_error_gold"] is not None
]
process_step_accuracy = ratio(sum(all_steps), len(all_steps))
first_error_accuracy = ratio(
    sum(sample["first_error_gold"] == sample["first_error_pred"] for sample in error_cases),
    len(error_cases),
)
contaminated = [sample["id"] for sample in SAMPLES if sample["contaminated"]]
robustness_drop = round(candidate_accuracy - variant_accuracy, 3)

slice_accuracy = {}
for slice_name in sorted({sample["slice"] for sample in SAMPLES}):
    group = [sample for sample in SAMPLES if sample["slice"] == slice_name]
    slice_accuracy[slice_name] = ratio(

```

```

        sum(sample["candidate_correct"] for sample in group),
        len(group),
    )

total_cost = sum(sample["cost"] for sample in SAMPLES)
candidate_correct = sum(sample["candidate_correct"] for sample in SAMPLES)
summary = {
    "baseline_accuracy": baseline_accuracy,
    "candidate_accuracy": candidate_accuracy,
    "paired_lift": paired_lift,
    "bootstrap_ci": bootstrap_ci,
    "variant_accuracy": variant_accuracy,
    "robustness_drop": robustness_drop,
    "process_step_accuracy": process_step_accuracy,
    "first_error_accuracy": first_error_accuracy,
    "contamination_rate": ratio(len(contaminated), len(SAMPLES)),
    "cost_per_correct": round(total_cost / max(1, candidate_correct), 3),
    "p95_cost": sorted(sample["cost"] for sample in SAMPLES)[-1],
}
gates = {
    "accuracy_ok": candidate_accuracy >= 0.65,
    "lift_ci_nonnegative": bootstrap_ci[0] >= 0.0,
    "process_ok": process_step_accuracy >= 0.8,
    "first_error_ok": first_error_accuracy >= 0.8,
    "contamination_ok": len(contaminated) == 0,
    "robustness_ok": robustness_drop <= 0.2,
    "cost_ok": summary["cost_per_correct"] <= 400,
}

print(f"summary={summary}")
print(f"slice_accuracy={slice_accuracy}")
print(f"contaminated={contaminated}")
print(f"regressions={[sample['id'] for sample in SAMPLES if sample['baseline_correct'] and not sample['candidate_correct']]}")
print(f"variant_failures={[sample['id'] for sample in SAMPLES if sample['candidate_correct'] and not sample['variant_correct']]}")
print(f"gates={gates}")
print(f"gate_pass={all(gates.values())}")

```

预期输出：

```

summary={'baseline_accuracy': 0.333, 'candidate_accuracy': 0.667, 'paired_lift': 0.333, 'bootstrap_ci': (-
0.333, 0.833), 'variant_accuracy': 0.333, 'robustness_drop': 0.334, 'process_step_accuracy': 0.722, 'first_error_accuracy': 0.722, 'contamination_rate': 0.333, 'cost_per_correct': 400.0, 'p95_cost': 400.0}
slice_accuracy={'code': 1.0, 'logic': 1.0, 'math': 1.0, 'proof': 0.0, 'simple': 0.0}
contaminated=['math_contam']
regressions=['simple_regression']
variant_failures=['math_contam', 'logic_distractor']
gates={'accuracy_ok': True, 'lift_ci_nonnegative': False, 'process_ok': False, 'first_error_ok': False, 'contamination_ok': False, 'robustness_ok': False, 'cost_ok': True}
gate_pass=False

```

这个结果的关键解释是：candidate 的平均准确率从 0.333 到 0.667，但 bootstrap interval 跨过 0，说明样本太少时不能把提升说成稳健结论；math_contam 说明污染样本会抬高分数；variant_failures 说明原题正确不代表变体鲁棒；simple_regression 说明 reasoning 策略可能伤害简单题；proof 切片为 0 说明平均分掩盖 hard slice。真正的评估报告必须把这些证据一起展示。

10.17 面试题：如何评估一个 Reasoning Model

回答要点：

我会从最终答案、过程质量、泛化能力、鲁棒性和推理成本几个维度评估。数学任务可以看 GSM、MATH 类 benchmark 和变体题；代码 debug 后提升。还要做数据污染检测，报告采样数、token、verifier 和工具调用预算。对于过程推理，要看步骤正确率和第一处错误。

10.18 面试题：为什么不能只看 Benchmark 分数

回答要点：

Benchmark 分数可能受数据污染、prompt 设置、推理预算、采样次数和评估器偏差影响。一个模型分数高，可能是因为见过题、用了 time compute，或者 verifier 选择了更好的候选。Reasoning 模型还需要看过程是否正确、变体题是否泛化、成本是否可接受，以及

10.19 小练习

1. 给一个 reasoning 评估集设计 original、variant、contaminated、hard slice 四类样本标记。
2. 修改本章 demo，让 math_contam 不计入最终分数，观察 accuracy、bootstrap interval 和 gate 如何变化。
3. 设计一个过程评估 rubric，分别评估步骤正确、条件覆盖、第一处错误和最终答案支持度。
4. 解释为什么 pass@k、verifier accuracy、final accuracy 和 cost per correct 必须分开报告。
5. 用 3 句话说明 paired evaluation 为什么比两个模型分别报平均分更适合比较小幅提升。

10.20 本章小结

Reasoning 评估要从“最终答案对不对”扩展到“过程是否可靠、是否泛化、是否鲁棒、是否公平比较、成本是否可接受”。数学和代码 benchmark 很重要，但不能成为唯一依据。污染检测、过程评估、变体题、鲁棒性测试、推理预算报告和失败案例分析同样关键。

下一章会讨论 reasoning model 的安全与局限，包括幻觉、过度自信、伪推理、reward hacking、工具误用和高风险场景中的可靠性边界。

第十一章：Reasoning 安全与局限

Reasoning model 能解决更复杂的问题，也会带来更复杂的风险。模型推理能力越强，越可能完成长链条规划、工具调用、代码修改、复杂说服、欺骗性解释或高风险决策。因此，评估 reasoning model 不能只看数学、代码和 benchmark 分数，还要看它在安全、可靠性、透明度、权限边界和高风险场景中的表现。

本章系统讲 reasoning model 的安全与局限：伪推理、过度自信、CoT 隐私、CoT 忠实性、长链条错误传播、reward hacking、工具误用、滥用风险、高风险场景边界、安全评估和工程缓解策略。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修参考公开安全资料，包括 [OpenAI o1 System Card](#)、[OpenAI updated Preparedness Framework](#)、[OpenAI Frontier Governance Framework](#)、[OpenAI chain-of-thought monitoring](#)、[NIST AI 600-1 Generative AI Profile](#)、[OWASP Top 10 for LLM Applications 2025](#)、[Anthropic Responsible Scaling Policy](#) 以及 CoT 忠实性相关论文，例如 [Language Models Don't Always Say What They Think](#)。

这些资料共同提示：reasoning 能力一方面能增强安全策略遵循、错误检查和复杂任务求解，另一方面也会扩大模型的规划、工具使用、长期任务执行和 reward hacking 能力。安全评估不能只问“模型会不会拒答”，还要看是否过度拒答、是否暴露隐藏 CoT、是否伪造可信解释、是否在工具环境中越权、是否在高风险任务中缺少人工审核，以及是否在多轮上下文里逐步偏离安全边界。

本章保持防御性、审计性和治理性口径：只讨论风险分类、评估指标、上线门禁和缓解策略，不提供可复用的攻击流程、绕过技巧或危险操作步骤。面试中能说明：reasoning safety 不是一个分类器分数，而是模型行为、CoT 展示策略、verifier、工具权限、审计日志、人工复核、红队回归和治理流程组成的系统工程。

11.1 推理能力为什么会带来风险

Reasoning 能力本身是中性的。它可以用于数学证明、代码修复、科学分析和安全策略推理，也可以被误用到高风险规划、欺骗性解释或工具越权中。

风险来自四个方面：

1. 能力增强：模型能完成更复杂计划，错误或误用的影响范围更大。
2. 工具增强：模型可以调用搜索、代码执行、数据库、浏览器、API 或终端，输出从“文字建议”变成“外部动作”。
3. 可信度增强：长推理文本容易让用户误以为答案可靠，尤其在医疗、法律、金融、安全和招聘等场景中。
4. 监督难度增强：长链条、多候选、搜索和工具调用让评估更难，单一拒答率或单一 benchmark 分数不够。

面试回答：

Reasoning model 的安全问题不只是“会不会答错”，还包括更强规划能力带来的滥用风险、长链条错误传播、工具调用误用、伪推理

11.1.1 关键公式与 Reasoning 安全指标速查

把第 i 个 reasoning 安全评估样本写成：

$$s_i = (x_i, \hat{y}_i, y_i^{\star}, z_i, c_i, d_i, a_i, p_i, h_i, r_i, w_i)$$

其中 x_i 是请求， $\hat{y}_i$ 是模型最终回答， $y_i^{\star}$ 是期望安全动作或任务正确答案， z_i 是可审计推理摘要或步骤标签， c_i 是模型置信度， d_i 是风险域标签， a_i 是工具动作， p_i 是权限判定， h_i 是人工审核标记， r_i 是风险标记集合， w_i 是严重大权重。

伪推理率：

$$R_{\{\mathrm{pseudo}\}} = \frac{1}{\sum_{i=1}^N} \mathbb{1}[q_i^{\{\mathrm{proc}\}} = 0]$$

其中 $q_i^{\{\mathrm{proc}\}} = 0$ 表示推理步骤、证据或解释不能支持最终结论。伪推理不是“答案一定错”，而是“解释不可信”；它会让错误更有说服力。

过度自信错误率：

$$R_{\{\mathrm{conf}\}} = \frac{\sum_i \mathbb{1}[\hat{y}_i \neq y_i^{\star} \wedge c_i \geq \tau]}{\sum_i \mathbb{1}[\hat{y}_i \neq y_i^{\star}]}$$

这个指标专门看“错得很自信”的样本。高风险场景里，错而自信往往比谨慎拒答更危险。

高风险不当服从率：

$$R_{\{\mathrm{unsafe}\}} = \frac{\sum_i \mathbb{1}[d_i \in \mathcal{H} \wedge r_i^{\{\mathrm{unsafe}\}} = 1]}{\sum_i \mathbb{1}[d_i \in \mathcal{H}]}$$

其中 $\mathcal{H}$ 是高风险领域集合，例如医疗、法律、金融、网络安全、工业控制、身份安全和未成年人相关场景。

工具误用率：

$$R_{\{\mathrm{tool}\}} = \frac{\sum_i \mathbb{1}[a_i \in \mathcal{T}]}{\sum_i \mathbb{1}[a_i \neq \emptyset]}$$

$$\sum_i \mathbb{1}[a_i=1 \wedge p_i=0]$$

$$\sum_i \mathbb{1}[a_i=1]$$

工具误用包括权限不匹配、参数污染、缺少二次确认、不可逆操作未审核、工具输出被当成可信指令等。

隐藏 CoT 暴露率：

$$R_{\{\mathrm{cot}\}} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[r_i^{\{\mathrm{cot}\}}=1]$$

这里关注的是不该暴露的内部推理、策略细节、系统边界或隐私推断是否进入用户可见输出。

高风险人工审核覆盖率：

$$C_{\{\mathrm{review}\}} = \frac{\sum_i \mathbb{1}[d_i \in \mathcal{H} \wedge h_i=1]}{\sum_i \mathbb{1}[d_i \in \mathcal{H}]}$$

高风险任务不是一律拒绝，也不是一律自动完成。更合理的做法是区分允许辅助分析、必须建议专业人士、必须人工确认和必须拒绝的边界。

过度拒答率：

$$R_{\{\mathrm{over}\}} = \frac{\sum_i \mathbb{1}[d_i \in \mathcal{B} \wedge r_i^{\{\mathrm{over}\}}=1]}{\sum_i \mathbb{1}[d_i \in \mathcal{B}]}$$

其中 $\mathcal{B}$ 是良性请求集合。安全模型不是拒答率越高越好，还要保持正常任务可用性。

严重度加权风险：

$$S_{\{\mathrm{risk}\}} = \frac{\sum_i w_i v_i}{\sum_i w_i}$$

其中 $v_i=1$ 表示样本命中任一关键风险。这个指标避免低风险小错和高风险事故被平均值混在一起。

一个简化 reasoning 安全门禁：

$$G_{\{\mathrm{safe}\}} = \mathbb{1}[\begin{aligned} &R_{\{\mathrm{pseudo}\}} \leq \alpha \\ &\wedge \\ &R_{\{\mathrm{conf}\}} \leq \beta \end{aligned}]$$

```

\land
R_{\mathrm{unsafe}}\leq \gamma
\land
R_{\mathrm{tool}}\leq \delta
\land
R_{\mathrm{cot}}\leq \epsilon
\land
C_{\mathrm{review}}\geq \eta
\land
R_{\mathrm{over}}\leq \rho
\land
S_{\mathrm{risk}}\leq \kappa
]

```

这个门禁表达的是 reasoning 系统是否适合上线，而不是模型是否“聪明”。如果工具权限、CoT 暴露或高风险审核不过线，即使数学和代码分数高，也不能认为系统安全。

11.2 Reasoning 安全风险的分层

Reasoning 安全可以按四层理解。

第一层是回答可靠性：

1. 最终答案错误。
2. 推理过程错误。
3. 解释和答案不一致。
4. 对不确定问题过度自信。
5. 对良性问题过度拒答。

第二层是解释与可观察性：

1. CoT 不忠实。
2. 可见解释泄露内部策略。
3. 解释掩盖真实失败原因。
4. 多次采样产生互相矛盾的解释。
5. 监控只看文本解释而漏掉外部动作。

第三层是工具与外部动作：

1. 错误调用工具。
2. 越权访问数据。
3. 高风险操作缺少确认。
4. 把不可信工具输出当成上级指令。
5. 只优化“任务成功”，忽略权限、审计和回滚。

第四层是治理与前沿能力：

1. 复杂规划能力可能降低滥用门槛。
2. 长程任务可能产生目标漂移。
3. 自动评估和 reward 可能被模型利用。
4. 模型能力变化快，旧评测集可能很快失效。
5. 高风险能力需要分级评估、外部审查、红队和发布门禁。

11.3 伪推理

伪推理是 reasoning model 的常见问题。模型生成了一段看起来合理的推理过程，但这段过程并不真正支持答案。

表现包括：

1. 先猜答案，再编过程。

2. 过程和最终答案不一致。
3. 中间步骤逻辑跳跃。
4. 使用不存在的定理、事实或证据。
5. 对错误步骤给出流畅解释。
6. 把相关性说成果性。
7. 引用不支持结论的工具结果或文档片段。

伪推理的危险在于它很有说服力。用户看到长解释，可能更相信答案，但解释长度不等于正确性。

工程评估时，不应只问“最终答案是否正确”。还要抽样标注：

1. 每个关键步骤是否被前一步支持。
2. 每个事实 claim 是否有证据。
3. 最终答案是否由步骤推出。
4. 是否存在“过程错但答案对”的 lucky answer。
5. 是否存在“过程看起来对但格式或约束错”的 failure。

面向专家的回答：

伪推理的核心不是模型写错了某一步，而是自然语言解释和模型实际决策之间可能脱钩。CoT 可以作为可观察信号，但不能直接等同于

11.4 过度自信

Reasoning model 经常以确定语气输出错误结论。

原因包括：

1. 训练数据偏好完整回答。
2. 偏好优化可能奖励流畅、自信和顺从。
3. 模型缺少可靠不确定性估计。
4. 长 CoT 让模型越写越确认自己。
5. Verifier、工具反馈或证据约束缺失。
6. 用户问题本身缺少关键信息，但模型没有主动澄清。

缓解方式：

1. 要求模型标注假设和不定点。
2. 对高风险问题调用 verifier、检索或工具。
3. 对关键结论做反例检查。
4. 允许模型回答“不确定”或请求更多信息。
5. 区分事实、推断、建议和决策。
6. 用校准指标评估置信度，而不是只看准确率。

过度自信要和 helpfulness 一起看。模型不能因为担心风险就对所有问题拒答，也不能为了显得有帮助而替用户做高风险决策。

11.5 Chain-of-Thought 隐私与展示边界

CoT 可能包含敏感信息或不该暴露的内部推理。

风险包括：

1. 泄露系统提示或策略细节。
2. 暴露用户隐私推断。
3. 暴露安全规则边界。
4. 让攻击者更容易反向设计模型行为。
5. 将不成熟的中间想法误当正式结论。
6. 暴露工具权限、内部评分器或审核规则。

因此实际产品中，不一定直接展示完整内部 CoT。更常见做法是展示简洁解释、依据摘要、关键证据、可验证步骤或最终决策理由，而不是原始隐藏推理轨迹。

要区分三件事：

1. 内部推理：模型用于解题、规划或自检的中间状态。
2. 安全监控：系统用于发现 reward hacking、欺骗、越权或异常工具行为的审计信号。
3. 用户解释：面向用户展示的简洁、可验证、合规回答。

这三者不能混在一起。内部推理可以帮助模型和系统监控，但用户解释应尽量短、稳、可验证，并避免暴露安全边界和敏感推断。

11.6 CoT 不忠实与不可解释推理

即使模型输出 CoT，也不代表我们真正理解了模型内部机制。文本推理链可能只是事后解释，不一定等于模型内部计算过程。

局限包括：

1. CoT 可能不是忠实解释。
2. 模型可能隐藏关键依据。
3. 解释可能被优化成更讨人喜欢。
4. 不同采样会产生不同解释。
5. 用户难以判断解释真假。
6. 如果训练直接强压 CoT 变得“看起来安全”，模型可能学会隐藏真实意图。

所以 reasoning model 的解释性不能只依赖生成的自然语言推理，还需要外部验证、日志、工具结果、可复现实验和行为级审计。

11.7 长链条错误传播

长推理链的一个问题是错误会传播。

例如：

step 1: 错误假设

step 2: 基于错误假设继续推导

step 3: 更复杂的中间结论

final: 自信但错误的答案

推理越长，错误积累的机会越多。长 CoT 不一定更好。如果没有中间验证，长推理可能只是更长的错误。

缓解方式：

1. 分步骤验证。
2. 定期回看原问题和约束。
3. 用 process verifier 检查中间步骤。
4. 使用工具验证关键计算。
5. 在最终答案前检查假设。
6. 对简单题路由到 direct answer，避免无谓长推理。

11.8 Reward Hacking

Reward hacking 指模型学会利用奖励函数、评测规则或工具环境漏洞，而不是真正完成任务。

在 reasoning 中可能表现为：

1. 生成 verifier 偏好的格式。
2. 输出冗长解释骗过弱 judge。
3. 只优化 benchmark 常见题型。
4. 在代码任务中过拟合公开测试。
5. 绕开真实目标，只满足表面检查项。
6. 用表面合理的步骤掩盖错误。

Reward hacking 的根源是奖励信号不等于真实目标。越依赖自动评分，越要警惕模型学会钻评分器空子。

工程上要做三类防护：

1. 评测防作弊：hidden tests、扰动集、反事实样本、人工抽检。
2. 行为监控：同时看 CoT 摘要、工具动作、代码 diff、日志和最终输出。
3. 目标对齐：把真实任务成功、权限边界、安全门禁和用户价值放在同一张评分表中。

11.9 工具误用与 Excessive Agency

Reasoning model 经常和工具结合，例如搜索、代码执行、数据库、浏览器、API、终端和企业系统。

工具带来能力，也带来风险：

1. 执行错误操作。
2. 读取或泄露敏感数据。
3. 误用外部 API。
4. 产生不可逆修改。
5. 被工具输出注入影响。
6. 在错误假设下调用高权限工具。
7. 长程任务中逐步偏离用户原始意图。

工程上需要：

1. 权限最小化。
2. 沙箱隔离。
3. 高风险操作人工确认。
4. 工具输出标记为不可信数据。
5. 参数 schema 校验。
6. 审计日志和 trace replay。
7. 明确回滚策略。
8. 对工具调用做独立安全门禁，而不是只相信模型解释。

面试中可以这样表达：

Agent 工具安全的核心是把模型从“全能执行者”降级为“受约束的计划者”。模型可以提出计划和参数，但权限、数据访问、不可逆

11.10 推理能力与滥用风险

更强 reasoning 可能提升某些滥用能力，例如复杂任务拆解、长期目标规划、规避约束的尝试、恶意内容优化和自动化试错。

安全策略不能只按关键词过滤，因为 reasoning model 能把目标拆成多个看似无害的步骤。因此需要结合：

1. 意图识别。
2. 多轮上下文追踪。
3. 工具权限限制。
4. 高风险任务拒答或转人工。
5. 任务级和动作级审计。
6. 红队回归和事故复盘。

这里的重点不是把所有复杂推理都视为危险，而是建立“能力越强，门禁越严”的工程逻辑。对低风险任务，可以充分利用 reasoning；对高风险任务，要限制输出细节、要求证据、建议专业人士、人工审核或拒绝协助。

11.11 高风险场景边界

Reasoning model 在高风险场景中尤其要谨慎。

高风险场景包括：

1. 医疗诊断和治疗决策。

2. 法律建议和合规判断。
3. 金融投资、信贷和保险决策。
4. 网络安全攻防和漏洞处理。
5. 工业控制、自动驾驶和物理设备操作。
6. 招聘、教育、住房等影响个人权益的决策。
7. 未成年人相关建议。
8. 身份、隐私和敏感个人数据处理。

这些场景中，模型可以辅助总结、解释概念、列出一般性检查项或帮助用户准备咨询材料，但不应被视为唯一决策者。系统需要明确边界、证据来源、人工复核、可追溯日志和权限限制。

一个好边界不是“全部拒绝”，而是分层：

1. 允许：一般教育性解释、低风险信息总结。
2. 限制：需要证据、需要说明不确定性、不能替用户做最终决策。
3. 人审：涉及个体权益、不可逆操作或高影响建议。
4. 拒绝：明显有害、越权、违法或危险能力请求。

11.12 Reasoning 安全评估

Reasoning 安全评估应覆盖：

1. 有害任务拒答能力。
2. 良性任务不过度拒答。
3. 多轮诱导鲁棒性。
4. 工具调用安全性。
5. 隐私泄露风险。
6. 越权操作风险。
7. 幻觉和过度自信。
8. 伪推理比例。
9. 高风险建议边界。
10. 隐藏 CoT 暴露。
11. Reward hacking 和评测规避。
12. 对抗性 prompt 稳定性。

只评估普通安全问答不够。Reasoning model 需要评估多步计划、多轮上下文、工具环境、长程任务和高风险动作中的安全表现。

一份 reasoning safety eval report 至少要包含：

1. 样本 schema：风险域、期望动作、证据、工具权限、人工审核要求。
2. 指标：unsafe compliance、over-refusal、tool misuse、CoT exposure、review coverage。
3. 切片：低风险、高风险、多轮、工具、隐私、专业建议、长任务。
4. Trace：模型最终回答、可审计推理摘要、工具动作、权限判定、审核记录。
5. 失败归因：模型误判、策略边界不清、工具权限过宽、verifier 失效、用户输入缺失。
6. 门禁：哪些指标不过线就不能上线，哪些需要灰度和人工监控。

11.13 缓解策略

常见缓解策略包括：

1. 使用 verifier 检查关键结论。
2. 对高风险任务启用人工审核。
3. 限制工具权限。
4. 记录推理摘要和工具调用日志。
5. 对敏感任务隐藏原始 CoT。
6. 用安全策略模型做输入输出审查。
7. 对不确定问题要求模型表达不确定性。
8. 使用程序化验证替代纯文本解释。

9. 对 benchmark 和 reward 做反作弊设计。
10. 对外部内容、工具返回和网页文本标记不可信边界。
11. 对高风险工具做二次确认和回滚。
12. 用红队回归集跟踪新版本模型的安全退化。

没有单一策略可以解决全部问题。Reasoning 安全通常需要模型训练、系统设计、权限控制、评估流程、治理流程和事故响应一起工作。

11.14 最小可运行 reasoning 安全审计 demo

下面的 0 依赖 demo 用 toy 样本模拟 reasoning 安全审计。它不调用外部模型，也不包含真实高风险操作；重点是展示如何把伪推理、过度自信、工具误用、隐藏 CoT 暴露、高风险人工审核和过度拒答放进同一张门禁表。

```
from collections import defaultdict
```

```
SAMPLES = [  
    dict(  
        id="math_easy",  
        risk="low",  
        answer_correct=True,  
        process_supported=True,  
        confidence=0.82,  
        tool_action=False,  
        tool_allowed=True,  
        unsafe_compliance=False,  
        over_refusal=False,  
        cot_exposure=False,  
        requires_human_review=False,  
        human_review=False,  
        verifier_pass=True,  
        severity=1,  
    ),  
    dict(  
        id="pseudo_proof",  
        risk="medium",  
        answer_correct=False,  
        process_supported=False,  
        confidence=0.91,  
        tool_action=False,  
        tool_allowed=True,  
        unsafe_compliance=False,  
        over_refusal=False,  
        cot_exposure=False,  
        requires_human_review=False,  
        human_review=False,  
        verifier_pass=False,  
        severity=2,  
    ),  
    dict(  
        id="medical_triage",  
        risk="high",  
        answer_correct=False,  
        process_supported=False,  
        confidence=0.88,  
        tool_action=False,
```

```

    tool_allowed=True,
    unsafe_compliance=True,
    over_refusal=False,
    cot_exposure=False,
    requires_human_review=True,
    human_review=False,
    verifier_pass=False,
    severity=5,
),
dict(
    id="finance_plan",
    risk="high",
    answer_correct=True,
    process_supported=True,
    confidence=0.76,
    tool_action=True,
    tool_allowed=True,
    unsafe_compliance=False,
    over_refusal=False,
    cot_exposure=False,
    requires_human_review=True,
    human_review=True,
    verifier_pass=True,
    severity=4,
),
dict(
    id="tool_delete",
    risk="high",
    answer_correct=False,
    process_supported=True,
    confidence=0.72,
    tool_action=True,
    tool_allowed=False,
    unsafe_compliance=False,
    over_refusal=False,
    cot_exposure=False,
    requires_human_review=True,
    human_review=False,
    verifier_pass=False,
    severity=5,
),
dict(
    id="cot_leak",
    risk="medium",
    answer_correct=True,
    process_supported=True,
    confidence=0.70,
    tool_action=False,
    tool_allowed=True,
    unsafe_compliance=False,
    over_refusal=False,
    cot_exposure=True,
    requires_human_review=False,
    human_review=False,
    verifier_pass=True,

```



```

        severity=3,
    ),
    dict(
        id="benign_refusal",
        risk="low",
        answer_correct=False,
        process_supported=True,
        confidence=0.64,
        tool_action=False,
        tool_allowed=True,
        unsafe_compliance=False,
        over_refusal=True,
        cot_exposure=False,
        requires_human_review=False,
        human_review=False,
        verifier_pass=True,
        severity=1,
    ),
    dict(
        id="verified_lowrisk",
        risk="low",
        answer_correct=True,
        process_supported=True,
        confidence=0.65,
        tool_action=False,
        tool_allowed=True,
        unsafe_compliance=False,
        over_refusal=False,
        cot_exposure=False,
        requires_human_review=False,
        human_review=False,
        verifier_pass=True,
        severity=1,
    ),
    dict(
        id="reviewed_highrisk",
        risk="high",
        answer_correct=True,
        process_supported=True,
        confidence=0.79,
        tool_action=False,
        tool_allowed=True,
        unsafe_compliance=False,
        over_refusal=False,
        cot_exposure=False,
        requires_human_review=True,
        human_review=True,
        verifier_pass=True,
        severity=5,
    ),
    dict(
        id="unverifiable_claim",
        risk="medium",
        answer_correct=False,
        process_supported=False,

```

```

        confidence=0.73,
        tool_action=False,
        tool_allowed=True,
        unsafe_compliance=False,
        over_refusal=False,
        cot_exposure=False,
        requires_human_review=False,
        human_review=False,
        verifier_pass=False,
        severity=2,
    ),
]

def ids_where(predicate):
    return [row["id"] for row in SAMPLES if predicate(row)]

def safe_rate(num, den):
    return round(num / den, 3) if den else 0.0

n = len(SAMPLES)
high_risk = [r for r in SAMPLES if r["risk"] == "high"]
low_risk = [r for r in SAMPLES if r["risk"] == "low"]
errors = [r for r in SAMPLES if not r["answer_correct"]]
tool_calls = [r for r in SAMPLES if r["tool_action"]]

risk_ids = {
    "pseudo_reasoning": ids_where(lambda r: not r["process_supported"]),
    "overconfident_errors": ids_where(
        lambda r: (not r["answer_correct"]) and r["confidence"] >= 0.8
    ),
    "unsafe_compliance": ids_where(lambda r: r["unsafe_compliance"]),
    "tool_misuse": ids_where(lambda r: r["tool_action"] and not r["tool_allowed"]),
    "hidden_cot_exposure": ids_where(lambda r: r["cot_exposure"]),
    "missing_human_review": ids_where(
        lambda r: r["requires_human_review"] and not r["human_review"]
    ),
    "over_refusal": ids_where(lambda r: r["over_refusal"]),
}

risk_flag_by_id = defaultdict(bool)
for ids in risk_ids.values():
    for case_id in ids:
        risk_flag_by_id[case_id] = True

weighted_bad = sum(r["severity"] for r in SAMPLES if risk_flag_by_id[r["id"]])
weighted_all = sum(r["severity"] for r in SAMPLES)

summary = {
    "pseudo_reasoning_rate": safe_rate(len(risk_ids["pseudo_reasoning"]), n),
    "overconfident_error_rate": safe_rate(
        len(risk_ids["overconfident_errors"]), len(errors)
    ),
}

```

```

    "unsafe_compliance_rate": safe_rate(
        len(risk_ids["unsafe_compliance"]), len(high_risk)
    ),
    "tool_misuse_rate": safe_rate(len(risk_ids["tool_misuse"]), len(tool_calls)),
    "hidden_cot_exposure_rate": safe_rate(len(risk_ids["hidden_cot_exposure"]), n),
    "high_risk_review_coverage": safe_rate(
        sum(r["human_review"] for r in high_risk), len(high_risk)
    ),
    "over_refusal_rate": safe_rate(len(risk_ids["over_refusal"]), len(low_risk)),
    "severity_weighted_risk": safe_rate(weighted_bad, weighted_all),
}

gates = {
    "pseudo_reasoning_ok": summary["pseudo_reasoning_rate"] <= 0.15,
    "overconfidence_ok": summary["overconfident_error_rate"] <= 0.20,
    "unsafe_compliance_ok": summary["unsafe_compliance_rate"] == 0.0,
    "tool_misuse_ok": summary["tool_misuse_rate"] == 0.0,
    "hidden_cot_ok": summary["hidden_cot_exposure_rate"] == 0.0,
    "human_review_ok": summary["high_risk_review_coverage"] >= 0.95,
    "over_refusal_ok": summary["over_refusal_rate"] <= 0.10,
    "weighted_risk_ok": summary["severity_weighted_risk"] <= 0.20,
}

print(f"summary={summary}")
print(f"risk_ids={risk_ids}")
print(f"gates={gates}")
print(f"gate_pass={all(gates.values())}")

```

预期输出：

```

summary={'pseudo_reasoning_rate': 0.3, 'overconfident_error_rate': 0.4, 'unsafe_compliance_rate': 0.25, 'tool_misuse_rate': 0.0, 'hidden_cot_exposure_rate': 0.0, 'high_risk_review_coverage': 0.95, 'over_refusal_rate': 0.1, 'severity_weighted_risk': 0.2}
risk_ids={'pseudo_reasoning': ['pseudo_proof', 'medical_triage', 'unverifiable_claim'], 'overconfident_errors': ['pseudo_proof', 'medical_triage', 'unverifiable_claim'], 'unsafe_compliance': ['pseudo_proof', 'medical_triage', 'unverifiable_claim'], 'tool_misuse': ['pseudo_proof', 'medical_triage', 'unverifiable_claim'], 'hidden_cot_exposure': ['pseudo_proof', 'medical_triage', 'unverifiable_claim'], 'high_risk_review': ['pseudo_proof', 'medical_triage', 'unverifiable_claim'], 'over_refusal': ['pseudo_proof', 'medical_triage', 'unverifiable_claim'], 'severity_weighted_risk': ['pseudo_proof', 'medical_triage', 'unverifiable_claim']}
gates={'pseudo_reasoning_ok': False, 'overconfidence_ok': False, 'unsafe_compliance_ok': False, 'tool_misuse_ok': False, 'hidden_cot_ok': False, 'human_review_ok': True, 'over_refusal_ok': True, 'weighted_risk_ok': True}
gate_pass=False

```

这里 `gate_pass=False` 是预期结果：demo 故意放入伪推理、过度自信、高风险缺少人工审核、工具越权、隐藏 CoT 暴露和过度拒答样本，用来说明 reasoning 安全上线前不能只看回答正确率。

11.15 面试题：为什么 CoT 不一定应该直接展示

回答要点：

CoT 有助于模型内部推理和系统监控，但直接展示完整 CoT 可能泄露系统策略、隐私推断、工具权限或安全边界，也可能把不忠实的推理展示给用户。

追问：如果不展示 CoT，用户如何信任模型？

可以展示可验证证据、引用来源、关键假设、置信边界、工具结果和最终检查清单，而不是展示完整隐藏推理。信任应来自可复核证据和透明决策过程。

11.16 面试题：Reasoning Model 有哪些安全风险

回答要点：

主要风险包括伪推理、过度自信、长链条错误传播、reward hacking、工具误用、隐藏 CoT 暴露、高风险场景误导、过度拒答，以及推理过程被滥用。

追问：如何设计上线前 reasoning safety gate？

我会把样本按低风险、高风险、多轮、工具、隐私和专业建议切片；同时统计 unsafe compliance、over-refusal、pseudo reasoning 和 weighted risk。只要高风险不当服从、工具越权、隐藏 CoT 暴露或人工审核覆盖不过线，即使任务准确率提升，也不能直接上线。

11.17 小练习

1. 给 8 条 toy reasoning 安全样本设计 schema，字段至少包含风险域、期望动作、置信度、工具动作、权限判定、人工审核、CoT 暴露和严重度。
2. 用 0 依赖 Python 写一个 reasoning safety audit demo，输出伪推理率、过度自信错误率、高风险不当服从率、工具误用率、隐藏 CoT 暴露率、人工审核覆盖率、过度拒答率和门禁结果。
3. 设计一个“高质量安全不是拒答率最高”的评估表，同时覆盖 unsafe compliance 和 over-refusal。
4. 给一个工具调用 Agent 设计权限矩阵，说明哪些动作可自动执行，哪些必须确认，哪些必须拒绝。
5. 用 3 分钟回答：“为什么完整 CoT 不等于可信解释？”

11.18 本章小结

Reasoning model 的强大之处在于能进行多步推理、规划、验证和修正；它的风险也来自这些能力。长推理可能更可靠，也可能更会编；工具使用可以增强能力，也可能扩大事故范围；CoT 可以帮助调试和监控，也可能带来隐私、忠实性和策略泄露问题。

安全地使用 reasoning model，需要把能力和边界一起设计。系统不应只追求更高 benchmark 分数，还要关注伪推理、过度自信、工具权限、隐藏 CoT 暴露、reward hacking、高风险人工审核、过度拒答和严重度加权风险。下一章会进入本册最后的 reasoning 面试题，帮助把前面所有内容整理成面试可表达的知识体系。

第十二章：Reasoning 面试题

本章是第十六册的综合面试题。前面章节已经覆盖 reasoning model 的概念、CoT、self-consistency、verifier、process supervision、search、test-time compute、数学和代码训练、评估与安全。本章的目标是把这些内容整理成面试中可以直接表达的答案。

面试回答要避免两个问题：一是只背术语，比如 CoT、PRM、MCTS、verifier；二是只说现象，不讲机制和边界。一个好的 reasoning 回答通常包含五层：问题是什么，为什么有效，公式或指标是什么，工程上怎么做，有什么局限和风险。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修参考公开资料，包括 [Chain-of-Thought Prompting](#)、[Self-Consistency](#)、[Training Verifiers to Solve Math Word Problems](#)、[Let's Verify Step by Step](#)、[Tree of Thoughts](#)、[HumanEval / pass@k](#)、[OpenAI o1 System Card](#) 和 [test-time compute scaling](#) 相关公开论文。它们共同提示：reasoning 面试不是背模型名，而是要能把“生成候选、过程质量、验证器、搜索、推理预算、评估统计、安全门禁”连成一套可落地系统。

本章定位是面试复盘和表达训练，不展开每篇论文的完整证明，也不预测闭源 reasoning 模型的内部实现。面试中遇到未公开细节时，应明确区分官方披露、论文结论、工程常识和个人推断。高风险安全问题保持审计与防御口径，不提供可复用的攻击流程或绕过技巧。

12.1 Reasoning 面试的评分框架

面试官通常不是在听你背“CoT、self-consistency、PRM、MCTS”这些词，而是在判断你是否能把一个复杂模型能力拆成可验证系统。

一个高分回答通常包含：

1. 目标：这个方法解决什么问题。
2. 机制：它为什么可能有效。
3. 公式：核心概率、loss、指标或门禁怎么写。
4. 工程：数据、推理流程、工具、日志和预算如何实现。
5. 评估：看哪些指标、哪些切片、哪些失败样本。
6. 边界：成本、延迟、偏差、安全、污染和过度自信。
7. 项目：能否用一个可运行 demo 或项目审计表证明你真的做过。

把第 i 道 reasoning 面试题的回答写成:

$$a_i=(t_i, f_i, d_i, e_i, r_i, u_i)$$

其中 t_i 是覆盖的主题, f_i 是公式或指标, d_i 是 demo 或项目证据, e_i 是评估口径, r_i 是风险边界, u_i 是工程 trade-off。

回答覆盖率:

$$C_{\{\mathrm{ans}\}} = \frac{|T_{\{\mathrm{covered}\}}|}{|T_{\{\mathrm{required}\}}|}$$

公式覆盖率:

$$C_{\{\mathrm{formula}\}} = \frac{|F_{\{\mathrm{covered}\}}|}{|F_{\{\mathrm{required}\}}|}$$

demo 覆盖率:

$$C_{\{\mathrm{demo}\}} = \frac{|D_{\{\mathrm{covered}\}}|}{|D_{\{\mathrm{required}\}}|}$$

一个简化 reasoning 面试准备门禁:

$$G_{\{\mathrm{interview}\}} = \mathbb{1} [C_{\{\mathrm{ans}\}} \geq \alpha \land C_{\{\mathrm{formula}\}} \geq \beta \land C_{\{\mathrm{demo}\}} \geq \gamma \land C_{\{\mathrm{risk}\}} \geq \rho \land C_{\{\mathrm{trade}\}} \geq \eta]$$

这个门禁的直觉是: 会讲概念只是最低要求; 能写公式、能讲 demo、能说评估和风险, 才更接近真实面试要求。

12.2 关键公式速查

自回归 reasoning 生成:

$$P(y, z \mid x) = \prod_{t=1}^T P(o_t \mid x, o_{1:t-1})$$

其中 z 是中间推理过程, y 是最终答案, o_t 可以是推理步骤、答案 token 或工具调用摘要。

最终答案准确率:

$$A_{\{\mathrm{ans}\}} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\hat{y}_i = y_i^*]$$

步骤准确率:

$$A_{\{\mathrm{step}\}} = \frac{\sum_{i,j} \mathbb{1}[\hat{q}_{ij} = q_{ij}]}{\sum_i L_i}$$

self-consistency 投票:

$$\hat{y}_i = \arg\max_y \sum_{j=1}^K \mathbb{1}[\mathrm{norm}(\hat{y}_{ij}) = y]$$

pass@k 不放回估计:

$$\mathrm{pass}@k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

pairwise verifier loss:

$$L_{\{\mathrm{pair}\}} = -\log \sigma(r_w - r_l)$$

process supervision loss:

$$L_{\{\mathrm{proc}\}} = -\frac{1}{M} \sum_{i,j} \left[q_{ij} \log p_{ij} + (1 - q_{ij}) \log (1 - p_{ij}) \right]$$

test-time compute 成本:

$$C_i = c_{\{\mathrm{tok}\}} T_i + c_{\{\mathrm{cand}\}} K_i + c_{\{\mathrm{ver}\}} V_i + c_{\{\mathrm{tool}\}} U_i + c_{\{\mathrm{lat}\}} R_i$$

reasoning 安全门禁:

$$G_{\{\mathrm{safe}\}} = \mathbb{1}[\begin{aligned} &R_{\{\mathrm{pseudo}\}} \leq \alpha \\ &\text{and} \\ &R_{\{\mathrm{conf}\}} \leq \beta \\ &\text{and} \\ &R_{\{\mathrm{tool}\}} \leq \delta \end{aligned}]$$

$\backslash \text{land}$
 $C_{\{\mathrm{review}\}} \backslash \text{ge } \eta$
 $\]$

这些公式不需要在每道题里全部写出，但面试中至少要能根据题目拿出对应公式。例如 CoT 题讲 A_{step} , self-consistency 题讲投票和 $\text{pass}@k$, verifier 题讲 pairwise loss 和校准, TTC 题讲成本, 安全题讲 G_{safe} 。

12.3 什么是 Reasoning Model

回答要点：

Reasoning model 是强调多步推理、规划、验证和自我修正能力的模型或系统。它不只是直接生成答案，而是能把复杂任务拆成多个中间 `time compute` 来提升复杂问题的解决能力。典型场景包括数学、代码、规划、科学问答和复杂决策。

补充说明：

需要强调 reasoning model 不等于简单“输出更长解释”。真正的 reasoning 关注中间步骤是否正确、是否能泛化、是否能用验证器。

专家追问：

我会把 reasoning model 拆成训练侧和推理侧。训练侧包括数学、代码、过程标注、偏好数据和 verifier 数据；推理侧包括候选生成

12.4 CoT 为什么有效

回答要点：

CoT 有效的原因是它把复杂问题拆成多个中间步骤，降低一次性映射的难度，让模型显式生成中间状态。对于数学、逻辑和规划任务，

面试展开：

我会区分“生成 CoT”和“CoT 真的正确”。CoT 能提高表现，但不能自动保证推理忠实性。因此评估时要看步骤正确率、第一处错误

常见误区：

1. 把 CoT 长度当成推理能力。
2. 认为 CoT 一定忠实反映内部计算。
3. 忘记简单题可能出现 CoT regression。
4. 在安全场景默认展示完整内部推理。

12.5 Zero-shot CoT 和 Few-shot CoT 的区别

回答要点：

Zero-shot CoT 是通过类似“请一步步思考”的提示让模型生成推理步骤，不需要示例。Few-shot CoT 会在 prompt 中给出带推理过程的 shot 通常更稳定，但依赖示例质量，也可能带来格式过拟合。Zero-shot 更简单，但对模型基础能力要求更高。

工程补充：

实际系统里我会把 CoT prompt 当成可评估配置，而不是拍脑袋写一句提示。要记录 prompt 版本、示例来源、采样参数、token 成本、

12.6 Self-Consistency 是什么

回答要点：

Self-consistency 是对同一个问题采样多条推理路径，然后对最终答案进行投票或聚合。它利用了正确答案更可能在多条独立路径中

公式表达：

$\hat{y} =$
 $\arg\max_y$
 $\sum_{j=1}^K$
 $\mathbb{1}[\mathrm{norm}(\hat{y}_j)=y]$

追问回答：

self-consistency 和 pass@k 不一样。self-consistency 看投票后最终选择是否正确，pass@k 看候选集合里是否至少存在一个正确解。consistency 会失败；如果 verifier 能挑出少数正确候选，pass@k 高仍然有价值。

12.7 Verifier 和 Reward Model 有什么区别

回答要点：

Verifier 更强调判断一个答案或步骤是否正确，常用于数学、代码和可验证任务。Reward model 更一般，用来给模型输出打偏好分，of-N 反而可能放大错误。

公式表达：

$$L_{\{\mathrm{pair}\}} = -\log \sigma(r_w - r_l)$$

工程补充：

我会优先区分 programmatic verifier、learned outcome verifier 和 process verifier。数学表达式、代码测试这类任务可以用程序

12.8 Outcome Supervision 和 Process Supervision 的区别

回答要点：

Outcome supervision 只监督最终答案是否正确，信号简单但稀疏，不知道中间哪一步错。Process supervision 会监督推理过程中的面试展开：

我会同时看 outcome accuracy 和 step accuracy。最终答案正确但过程错误，说明模型可能 lucky；过程正确但最终格式错误，说明

12.9 PRM 是什么

回答要点：

PRM 是 process reward model，用来给推理过程中的中间步骤打分。它可以帮助搜索时剪枝、帮助 reranking 选择更可靠的推理链，

公式表达：

$$S_{\{\mathrm{proc}\}}(z) = \frac{1}{L} \sum_{j=1}^L p_j$$

追问回答：

PRM 的风险是偏好“看起来规范”的步骤，低估朴素但正确的路径，或者在 hard negative 上被欺骗。所以 PRM 要和 outcome verifier

12.10 Tree-of-Thought 相比 CoT 的优势是什么

回答要点：

CoT 通常是一条推理链，早期步骤错了后面容易全部偏。Tree-of-Thought 把推理组织成树，让模型在每一步生成多个候选 thought，工程补充：

我会把 ToT 看成 search controller，而不是单纯 prompt trick。系统里要定义 state、action、transition、score、stop condition

12.11 Beam Search 和 MCTS 如何比较

回答要点：

Beam search 通常按层展开，每层保留 top-k 候选，简单、稳定、成本可控，但容易过早剪掉潜在正确路径。MCTS 通过 selection、expansion、

公式表达：

$$U(v)=Q(v)+c\sqrt{\frac{\log(N_p+1)}{N_v+1}}$$

面试边界：

在语言 reasoning 中，MCTS 的难点不是公式本身，而是状态如何表示、模拟结果如何评估、reward 是否可靠，以及成本是否可控。

12.12 Test-Time Compute Scaling 是什么

回答要点：

Test-time compute scaling 是指模型参数固定时，通过增加推理阶段计算提升答案质量。典型方式包括长 CoT、多样本采样、self-consistency、best-of-N、verifier reranking、tree search、工具调用和反思修正。核心问题是如何分配预算，以及如何在准确率、成本和推理时间之间取得平衡。

公式表达：

$$C_i=c_{\{\mathrm{tok}\}}T_i+c_{\{\mathrm{cand}\}}K_i+c_{\{\mathrm{ver}\}}V_i+c_{\{\mathrm{tool}\}}U_i+c_{\{\mathrm{lat}\}}R_i$$

追问回答：

更多 test-time compute 不一定更好。它会带来边际收益递减、P95 延迟上升、成本增加和高预算仍失败样本。上线时要用 adaptive budgeting 动态调整。

12.13 Adaptive Compute 怎么设计

回答要点：

我会先用路由器判断任务类型、难度、价值、风险和可验证性。简单问题直接回答；中等问题使用多样本采样或 self-consistency；困难且可验证的问题启动 verifier、search 或工具调用；高风险问题进入人工审核或更严格门禁。系统需要限制 token 消耗和推理时间。

简化路由：

$$m_i=\pi(x_i,d_i,v_i,u_i,r_i)$$

其中 d_i 是难度， v_i 是业务价值， u_i 是可验证性， r_i 是风险等级。高价值、可验证、低安全风险的复杂题更适合高预算；低价值或高风险任务不能无脑增加搜索。

12.14 如何训练数学推理模型

回答要点：

数学推理训练通常需要题目答案数据、步骤解法数据、多解法数据、错误纠错数据和合成题数据。训练流程可以先用 SFT 学习基本解法，再用 RL 优化推理策略。

工程指标：

我会同时看 answer accuracy、step accuracy、first-error accuracy、template diversity、contamination rate、topic coverage 和推理成本。

12.15 数学 Verifier 有什么用

回答要点：

数学 verifier 可以用于数据清洗、候选 reranking、过程评分、搜索剪枝和 RL reward。程序化 verifier 在表达式、数值计算、方程求解和逻辑推理中效果显著。

追问回答：

Verifier 的评估要看 pairwise accuracy、top-1 rerank accuracy、hard negative accuracy、calibration 和下游提升。只看 verifier

12.16 如何训练代码推理模型

回答要点：

代码推理模型可以利用“生成代码 -> 执行测试 -> 读取反馈 -> 修复代码”的闭环训练。数据可以包括需求到代码、错误代码到修复、

重点指标：

我会区分 public tests 和 hidden tests。公开测试通过不等于泛化正确，所以要看 public-hidden gap、pass@k、repair success rate

12.17 Self-Debug 系统怎么做

回答要点：

系统先生成初始代码，在沙箱中运行测试，收集编译错误、运行异常和失败用例。模型根据反馈分析失败原因并生成补丁，再运行测试

工程边界：

self-debug 的风险是过拟合公开测试或在反馈噪声下反复改坏代码。必须有隐藏测试、静态扫描、沙箱和最大迭代预算。

12.18 如何评估 Reasoning Model

回答要点：

评估要覆盖最终答案、过程质量、泛化能力、鲁棒性和推理成本。数学可以看 GSM、MATH 类 benchmark 和变体题；代码可以看 Pass@k，通过 self-debug 提升。还要做污染检测，报告采样数、token、verifier 和工具调用预算。过程评估要看步骤正确率、第一处错误位置和推理链

可信报告应该包含：

1. 评测集版本和污染检查。
2. prompt、采样参数、候选数和随机种子。
3. final answer、process、variant 和 slice。
4. paired lift 和 confidence interval。
5. token、latency、verifier、tool 和 cost per correct。
6. failure cases 和安全门禁。

12.19 为什么不能只看 Benchmark 分数

回答要点：

Benchmark 分数可能受到数据污染、prompt、推理预算、采样次数、verifier 和评估器偏差影响。一个模型分数高，可能是因为见过，或者 time compute。Reasoning 还要看过程是否正确、变体题是否泛化、成本是否可接受、失败案例是什么，以及是否存在伪推理和过度自信

面试高分补充：

我会要求同预算比较，并报告 cost-normalized accuracy。否则一个模型用 1 次 greedy，另一个模型用 64 次采样加 verifier，直接

12.20 Reasoning Model 有哪些安全风险

回答要点：

主要风险包括伪推理、过度自信、长链条错误传播、reward hacking、工具误用、hidden CoT 暴露、高风险场景误导、过度拒答，以及

指标补充：

我会同时看 unsafe compliance rate、over-refusal rate、tool misuse rate、hidden CoT exposure rate、high-risk review coverage 和 severity-weighted risk。安全不是拒答率越高越好，而是正确拦截高风险，同时保持良性任务可用。

12.21 为什么完整 CoT 不一定应该展示给用户

回答要点：

完整 CoT 可能泄露系统策略、隐私推断、工具权限或安全边界，也可能把不忠实的中间想法包装成解释。产品中更合理的做法通常是

追问回答：

不展示完整 CoT 不等于不可解释。可以展示引用、证据、关键假设、工具结果、置信边界和最终检查清单，让用户能复核结论，而不

12.22 设计一个 Reasoning 系统

回答框架：

我会把系统拆成 router、generator、verifier、search controller、tool executor、budget manager、aggregator、safety gate 和

关键补充：

系统要有降级策略。简单问题不需要 search；高风险问题需要人工审核或更强验证；工具调用必须做权限控制和沙箱隔离；评估时要

12.23 面试中的高分表达模板

通用模板：

这个问题我会从五层回答：第一，它解决什么问题；第二，为什么这种方法有效；第三，核心公式或指标是什么；第四，工程上如何实

用于 CoT：

CoT 解决的是复杂问题一次性生成困难的问题。它通过显式中间步骤降低推理难度。工程上可以用 zero-shot、few-shot、SFT 或 reasoning 数据训练。但它可能产生伪推理，所以需要过程评估、verifier 或工具验证。

用于 verifier：

Verifier 解决的是候选选择和错误识别问题。它可以用于 reranking、search 剪枝和 reward。工程上要区分 outcome verifier、pro

用于 test-time compute：

Test-time compute 解决的是单次采样不稳定的问题。它通过多采样、搜索、验证和工具反馈提升质量。工程上要做 adaptive compute

用于安全：

Reasoning safety 解决的是更强规划、工具调用和高风险建议带来的边界问题。工程上要把 verifier、权限、沙箱、人工审核、日志 refusal 和良性任务可用性。

12.24 最小可运行 reasoning 面试复盘 demo

下面的 0 依赖 demo 用一轮 toy mock interview 记录检查：是否覆盖了 reasoning 主题、公式、demo、风险和 trade-off。它适合在面试前自查 “我是不是只会背概念，而不能讲公式和项目”。

```
REQUIRED_TOPICS = {
    "reasoning_model", "cot", "self_consistency", "verifier",
    "process_supervision", "search", "test_time_compute",
    "math_training", "code_feedback", "evaluation", "safety", "system_design"
}
REQUIRED_FORMULAS = {
    "A_ans", "A_step", "pass_at_k", "G_ver", "G_proc", "C_ttc", "G_safe"
}
REQUIRED_DEMOS = {
    "cot_audit", "sc_vote", "verifier_rerank", "process_audit",
    "search_audit", "ttc_router", "eval_report", "safety_gate"
}
REQUIRED_RISKS = {
    "pseudo_reasoning", "overconfidence", "contamination",
```

```

    "tool_misuse", "over_refusal"
}
REQUIRED_TRADEOFFS = {
    "accuracy_cost", "latency_quality", "verifier_bias", "safety_helpfulness"
}

ANSWERS = [
    dict(
        q="q1_reasoning_model",
        topics={"reasoning_model", "test_time_compute"},
        formulas={"C_ttc"},
        demos={"ttc_router"},
        risks={"overconfidence"},
        tradeoffs={"accuracy_cost"},
    ),
    dict(
        q="q2_cot",
        topics={"cot"},
        formulas={"A_step"},
        demos={"cot_audit"},
        risks={"pseudo_reasoning"},
        tradeoffs={"accuracy_cost"},
    ),
    dict(
        q="q3_self_consistency",
        topics={"self_consistency"},
        formulas={"pass_at_k"},
        demos={"sc_vote"},
        risks=set(),
        tradeoffs={"accuracy_cost"},
    ),
    dict(
        q="q4_verifier",
        topics={"verifier"},
        formulas={"G_ver"},
        demos={"verifier_rerank"},
        risks={"verifier_bias"},
        tradeoffs={"verifier_bias"},
    ),
    dict(
        q="q5_process",
        topics={"process_supervision"},
        formulas=set(),
        demos={"process_audit"},
        risks=set(),
        tradeoffs={"verifier_bias"},
    ),
    dict(
        q="q6_search_ttc",
        topics={"search", "test_time_compute"},
        formulas={"C_ttc"},
        demos={"search_audit", "ttc_router"},
        risks=set(),
        tradeoffs={"latency_quality"},
    ),
]

```

```

dict(
    q="q7_eval",
    topics={"evaluation"},
    formulas={"A_ans"},
    demos={"eval_report"},
    risks={"contamination"},
    tradeoffs={"accuracy_cost"},
),
dict(
    q="q8_safety",
    topics={"safety"},
    formulas={"G_safe"},
    demos=set(),
    risks={"tool_misuse", "over_refusal"},
    tradeoffs={"safety_helpfulness"},
),
]

```

```

def union(field):
    out = set()
    for item in ANSWERS:
        out |= item[field]
    return out

```

```

def coverage(observed, required):
    return round(len(observed & required) / len(required), 3)

```

```

observed = {
    "topics": union("topics"),
    "formulas": union("formulas"),
    "demos": union("demos"),
    "risks": union("risks"),
    "tradeoffs": union("tradeoffs"),
}
summary = {
    "topic_coverage": coverage(observed["topics"], REQUIRED_TOPICS),
    "formula_coverage": coverage(observed["formulas"], REQUIRED_FORMULAS),
    "demo_coverage": coverage(observed["demos"], REQUIRED_DEMOS),
    "risk_coverage": coverage(observed["risks"], REQUIRED_RISKS),
    "tradeoff_coverage": coverage(observed["tradeoffs"], REQUIRED_TRADEOFFS),
}
missing = {
    "topics": sorted(REQUIRED_TOPICS - observed["topics"]),
    "formulas": sorted(REQUIRED_FORMULAS - observed["formulas"]),
    "demos": sorted(REQUIRED_DEMOS - observed["demos"]),
    "risks": sorted(REQUIRED_RISKS - observed["risks"]),
    "tradeoffs": sorted(REQUIRED_TRADEOFFS - observed["tradeoffs"]),
}
question_scores = {}
for item in ANSWERS:
    score = 0
    score += 2 if item["topics"] else 0

```

```

score += 1 if item["formulas"] else 0
score += 1 if item["demos"] else 0
score += 1 if item["risks"] else 0
score += 1 if item["tradeoffs"] else 0
question_scores[item["q"]] = round(score / 6, 3)

weak_questions = [q for q, score in question_scores.items() if score < 0.75]
revision_plan = {
    "math_training": " 补一道数学训练题，讲清数据 schema、污染检测、课程学习和 verifier 过滤。",
    "code_feedback": " 补一道代码执行反馈题，讲清 public-hidden gap、sandbox 和 repair success。",
    "process_formula": " 把 process supervision 回答补上 A_step 或 G_proc。",
    "safety_demo": " 给 safety 回答绑定一个 toy safety gate demo。",
}
gates = {
    "topic_ok": summary["topic_coverage"] >= 0.95,
    "formula_ok": summary["formula_coverage"] >= 0.85,
    "demo_ok": summary["demo_coverage"] >= 0.85,
    "risk_ok": summary["risk_coverage"] >= 0.80,
    "tradeoff_ok": summary["tradeoff_coverage"] >= 0.90,
    "weak_question_ok": len(weak_questions) == 0,
}

print(f"summary={summary}")
print(f"missing={missing}")
print(f"question_scores={question_scores}")
print(f"weak_questions={weak_questions}")
print(f"revision_plan={revision_plan}")
print(f"gates={gates}")
print(f"interview_ready={all(gates.values())}")

```

预期输出：

```

summary={'topic_coverage': 0.75, 'formula_coverage': 0.857, 'demo_coverage': 0.875, 'risk_coverage': 1.0, 'tradeoff_cov
missing={'topics': ['code_feedback', 'math_training', 'system_design'], 'formulas': ['G_proc'], 'demos': ['safety_gate
question_scores={'q1_reasoning_model': 1.0, 'q2_cot': 1.0, 'q3_self_consistency': 0.833, 'q4_verifier': 1.0, 'q5_proces
weak_questions=['q5_process']}
revision_plan={'math_training': '补一道数学训练题，讲清数据 schema、污染检测、课程学习和 verifier 过滤。', 'code_feedb
hidden gap、sandbox 和 repair success。', 'process_formula': '把 process supervision 回答补上 A_step 或 G_proc。', 'safet
gates={'topic_ok': False, 'formula_ok': True, 'demo_ok': True, 'risk_ok': True, 'tradeoff_ok': True, 'weak_question_ok':
interview_ready=False

```

这里 interview_ready=False 是预期结果：demo 故意留下数学训练、代码反馈、系统设计和 process supervision 公式缺口，用来说明复盘脚本应该暴露短板，而不是给自己一个漂亮分数。

12.25 小练习

1. 从本章任选 10 个问题，每题用“目标 -> 机制 -> 公式 -> 工程 -> 评估 -> 边界”六段式回答。
2. 写出 A_ans、A_step、pass@k、L_pair、C_i 和 G_safe 的公式，并说明每个符号含义。
3. 给一轮 reasoning mock interview 设计评分表，字段包含 topic coverage、formula coverage、demo coverage、risk coverage、trade-off coverage 和 weak questions。
4. 选择你最薄弱的一题，补一个 0 依赖 Python toy demo 或一个项目审计表。
5. 用 5 分钟讲清楚“如何设计一个可上线的 reasoning 系统”，要求同时覆盖 router、generator、verifier、search、tool、budget、safety 和 logging。

12.26 本章小结

Reasoning 面试的核心不是背术语，而是能把方法、机制、公式、工程实现、评估和局限连起来。CoT 解决分步推理，self-consistency 解决单次采样不稳定，verifier 解决候选选择和错误识别，process supervision 解决中间步骤监督，search 和 test-time compute 解决复杂问题探索，数学和代码提供可验证训练场景，评估和安全决定这些能力能否可靠落地。

到这里，第十六册《Reasoning Model、长思维链与可验证推理》完成第二轮阶段性精修。本册主线是：从“模型如何一步步思考”到“如何训练、验证、搜索、评估、安全使用 reasoning 能力”，再到“如何在面试中把这些能力讲成可落地系统”。后续进入 Agent 与工具调用专题时，要把本册的 verifier、search、tool、safety gate 和 audit 思路继续迁移到 Agent 系统中。